

后端场景面试题速记通关版 _ 面试刷题 mianshiya.com

本资源来自面试鸭: <https://www.mianshiya.com>

推荐更多免费学编程资源:

1. 编程导航学习网站: [学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库: [开始刷题, 面试遇原题!](#)
3. 精选简历模板大全: [1 分钟搞定简历!](#)
4. AI 资源导航网站: [获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试: [随时随地提升面试能力](#)

可以用几行代码实现一个负载均衡器吗?

实现一个最简负载均衡器, 核心是选节点的策略。我们用 Java 写个轮询的例子, 10 行代码左右就能跑起来。

维护一个计数器, 每次有请求就取 `(i++) % servers.size()`, 把流量打到对应机器上。这就是**轮询**, 最简单的负载策略。

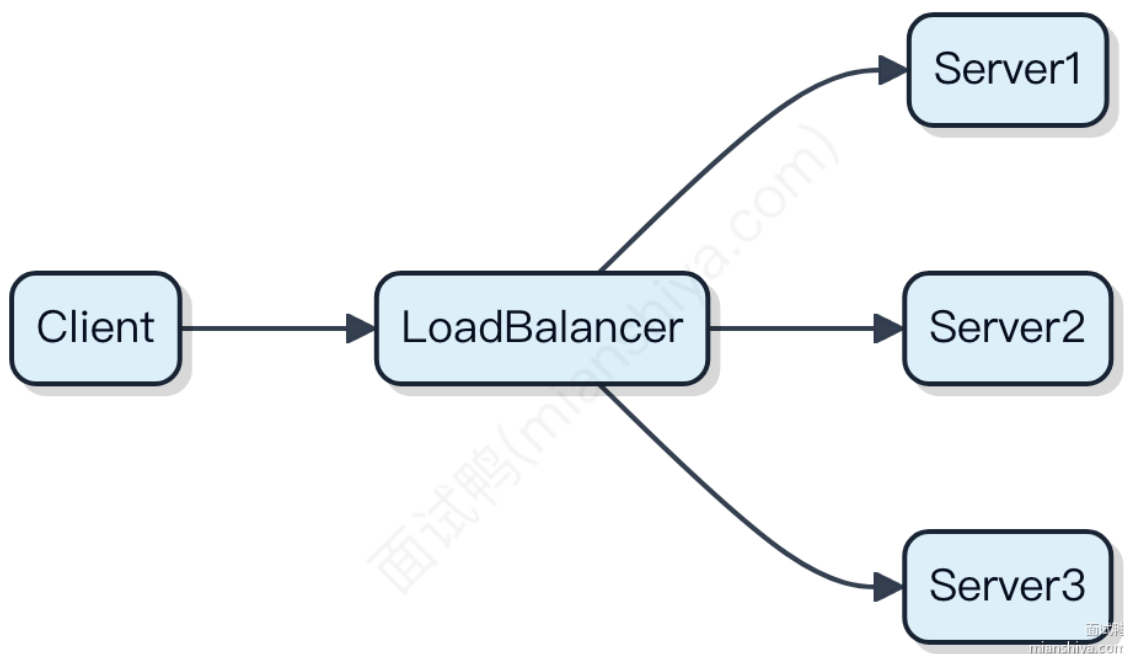
```
List<String> servers = Arrays.asList("192.168.0.1", "192.168.0.2", "192.168.0.3");
AtomicInteger idx = new AtomicInteger(0);

public String getNextServer() {
    int i = idx.getAndIncrement();
    return servers.get(i % servers.size());
}
```

如果要加权重, 可以按权重展开成列表, 比如 A:3, B:1 就生成 [A,A,A,B], 再轮询。这种叫**加权轮询**。

实际生产不会这么简单。Nginx 用的是平滑加权轮询, Spring Cloud LoadBalancer 提供了响应时间权重、随机、最少活跃等策略。Zookeeper 或 Nacos 下发节点列表, 配合健康检查自动剔除故障实例。

这类组件的关键是**无状态**, 能独立部署。像 Ribbon 这种客户端负载均衡, 直接在 JVM 里完成选择, 省去一次网络跳转。



现在需要你设计一个将已登录的用户踢下线的功能，如何实现？

用户踢下线本质是让某个用户的会话失效，关键在于**会话管理**和**实时通知**。

1) 集中式存储 session

别把登录状态放服务器内存，用 Redis 统一存 token 或 sessionId。每次请求都去查 Redis，这样你才能主动控制它的生命周期。

```
// 登录时写入
redis.set("login:token:" + token, userId, 30, TimeUnit.MINUTES);

// 踢人时直接删
redis.del("login:token:" + targetToken);
```

2) 前端轮询或 WebSocket 主动推送

光删 token 不够，客户端可能还在操作。要立刻断开，得通知前端。可以用 WebSocket 建立用户级通道。

```
// 踢人逻辑
stringRedisTemplate.convertAndSend("user-logout", userId + "");
// 前端收到后跳转登录页
```

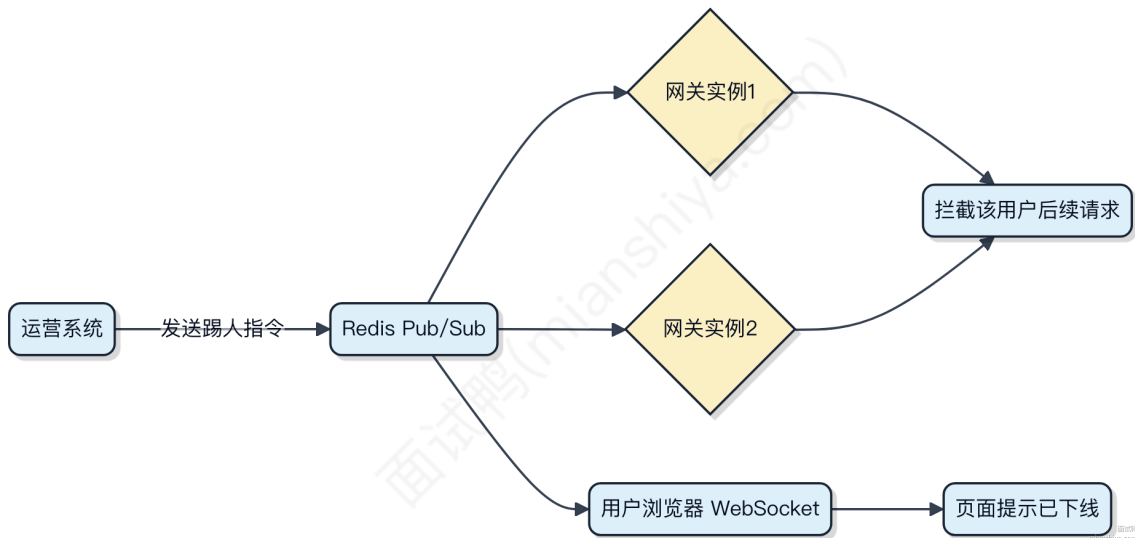
3) 网关层拦截续发请求

即使前端没及时退出，后续请求到网关（比如 Spring Cloud Gateway）时，先校验 token 是否已被注销。这块可以用全局过滤器做。

4) 考虑分布式场景

单机用内存 Map 记录状态行不通。集群环境下必须依赖外部存储，Redis 是最通用选择。ZooKeeper 也能做，但杀鸡用牛刀。

典型流程：运营后台触发踢人 → 后台发消息到 Redis Channel → 所有网关实例更新本地缓存 → 新请求被拦截 → WebSocket 推送下线指令。



注意粒度：按设备踢（多个登录点）还是按账号踢（全下线）。实现方式类似，只是 key 的维度不同。

不能使用大于小于号，如何判断两个正整数的大小？

这题其实考的是位运算和数学思维，面试官想看你能不能跳出常规的比较符。

1) 用减法判断符号位

两个正整数 a 和 b ，做 $a - b$ 。如果结果是正的，说明 $a > b$ ；负的则是 $a < b$ 。但不能直接用大于小于号，那怎么判断结果正负？看最高位（符号位）。Java 里可以转成 long 再右移 63 位，取符号位。

```
long diff = (long) a - b;
if (diff == 0) return "相等";
if ((diff >> 63) == 0) return "a 大于 b"; // 符号位为 0 是正数
else return "a 小于 b";
```

2) 逐位比较（从高位到低位）

把两个数转成二进制，从左往右找第一个不同的位。谁那位是 1，谁就大。这个适合在无法做算术运算的场景，比如某些硬件限制下。

```
for (int i = 31; i >= 0; i--) {
    int bitA = (a >> i) & 1;
    int bitB = (b >> i) & 1;
    if (bitA != bitB) {
        return bitA == 1 ? "a 大于 b" : "a 小于 b";
    }
}
```

3) 用除法特性

a/b 如果大于 1，说明 $a > b$ （前提是都是正整数且 $b \neq 0$ ）。但有精度问题，得配合整除判断，实用性不如前两种。

一般情况下，**减法 + 符号位检测**是最直接的替代方案。位扫描适合特定场景，比如你要在不支持算术运算的环境里搞事。面试写第一种就够了，第二种当加分项提一嘴。

ArrayList 里有 1 亿条数据，要怎么去重？

直接用 ArrayList 去重一亿条数据，性能基本就崩了。得换思路，不能硬搞。

核心是利用 **哈希结构** 的唯一性特性，把数据往 HashSet 或 LinkedHashSet 里塞，重复的自然被过滤掉。但一亿条数据内存占用不小，普通机器 JVM 堆设到 4~8G 都不一定扛得住，得先评估单机内存是否够用。

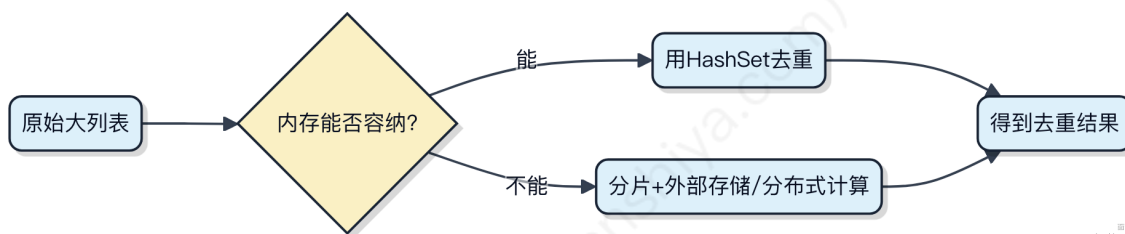
如果内存吃紧，优先考虑分批处理：

```
Set<String> set = new HashSet<>();
for (String item : hugeList) {
    set.add(item);
}
// 去重后转回 list
List<String> uniqueList = new ArrayList<>(set);
```

要是数据本身有业务顺序要求，比如要保留首次出现的顺序，那就用 LinkedHashSet，它维护了一个双向链表，能保序。

真到数据量大到单机搞不定，就得上外部工具。比如用 Spark 或 Flink 做分布式去重，或者先写进 HBase/Redis，用外部存储的唯一键能力来筛。

还有一种情况，数据全是整数且范围有限（比如用户 ID 在 10 亿内），可以用 RoaringBitmap，几亿数据压缩后可能只占几十 MB，效率极高。



如何设计一个点赞系统？

设计点赞系统，关键是在数据一致性、性能和扩展性之间找平衡。用户点赞这种操作并发高、写多读少，直接怼数据库肯定扛不住。

一般用分层架构来消化压力。前端写请求先打到消息队列，比如 Kafka 或 RocketMQ，把瞬时流量削峰填谷。这样后端服务不会被突发的点赞洪流冲垮，系统更稳。

数据存储通常分两层。热数据用 Redis 存，比如用 Set 或 Sorted Set 记录用户对某内容的点赞状态，查得快、写得也快。Redis 里可以按内容 ID 聚合点赞用户，也可以加个计数器用 INCR 维护总数。定时任务再把 Redis 的变更异步落库，MySQL 或 TiDB 都行，保证最终一致。

1) 用户点赞先写消息队列，响应立刻返回 2) 消费者从队列拉数据，更新 Redis 中的点赞集合和计数 3) 后台任务周期性将 Redis 数据合并到数据库



有个细节是去重。同一个用户不能重复点，Redis 的 Set 天然支持唯一性，SADD 操作直接搞定。取消点赞用 SREM 就行，干净利落。

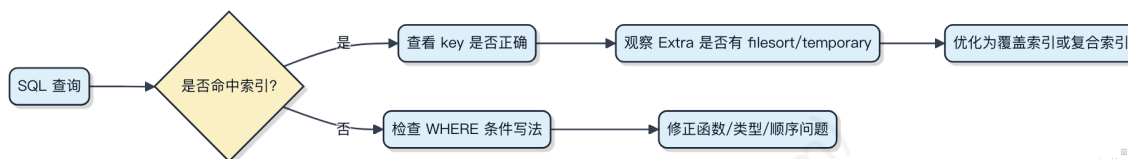
如果内容热度差异大，热门内容的计数器可能成为热点 key。可以分片计数，比如用 `user_id % 16` 分 16 个 key，查的时候再汇总。这种设计在微博、小红书这类产品里都验证过，能撑住千万级互动。

MySQL 中使用索引一定有效吗？如何排查索引效果？

索引不是银弹，用不对压根不生效。常见原因比如写法导致无法命中，像在索引字段上做函数操作 `WHERE YEAR(create_time) = 2023`，这直接让 B+ 树失效。或者隐式类型转换，比如索引字段是 `varchar`，但用数字查询没加引号，MySQL 会全表扫描。

联合索引要遵守最左前缀原则，中间跳字段或者顺序颠倒都可能让索引失效。比如索引 `(a,b,c)`，查 `b=1 and c=2` 就用不上。还有 `like` 以 `%` 开头，`LIKE '%abc'` 这种也无法利用索引查找。

想确认索引有没有起作用，核心是看执行计划。用 `EXPLAIN` 分析 SQL，重点关注 `type`、`key` 和 `Extra` 字段。`type` 至少到 `range` 级别才算有效使用，最好能达到 `ref` 或 `const`。`key` 显示实际走的索引，如果是 `NULL` 就说明没用上。`Extra` 里出现 `Using index` 表示覆盖索引命中，性能好；要是出现 `Using filesort` 或 `Using temporary`，那多半有优化空间。



线上排查可以用 `slow query log` 配合 `pt-query-digest` 分析慢查询，定位哪些 SQL 没走索引。有时候加了索引但数据量小，优化器觉得全表扫更快，也会放弃索引，这种情况一般不用管。

MySQL 中如何进行 SQL 调优？

SQL 调优不是上来就改语句，先知道慢在哪。第一步永远是开慢查询日志，把执行时间超过 1 秒的 SQL 捞出来，这个阈值可以根据业务调整，比如线上核心接口可能得设成 100ms。

找到慢 SQL 后，用 `explain` 看执行计划。重点关注 `type` 字段，如果是 `ALL` 就是全表扫描，`index` 是全索引扫描，最好能做到 `range` 或 `ref`。`Extra` 里出现 `Using filesort` 或 `Using temporary` 基本就有性能问题，说明排序或分组没走好索引。

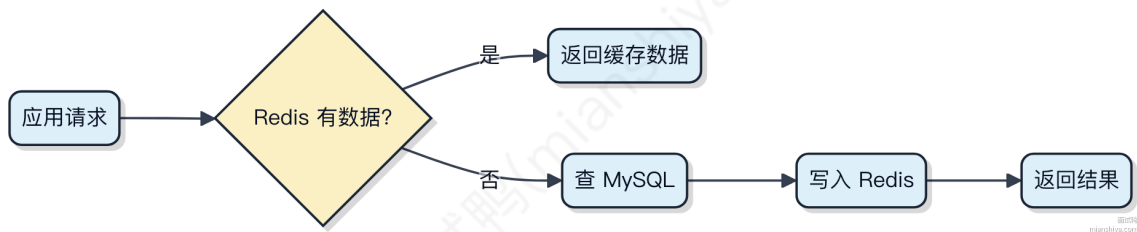
索引设计要结合查询条件。比如经常按 `user_id` 和 `create_time` 查订单，那 `(user_id, create_time)` 的联合索引就很合适。记住最左前缀原则，查询时如果跳过 `user_id` 直接查 `create_time`，这个联合索引就用不上。

```
-- 比如这张订单表
SELECT * FROM orders WHERE user_id = 123 AND create_time > '2023-01-01';
-- 建立复合索引
CREATE INDEX idx_user_time ON orders(user_id, create_time);
```

别忘了索引也有成本，写多读少的表加太多索引会拖慢插入更新。大字段比如 text 类型别放索引里，可以用前缀索引或者单独拎出去。

还有些场景需要改架构。单表数据上亿就得考虑分库分表，ShardingSphere 或者 MyCat 可以帮忙。但分了之后跨库 join 和事务就是难题，得从业务设计上规避。

最后提一嘴缓存。热点数据像商品信息，可以放 Redis，避免频繁打到数据库。但要注意缓存穿透、击穿这些坑，合理设置过期时间和空值。



MySQL 中如何解决深度分页的问题?

深度分页在 MySQL 里是个经典性能问题，尤其是用 LIMIT offset, size 这种方式，当 offset 很大时，比如 LIMIT 1000000, 20，数据库得先扫描前 100 万条数据，再取 20 条，查询速度直接崩掉。

根本原因在于这种分页方式依赖**物理偏移**，跳过的行越多，执行越慢。解决思路是避免使用大 offset，转而在**基于游标的分页**，也就是利用索引字段（通常是主键或有序列）做条件过滤。

比如你查订单列表，按创建时间排序，就可以记录上一页最后一条的时间戳和 ID，在下一页查询时加 where 条件：

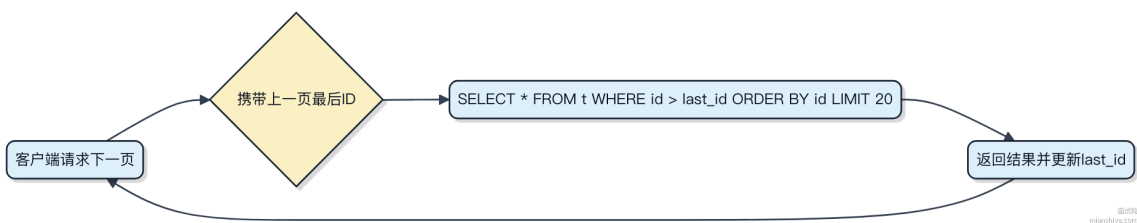
```
SELECT * FROM orders
WHERE id > 1000000
ORDER BY id
LIMIT 20;
```

这种方式能走主键索引，查询效率稳定在 $O(\log n)$ ，完全压根不经过前面那些无效扫描。

如果业务必须支持“跳转到第 N 页”，那就搞不定高性能了，只能提前算好每个分页的边界值，比如用延迟关联或者维护一个分页索引表，但成本高，一般推荐前端改成“加载更多”模式。

还有一个办法是结合业务场景用缓存，比如把热点数据预加载到 Redis，用 ZSET 按 score 排序后分页取，像 Feed 流系统里常见。

- 1) 核心是避免大 offset 的全范围扫描
- 2) 优先用 where + 索引字段替代 limit offset
- 3) 配合缓存或物化视图提升极端场景体验

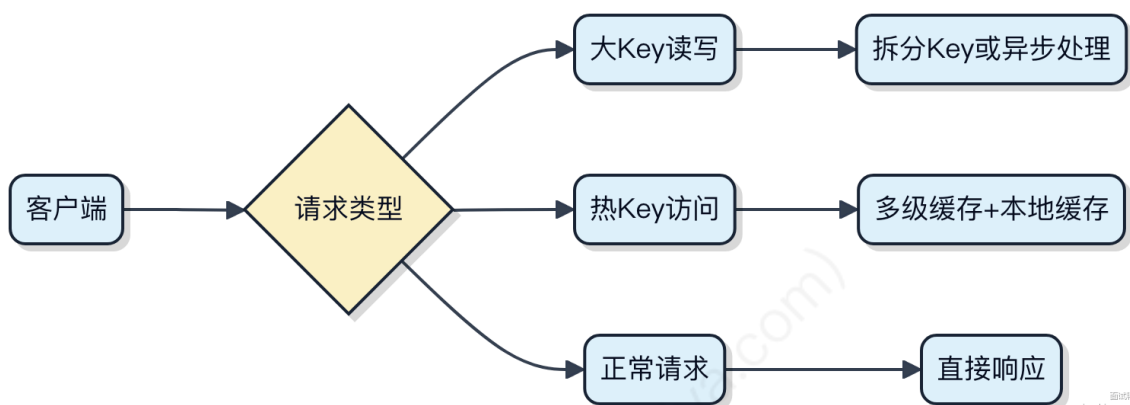


线上发现 Redis 机器爆了，如何优化？

Redis 机器扛不住了，先别急着加机器，得搞清楚到底是哪块出了问题。一般来说就是内存、CPU、网络或者连接数撑满了。

- 1) 先看是不是大 Key 或热 Key 搞的鬼。比如一个 Key 存了几百万条数据，查一次就卡住，删除也慢。用 `redis-cli --bigkeys` 扫一下，或者用 Redis 自带的监控工具看采样。要是发现某个 Key 特别大，考虑拆分，比如把一个大 Hash 拆成多个小 Hash，或者用本地缓存扛一部分。
- 2) 热点数据集中访问，容易压垮单个实例。像秒杀场景，一个商品信息被疯狂查。可以考虑多级缓存，比如加一层 Caffeine 做本地缓存，但要注意缓存一致性。或者用 Redis 分片，把热点打散到不同节点。
- 3) 持久化策略也可能背锅。RDB 快照如果数据量大，fork 子进程会卡主线程；AOF 日志写得频繁，磁盘 IO 上去也会拖慢响应。一般建议 RDB 和 AOF 结合，AOF 配置为每秒刷盘（`appendfsync everysec`），平衡性能和安全。
- 4) 连接数太多？可能是客户端没用连接池，或者连接泄漏。Spring Boot 里用 Lettuce 就比 Jedis 更省连接，因为它是基于 Netty 的长连接模型，压根不经过每次建连的开销。

最后才是横向扩容。Redis Cluster 能自动分片，但运维复杂度上去了。也可以考虑云厂商的 Tair 或者阿里云 Redis 版，自带热点探测和大 Key 分析功能，省点心。

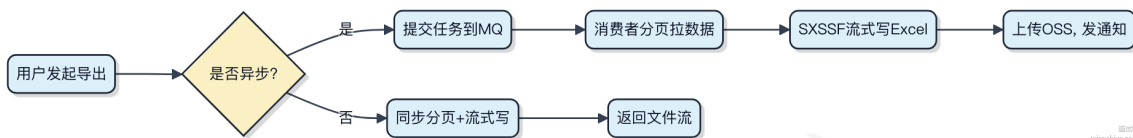


项目上有个导出 excel 场景发现很慢，怎么优化？

导出慢通常卡在数据查询和文件生成两个环节，得拆开看。

- 1) 先说数据查询。一次性捞几万行数据，数据库和应用服务器内存都扛不住。解决方案是改成分页查询，但别用 `OFFSET/LIMIT`，深分页效率太差。换成游标分页，比如用主键 ID 作为游标，每次查下一批时从上次结束的位置继续，数据库压力小很多。
- 2) 文件生成阶段，传统 POI 同步写入会把整个表格撑在内存里，数据量大了直接 OOM。这时候得切到 **SXSSFWorkbook**，它背后用了临时文件做缓冲，只保留固定数量的行在内存中，能处理几十万行不崩。更进一步，可以边查数据库边写文件，流式处理，整个过程内存恒定。
- 3) 最后是体验优化。异步导出+消息通知，用户提交任务后就释放连接，后台用定时任务或消息队列（比如 RocketMQ）驱动执行，完成后再发邮件或站内信。这样前端不卡，系统也能错峰处理。

如果并发导出多，加个限流，用 Redis 统计任务状态，避免同时跑太多挤占资源。



项目上需要导入一个几百万数据 excel 文件到数据库中，有哪些注意点？

处理几百万行 Excel 文件导入，本质是解决 **大文件解析** 和 **海量数据写入** 两个难题。直接一次性读取、加载到内存，JVM 压根扛不住，容易 OOM。

1) 必须用流式解析，不能全量加载

Apache POI 的 XSSF 是基于 DOM 的，百万数据直接炸。要用 **SXSSF** 或更推荐 **EasyExcel**，它们底层是 SAX 模式，边读边处理，内存只保留固定行数（比如默认 100 行），老数据处理完就 flush 掉。

2) 数据库写入要分批，别单条 insert

一条条插，网络来回几百次，数据库也扛不住。得用 `INSERT INTO ... VALUES (...), (...), (...)` 批量插入，每批控制在 500~1000 条。结合事务，每批提交一次，避免事务太大锁表。

3) 考虑异步 + 队列削峰

前端上传后立即返回“已接收”，后台用线程池或消息队列（比如 RabbitMQ、Kafka）逐步消费解析和入库，防止瞬时资源打满。

4) 加个进度反馈和错误回滚

导入过程长，用户不知道状态会焦虑。可以存 Redis 记录当前行数、成功/失败数。出错时最好能定位到哪一行有问题，记录日志，支持重试或跳过。

```
// EasyExcel 核心写法示意
EasyExcel.read(file, DataModel.class, new AnalysisEventListener<DataModel>() {
    private List<DataModel> buffer = new ArrayList<>();

    public void invoke(DataModel data, AnalysisContext context) {
        buffer.add(data);
        if (buffer.size() >= 500) {
            saveToDbInBatch(buffer);
            buffer.clear();
        }
    }

    public void doAfterAllAnalysed(AnalysisContext context) {
        if (!buffer.isEmpty()) {
            saveToDbInBatch(buffer);
        }
    }
}).sheet().doRead();
```



如果一笔订单，用户在微信和支付宝同时支付，会怎么样？

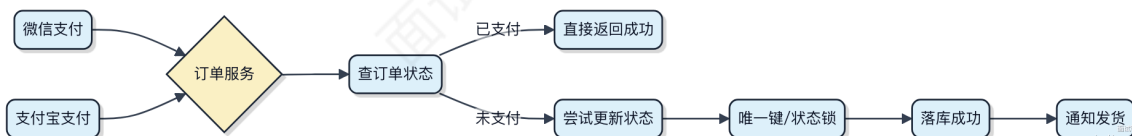
订单系统必须保证一笔订单只能成功支付一次，不管用户从微信、支付宝还是其他渠道发起。这其实是分布式场景下的**幂等性**问题。

- 1) 订单服务在收到支付请求时，得先查订单状态。如果已经支付成功，直接返回成功结果，压根不经过后续流程。
- 2) 关键在支付单的设计。每笔支付请求都会生成唯一的业务流水号，比如用订单ID + 渠道做唯一索引。MySQL 唯一约束能防止重复记录插入。
- 3) 状态机控制也很重要。订单状态从“待支付”到“已支付”是单向的。哪怕微信回调一次、支付宝回调一次，第二次更新会因为状态不符或唯一键冲突被拒绝。

```

// 伪代码示意
public boolean payCallback(PayNotify notify) {
    Order order = orderService.getById(notify.getOrderId());
    if (order.isPaid()) {
        return true; // 已支付，直接成功
    }
    // 尝试更新状态，加条件：当前状态必须是待支付
    int updated = orderMapper.updateStatus(
        order.getId(), OrderStatus.PAID, OrderStatus.WAIT_PAY);
    if (updated == 0) {
        return false; // 更新失败，说明已被处理
    }
    return true;
}
  
```

典型做法是把“防重”放在最前面，像秒杀一样，Redis 可以缓存订单支付结果，减少数据库压力。RocketMQ 这类中间件也能通过消息去重机制辅助。



一笔订单，在取消的那一刻用户刚好付款了，怎么办？

订单取消时用户刚好付款，本质是**状态冲突**问题，常见于高并发场景。关键不是时间点，而是系统如何保证最终一致性。

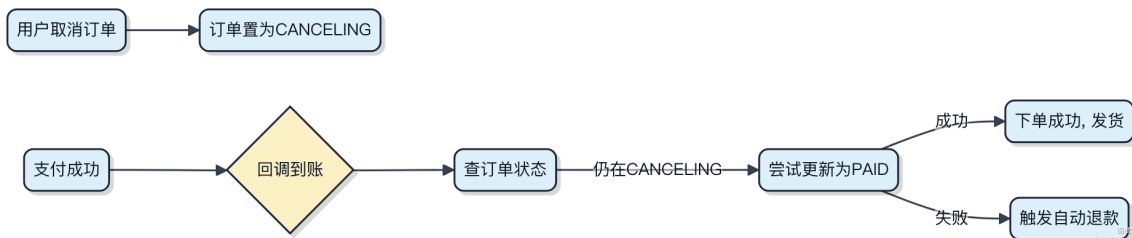
- 1) 核心思路是“以支付为准”，所有状态变更都要走可靠事件驱动。比如用户点击取消，订单服务只是发起取消流程，进入“取消中”状态，但不直接关单。
- 2) 支付成功会发一个全局唯一的支付回调，这个回调必须能穿透到订单系统。收到回调后，先查订单当前状态。如果发现正在取消，得判断取消请求是否在支付消息之前到达——靠分布式锁或版本号控制。
- 3) 一般做法是在订单表加个状态机字段，比如 `status` 和 `version`。支付回调处理时用 SQL：

```
UPDATE orders SET status = 'PAID', version = version + 1
WHERE order_id = ? AND status IN ('CREATED', 'CANCELING') AND version = ?
```

更新成功说明支付胜出；如果失败，再走补偿逻辑，比如通知用户“已退款”或人工介入。

- 4) 实际项目里，像支付宝、微信支付都要求商户做对账，每天跑一次核对。哪怕当时没处理好，也能通过**对账文件**发现异常订单，后台补单或退款。

整个过程不能靠人肉判断，得靠幂等设计和异步补偿。TCC、Saga 这类模式在这里也适用，但大多数公司用“**最终一致+人工兜底**”更实际。



从网关再到各个后端服务，如何设置 RPC 的超时时间，要考虑哪些问题？

RPC 超时设置不是拍脑袋定的，得一层层往下推演。网关进来的请求，每个环节都要有超时控制，否则一个慢调用就能把线程池拖死。

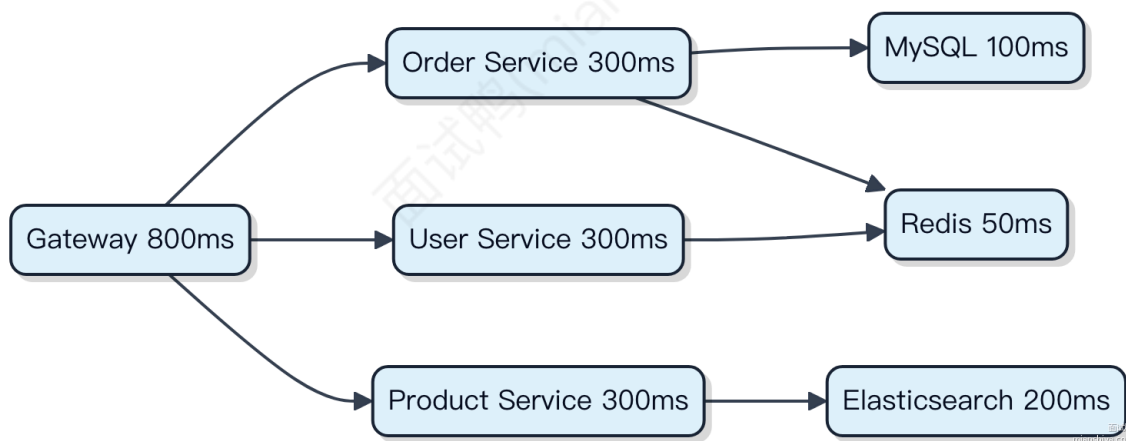
- 1) **链路传导**：网关收到请求后，会调用订单、用户等下游服务。假设网关总超时设了 800ms，那它调用的每个 RPC 超时加起来得小于这个数，不然没意义。比如你设了三个调用各 500ms，并发走也有可能整体超时翻倍。
- 2) **逐层收敛**：一般建议越靠近底层，超时时间越短。比如缓存层 Redis 调用通常不超过 50ms，数据库同理。如果一个 DB 查询要 200ms，那大概率是慢 SQL 该优化了，而不是延长超时扛住。
- 3) **隔离与重试**：不同服务要有独立的超时配置，不能共用一套。像支付这种核心链路可以单独设更严苛的超时。重试策略也要配合，比如最多重试 2 次，每次超时 100ms，避免雪崩。
- 4) **默认值陷阱**：很多框架默认超时是 -1（无限等待）或者 30 秒，上线就炸。必须显式配置，Spring Cloud Alibaba 的 `feign.client.config.default.connectTimeout` 和 `readTimeout` 就得写清楚。

代码上以 Feign 为例：

```
@FeignClient(name = "order-service", configuration = ClientConfig.class)
public interface OrderClient {
    @GetMapping("/orders/{id}")
    Order getOrder(@PathVariable("id") String id);
}
```

```
// 配置类
public class ClientConfig {
    @Bean
    public Request.Options options() {
        return new Request.Options(100, 800); // connect 100ms, read 800ms
    }
}
```

整个调用链路就像漏斗，上游时间宽一点，下游迅速收窄。监控也得跟上，用 SkyWalking 或 Prometheus 看 P99 延迟，动态调整才靠谱。



线上数据库连接池爆满问题排查

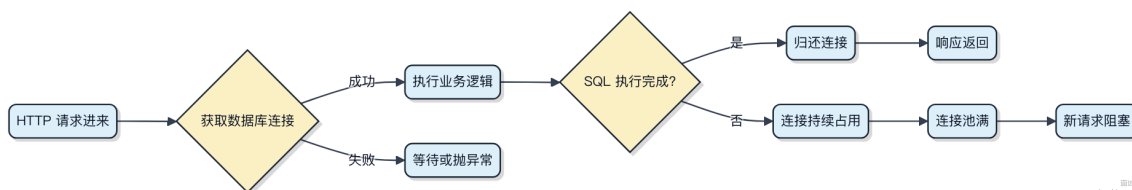
数据库连接池爆满，说白了就是应用拿不到连接了。一般会表现为接口大面积超时，监控里看到数据库等待连接的队列飙升，比如 Druid 的 activeCount 接近 maxActive。

问题得从上下两层看。上层是应用代码，常见的是连接没及时归还。比如用了 try-with-resources 但忘了写 close，或者事务注解范围太大，一个方法里塞了几十秒的业务逻辑，**连接**被长时间占用。还有种情况是异步线程里开了事务，主线程结束了连接还在。

下层是数据库本身。如果 DB 出现慢查询，哪怕应用代码没问题，连接也会卡在执行 SQL 阶段。这时候看监控会发现数据库 CPU 打满，或者有大量 state 为 Sending data 的会话。可以查 information_schema.processlist 找出慢 SQL。

排查步骤：

1) 先看应用日志和监控，确认是连接池耗尽还是数据库处理不过来。2) 如果是连接池问题，用 Arthas 这类工具 dump 线程栈，找 WAITING 在 DataSource.getConnection 的线程，看它之前在哪调用的。3) 结合慢 SQL 日志，看是否有某个 SQL 耗时突增。4) 检查代码里事务的传播行为和作用范围，@Transactional 别乱加。



每次进入订单列表页面都会触发全量同步？

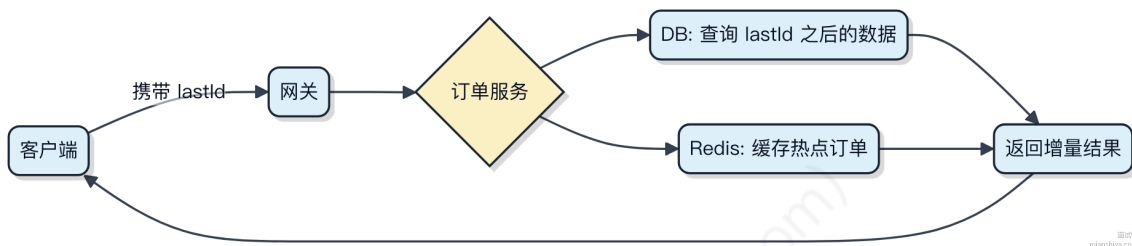
订单列表每次进入都全量同步，这设计大概率有问题，得先搞清楚“同步”的到底是啥。

如果是指每次打开页面都从数据库把所有订单查一遍，那其实不算“同步”，就是普通的查询。这种场景下，全量拉取在数据量小的时候还能扛住，但订单量一上来，比如超过 10 万条，查询慢、内存压力大，接口响应可能直接崩掉。正确的做法是分页查询，加上状态、时间等条件过滤，只拿用户关心的数据。

真正的“同步”一般指系统间数据一致性，比如客户端本地缓存和服务器数据对齐。这时候全量同步就更危险了，不仅耗流量，还可能压垮服务端。像美团或者淘宝的订单列表，都是靠增量同步 + 长轮询或 WebSocket 推送更新，客户端通过 lastId 或时间戳拉取新增或变更的订单，老数据本地缓存。

```
// 客户端带 lastTimestamp 请求
@GetMapping("/orders")
public List<Order> getOrders(@RequestParam Long lastTimestamp) {
    return orderService.getNewOrdersAfter(lastTimestamp);
}
```

- 1) 全量同步只适合首次加载或数据修复场景
- 2) 日常请求必须走增量机制，配合合理的缓存策略
- 3) 服务端要有防刷机制，避免恶意请求搞垮数据库



JDK 序列化问题排查

排查 JDK 序列化问题，关键得看清楚整个链路里对象是怎么被写出去又读回来的。很多问题其实出在序列化协议的隐式规则上。

- 1) 首先确认类是否实现了 `Serializable` 接口，并且最好显式定义 `serialVersionUID`。不定义的话 JVM 会根据类结构自动生成，一旦类有改动，前后不一致直接抛 `InvalidClassException`。生产环境建议固定这个值，避免因类变更导致反序列化失败。
- 2) 注意**非瞬态字段必须可序列化**。如果某个字段是自定义类型且没实现 `Serializable`，序列化时就会报 `NotSerializableException`。排查时重点看堆栈里的字段名，快速定位“罪魁祸首”。
- 3) 静态字段和 `transient` 字段不会被序列化。这是设计使然，但如果业务逻辑依赖这些值恢复状态，就会出问题。比如缓存状态写在 `static` 里，反序列化后为空，数据丢了。
- 4) 父类没有实现 `Serializable` 接口时，子类序列化只会保存子类字段，父类字段在反序列化后回到默认值（0、null 等）。这种情况可以用 `writeObject` 和 `readObject` 手动控制流程，像 `ArrayList` 就这么干过。

```
private void writeObject(ObjectOutputStream out) throws IOException {
    out.defaultWriteObject();
}
```

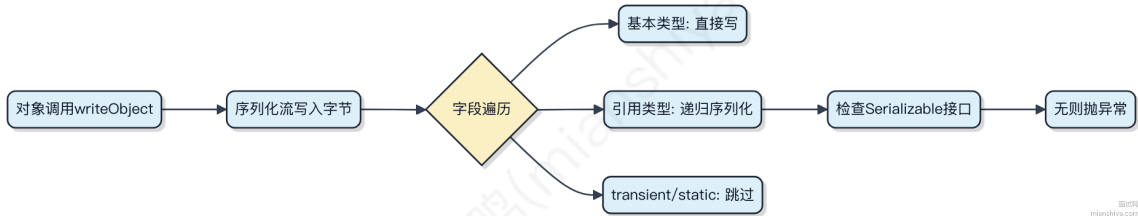
```

    out.writeInt(size);
}

```

5) 不同 JVM 版本间序列化格式可能有差异，尤其是涉及内部类、枚举或 lambda 表达式时。跨版本迁移要小心，建议用 JSON 或 Protobuf 这类稳定协议做持久化。

调用流程示意



Spring Cloud Gateway 500 问题排查

500 错误在 Spring Cloud Gateway 里，基本就是**内部异常被触发**，网关本身没处理好请求。别急着看下游服务，先确认是网关层崩了还是转发后出的问题。

第一反应应该是开 debug 日志。把 `org.springframework.cloud.gateway` 包的日志级别调到 DEBUG 或 TRACE，重点看 `FilteringWebHandler`、`RoutePredicateHandlerMapping` 这些组件的执行路径。有没有过滤器抛异常？路由匹配失败？断言不通过？

常见坑点有几个。自定义 `GlobalFilter` 写得不稳，比如在 pre 阶段做了空指针操作，直接炸在网关层。还有就是 Hystrix 或 Resilience4j 熔断后 fallback 处理不当，返回了个 null 或非法响应体，也会导致 500。另外跨域配置如果手动写了 filter，CORS 头处理出错也可能返回服务器错误。

代码上注意这类写法：

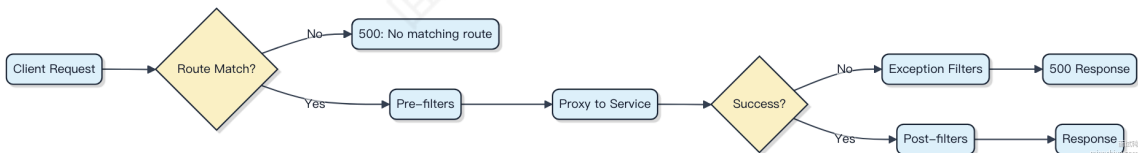
```

exchange.getResponse().setStatusCode(HttpStatus.INTERNAL_SERVER_ERROR);
return exchange.getResponse().setComplete();

```

这种手动设 500 的地方要排查是否条件判断过宽。

另一个关键是分清责任边界。用 curl 或 postman 直接打网关，如果还 500，那就是网关自己有问题。如果单独调下游是好的，走网关才 500，大概率是 predicates 或 filters 配置错了。比如加了 `StripPrefix` 导致路径 404 转成 500 异常流。



商家想要知道自己店铺卖的最好的 top 50 商品，如何实现这个功能？

用 Redis 的 **Sorted Set** 就能轻松搞定。每个商品卖一单，就给它的分数加一，查 top 50 直接按分数倒序取就行。

1) 每卖出一个商品，执行一条 `zadd` 指令

```
// 商品 ID 作为 member，销量 +1 作为 score 更新
redisTemplate.opsForZSet().incrementScore("sales_rank", "item_10086", 1);
```

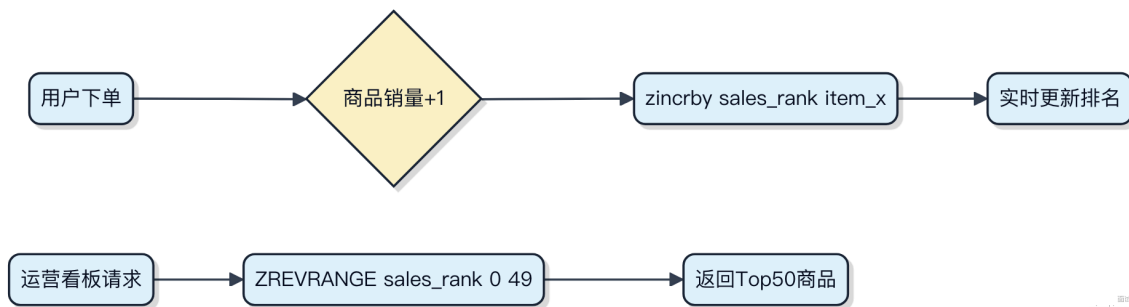
2) 查询销量最高的 50 个商品，直接 range 逆序取

```
// 从最高分开始取前 50
Set<String> top50 = redisTemplate.opsForZSet().reverseRange("sales_rank", 0, 49);
```

这个结构特别适合实时榜单场景，像淘宝的“热销榜”、抖音的“热榜”都是这么干的。Sorted Set 底层是跳表 + hash，插入和查询都是 $O(\log N)$ ，10 万商品里查 top 50 压根不费劲。

注意别把所有商品全放一个 key 里，后期数据大了会阻塞。可以按类目分桶，比如 `sales_rank:electronics`、`sales_rank:clothing`，读写更分散。

如果要支持日期维度，比如“今日 Top 50”，那就每天生成一个新 key，比如 `sales_rank:20240601`，定时任务凌晨合并前一天数据归档就行。



朋友圈点赞功能如何实现，简单说说？

朋友圈点赞功能的关键是平衡读写性能和数据一致性，尤其是热点动态的高并发场景。

1) 数据模型设计上，通常用两个集合存储：一个是点赞记录表（用户ID + 动态ID），另一个是缓存中的点赞数计数器。每次点赞先操作记录表，再异步更新计数。

2) 为了扛住高并发读，比如首页刷屏时批量展示点赞数，一般把计数存在 Redis 里。key 可以设计成 `like_count:feed_id`，查一条动态的点赞数就是 $O(1)$ 操作。

3) 写入时要防重复点赞，可以用 Redis 的 Set 结构存每个动态的点赞用户 ID，利用 `SADD` 原子性判断是否已点过。或者在数据库层面建唯一索引。

4) 对于超热内容，比如明星官宣，直接写数据库会成为瓶颈。这时候可以先写缓存+消息队列削峰，比如用 Kafka 接收点赞事件，后端消费批量落库，避免数据库被压垮。

5) 展示点赞列表时，如果人太多就不全显示，一般只显示共多少人，以及前几个好友或随机抽样。真要看全部，再分页查数据库。

代码层面核心操作类似：

```
// 用户点赞
Boolean isAdded = redisTemplate.opsForSet().add("likes:" + feedId, userId);
if (isAdded != null && isAdded) {
```

```
redisTemplate.opsForValue().increment("like_count:" + feedId);
}
```

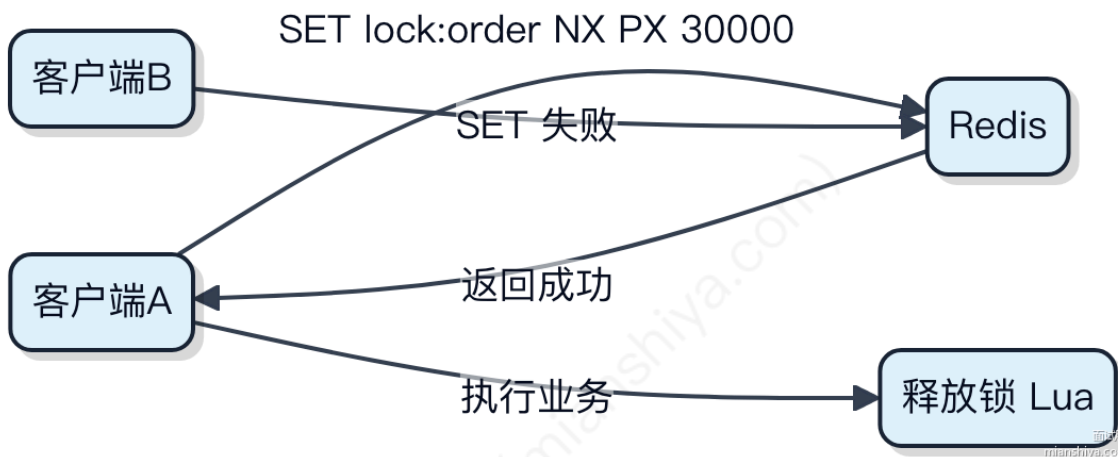
整个链路可以结合本地缓存、CDN、异步化来降低延迟，本质是用空间换时间，把脏活累活往后挪。

分布式锁一般都怎样实现？

分布式锁的实现，关键是要在多个节点之间协调对共享资源的访问，保证同一时刻只有一个客户端能拿到锁。常见的实现方式有三种。

- 1) 基于数据库实现，比如用 MySQL 的唯一索引或者乐观锁 version 字段。简单场景可以用，但性能差，一般扛不住高并发。
- 2) 基于 Redis，这是最常用的方案。利用 SET key value NX PX milliseconds 这个原子操作，value 一般是随机字符串或线程 ID，防止误删。释放锁时要用 Lua 脚本，确保“读取 value - 判断 - 删除”是原子的。Redisson 就是这么干的，还支持可重入、看门狗自动续期。
- 3) 基于 ZooKeeper，利用临时顺序节点和 Watch 机制。谁创建的节点序号最小，谁就获得锁。ZooKeeper 本身一致性更强，但系统复杂度也高，像 Curator 框架封装了分布式锁逻辑。

Redis 方案胜在性能好、易落地，大多数互联网公司用它。但要注意锁过期时间不好定，太短可能业务没执行完就被释放，太长又影响响应。所以 **可重入** 和 **自动续期** 很重要，不然容易出问题。



别自己从零写，直接上 Redisson 或 Curator，省事还靠谱。

如果让你统计每个接口每分钟调用次数怎么统计？

用滑动窗口计数器最实在，固定时间片加原子变量就能扛住高并发。

- 1) 每分钟初始化一个桶，key 是接口名 + 时间戳（分钟级），value 用 AtomicInteger 存调用量
- 2) 请求进来时算当前分钟 key，get or create 桶，increment
- 3) 过期桶定时清理，比如只保留最近 5 分钟，避免内存爆掉

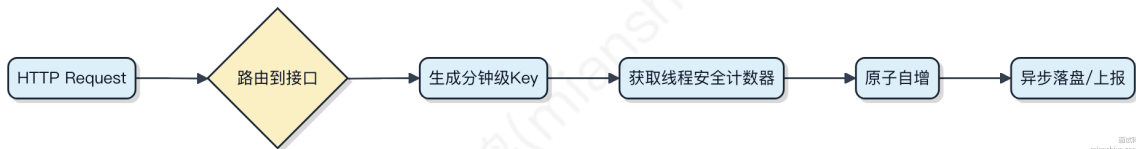
要是量大，可以加一级本地缓存 + 批量上报到 Redis 或 Kafka，比如每 10 秒 flush 一次，减少写压力。像 Sentinel 这种流量治理组件就是这么干的，本地用滑动窗口统计，聚合后走异步线程上报。

代码核心就这几句：

```
ConcurrentHashMap<String, AtomicInteger> counter = new ConcurrentHashMap<>();
String key = "api:/user/get:" + (System.currentTimeMillis() / 60000);
counter.computeIfAbsent(key, k -> new AtomicInteger()).incrementAndGet();
```

注意别用 synchronized，压根不经过锁才能扛住大流量。AtomicInteger CAS 操作在大多数场景足够快。

要是要求精确去重（比如防刷），就得上 Redis 的 INCR + EXPIRE 组合，靠分布式锁或 Lua 脚本保证原子性。但纯计数场景，本地内存+批量持久化更轻更快。



线上消息队列故障，兜底改造方案

消息队列挂了，最怕的就是数据丢了或者服务直接不可用。兜底的核心是保证关键链路还能走通，常见的做法就是引入降级和本地缓存机制。

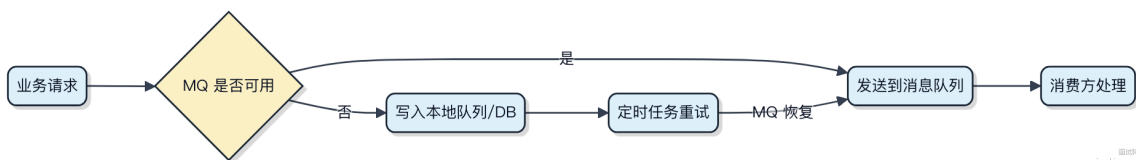
- 1) 先考虑业务场景是不是允许短暂不一致。比如订单创建这种，可以先把消息写到数据库的一个待发送表里，后面等 MQ 恢复了再异步补发。这个表其实就是个轻量级的持久化缓冲区，用定时任务去轮询重试就行。
- 2) 如果对实时性要求高，可以用本地内存队列做临时中转。比如用 Disruptor 或者 BlockingQueue 在 JVM 里暂存一下，同时配合健康检查开关，MQ 不可用时自动切到本地存储。注意得控制内存大小，避免 OOM。
- 3) 还有一种更稳的方案是多活架构，比如 RocketMQ 的 Dledger 模式，本身就有高可用能力。但如果整个集群都炸了，那就得靠外部手段，像阿里内部有些系统会把消息镜像一份到 Tair 这种分布式缓存里，双通道保障。

代码上其实很简单，关键是个状态判断：

```
if (mqService.isHealthy()) {
    mqService.send(msg);
} else {
    localQueue.offer(msg); // 落地或本地缓存
}
```

恢复的时候要有补偿流程，比如监听 MQ 可用事件，触发批量重放。整个过程要加监控和告警，不然你压根不知道啥时候该切回去。

要不要做兜底，得看业务容忍度。金融类交易一般必须有，后台日志采集这类丢了也没事的，完全可以不搞。



消息队列设计成推消息还是拉消息？推拉模式的优缺点？

消息队列到底是推还是拉，其实没有标准答案，关键看场景和设计目标。

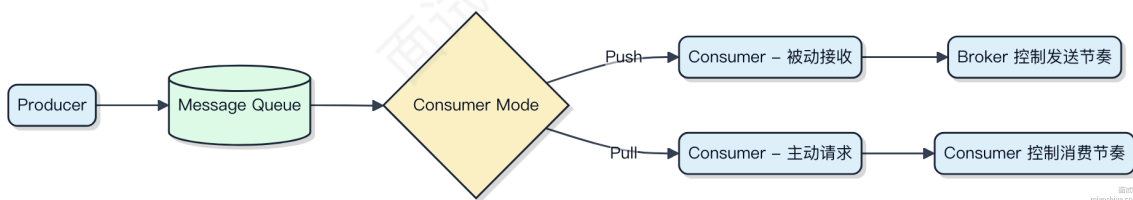
1) 推模式 (Push) 是服务端主动把消息发给消费者。好处是实时性好，消息一到就能立刻推下去，像 Kafka 的 consumer pull 其实底层用了长轮询模拟 push。但问题也很明显，如果消费者处理能力跟不上，容易被压垮，系统搞不定。

2) 拉模式 (Pull) 是消费者自己来取消息。灵活性高，消费者可以按自己的节奏消费，比如 RocketMQ 就是典型的 pull 模式。但缺点是可能有延迟，得靠轮询，太频繁浪费资源，间隔太久又不及时。

3) 所以很多系统走混合路线。Kafka 看起来是 pull，但通过参数控制每次拉取数量和等待时间，做到“准实时”。而 RabbitMQ 在 direct 等交换器下更像是 push，配合 QoS 限流，避免消费者过载。

真正的核心在于**流量控制**和**负载均衡**。push 容易造成雪崩，pull 可能增加 broker 查询压力。选择时得看吞吐要求、消费稳定性、网络开销这些维度。

举个例子，日志收集这种高吞吐场景用 pull 更稳；而订单状态通知这类强实时需求，适合可控的 push + 限流组合。



最终选哪种，不是技术决定，而是业务说了算。

什么是限流？限流算法有哪些？怎么实现的？

限流本质是控制系统的输入或输出流量，防止突发高并发压垮服务。比如秒杀场景，系统只能扛住 2000 QPS，那就要把超过的请求挡掉或排队。

常见的限流算法就三种：计数器、滑动窗口、漏桶和令牌桶。前两个其实是一类思路，后两个更精细。

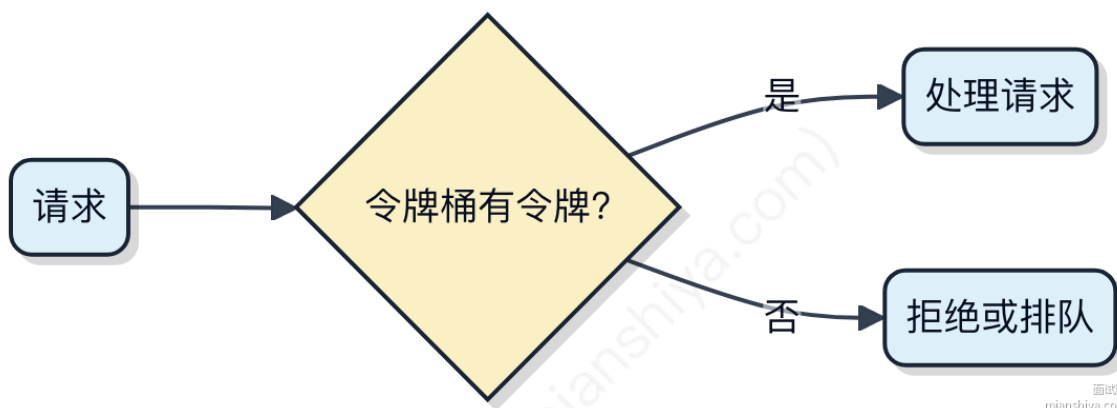
1) 计数器最简单，比如限制 1 秒最多 100 次请求，就用一个变量累加这 1 秒内的请求数，超了就拒绝。但它在时间边界会有突刺问题，比如第 1 秒最后 10ms 放行 100 个，第 2 秒开头又放行 100 个，瞬间 200 个请求打进来。

2) 滑动窗口解决了这个问题，把 1 秒拆成 10 个 100ms 的小格子，每次只看最近连续的 10 个格子的总请求数。这样流量更平滑，Redis + Lua 可以实现，Sentinel 里就有这玩意。

3) 漏桶模型强调恒定速率处理请求，不管进来的多快，都按固定速度“漏水”处理。超出桶容量的请求直接丢弃。它能削峰填谷，但无法应对短时间的合理 burst 流量。

4) 令牌桶反过来，允许一定程度的突发。比如每秒往桶里放 100 个令牌，请求来要先拿到令牌才能执行。桶里最多存 200 个令牌，意味着最多能承受 200 的瞬时并发，之后没令牌就等着或者被拒。Guava RateLimiter 用的就是这个。

代码上，用 Redis 实现分布式限流比较常见，比如用 INCR 维护窗口计数，配合 EXPIRE 设置过期时间。集群环境下也可以用 Sentinel 这种中间件统一控制。



线上 CPU 飙高如何排查?

CPU 飙高排查本质是找到占用 CPU 的元凶线程，并定位到代码堆栈。整个过程分三步走：先看系统，再看进程，最后看线程和堆栈。

- 1) 用 `top` 命令看整体 CPU 使用情况，如果 `%Cpu(s)` 的 `us`（用户态）或 `sy`（内核态）持续高于 80%，说明系统压力大。
- 2) 继续在 `top` 中按 `P` 按 CPU 排序，找到 Java 进程的 PID。
- 3) 执行 `top -H -p <PID>`，查看该进程下所有线程的 CPU 占用，找出 `top` 几个高消耗线程，记下其线程 ID (TID)。

接着把 TID 转成 16 进制（比如 `printf "%x\n" <TID>`），然后用 `jstack` 抓堆栈：`jstack <PID> | grep -A 50 <hex_TID>`，就能看到对应线程的调用栈，通常是 `RUNNABLE` 状态的业务线程在死循环、正则回溯、频繁 GC 或序列化卡住。

常见场景比如：

- 正则表达式存在灾难性回溯，像 `(.+) +` 匹配长字符串，CPU 直接拉满。
- 日志里频繁输出大对象 JSON，Jackson/Gson 在序列化时卡住。
- 外部依赖超时导致线程池积压，大量线程处于 `RUNNABLE` 等待网络返回。

代码上可以用 `ThreadMXBean` 主动监控，但线上第一反应还是 `top + jstack` 组合拳最稳。如果是容器环境，记得进宿主机操作或用 `kubectl exec`。



如何实现数据库的不停服迁移?

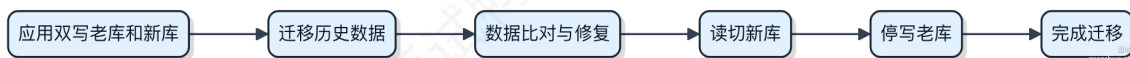
不停服迁移的核心是**双写同步**和**数据校验**，整个过程要保证业务无感、数据不丢。

- 1) 先做结构兼容改造。老库和新库字段设计得能共存，比如新增非空字段要有默认值，避免双写时报错。
- 2) 开启双写。在代码里把数据库操作改成同时写两份，老库和新库都落数据。读还是只从老库读，这时候新库只是个“影子库”。

```
// 伪代码示意：双写阶段
userDao.writeToOldDB(user);
userDao.writeToNewDB(user); // 异步发消息也可以，但要考虑失败重试
```

- 3) 跑数据迁移工具，把老库的历史数据搬过去。可以用 DataX 或 Canal + Kafka + Flink 这类组合，边搬边追增量。
- 4) 数据比对。拿工具抽样比对两边的数据一致性，比如阿里的 DataDiff，发现不一致就补。
- 5) 切换读流量。确认数据基本一致后，先把读请求切到新库，观察一段时间。
- 6) 下线双写。读稳定了，再把双写关掉，只写新库，老库归档。

整个过程最怕的是中间出错导致数据不一致。所以每一步都要有监控和回滚方案，比如用 **Canal** 监听 binlog 做实时反向补偿。



Java 写入文件到磁盘会经历哪些过程？

写文件这事儿，看起来简单，底层其实绕了好几道弯。从 Java 代码调 `write()` 开始，数据不是直接进磁盘的。

1) 用户态缓冲

Java 的 `FileOutputStream` 或 `BufferedOutputStream` 先把数据写到用户空间的缓冲区。这个阶段压根不涉及系统调用，只是内存拷贝。

2) 内核页缓存 (Page Cache)

真正触发 IO 是调用 `flush()` 或缓冲满时，JVM 发起 `write` 系统调用，数据从用户缓冲复制到内核的 Page Cache。这时候应用线程可能阻塞，也可能不阻塞，取决于是否用 NIO 的异步模式。

3) 脏页回写 (Writeback)

数据到了 Page Cache 并不算落盘，标记为“脏页”。由内核线程 `pdflush`（或现在的 `writeback` 子系统）在后台把脏页写入磁盘。时机受 `vm.dirty_ratio`、`vm.dirty_background_ratio` 这些参数控制。

4) 块设备层与磁盘控制器

数据经过虚拟文件系统 (VFS)、具体文件系统（比如 ext4）、块设备层，最后通过 IO 调度器（如 CFQ、NOOP）发给磁盘控制器。SSD 和 HDD 处理方式不同，但都涉及物理写入。

5) 持久化确认

只有当磁盘控制器真正把数据写入非易失介质，并返回确认信号，才算完成。如果中途断电，没刷下去的数据就丢了。

想确保数据落盘，得调 `channel.force(true)`，它对应 `fsync()` 系统调用，强制刷脏页和磁盘写缓存。

HashMap 是不是线程安全的？如果让你来实现一个线程安全的 HashMap 你要怎么设计？如果不用加锁你要怎么设计？

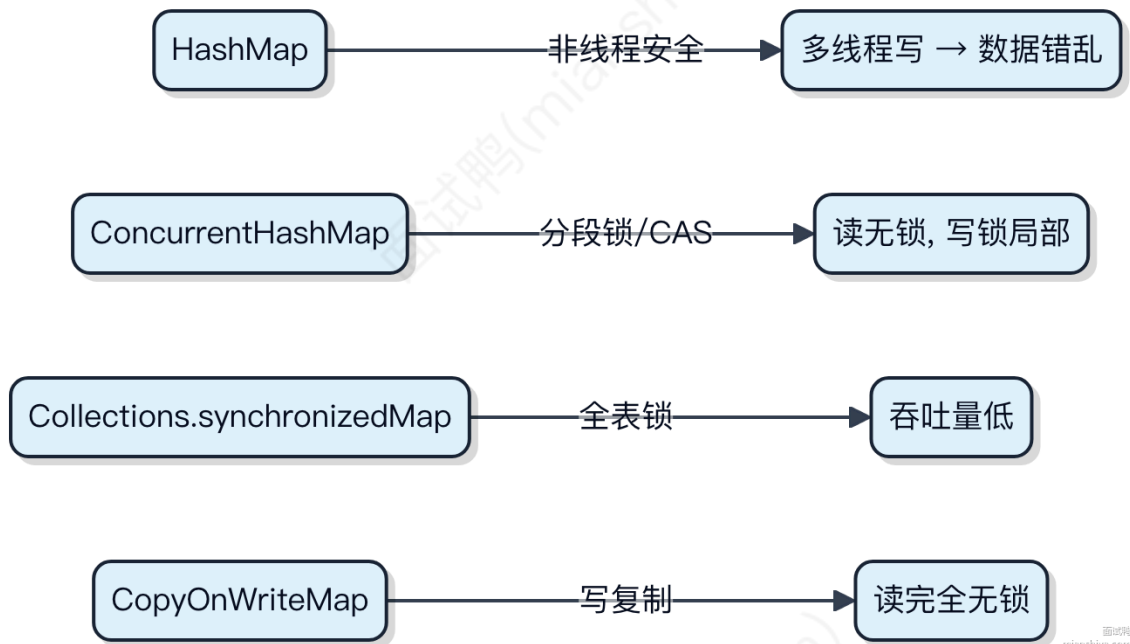
HashMap 本身不是线程安全的，多线程环境下并发写可能会导致链表成环、数据覆盖等问题。JDK 提供了 `Collections.synchronizedMap` 来包装出线程安全版本，但它是**粗粒度锁**，所有操作都互斥，性能差。

真正实用的方案是 **ConcurrentHashMap**。它不靠单一锁解决问题，而是把 map 拆成多个 segment 或桶，每个部分独立加锁。JDK 7 用分段锁 (ReentrantLock)，JDK 8 改为 **CAS + synchronized**，锁粒度细化到桶头节点。读操作

基本无锁，写操作只锁冲突的链头，高并发下能扛住几千 QPS。

如果让你手写一个无锁的线程安全 Map，思路得转向**不可变性和无锁结构**。比如用 `ConcurrentSkipListMap`，基于跳表实现有序 Map，底层用 CAS 操作维护节点指针，读写并发高，天然支持范围查询。或者用 `CopyOnWriteMap` 思路，写时复制整个 map，适合读远多于写的场景，像配置中心缓存这种。

- 1) 核心是减少锁竞争，不是消灭并发问题
- 2) 常见误区是给 `HashMap` 外层套 `synchronized`，这会成为性能瓶颈
- 3) 真要自研，优先考虑 JDK 已有方案的设计思想，别从零造轮子



如何避免用户重复下单（多次下单未支付，占用库存）

用户重复下单这事儿，本质是解决**接口幂等性**和**库存预占合理性**的问题。你得保证同一个用户提交多次，不会创建一堆待支付订单，不然库存很快就被占光了。

一般做法是结合“唯一标识 + 状态控制”来拦截重复请求。比如用户点击下单时，前端带上一个由客户端生成的**幂等令牌 (idempotent token)**，这个 token 一次有效。后端收到请求先校验 token 是否已使用，如果已经用过直接返回上次结果。

- 1) 用户发起下单前，先向服务端申请一个 token
- 2) 下单请求携带该 token，网关或订单服务做前置校验
- 3) 校验通过后处理业务，订单创建成功后将 token 标记为已使用（可用 Redis 缓存，TTL 对齐订单超时时间）
- 4) 同一 token 再次提交，直接返回原始订单信息，不走创建逻辑

库存这块要配合做**预占隔离**。订单创建时调用库存服务锁定库存，而不是直接扣减。可以用 `Redis` 或数据库行锁标记“预占状态”，超时未支付自动释放。

代码上简单说就是：

```
String result = redisTemplate.opsForValue().getAndSet("idempotent:" + token,
orderId);
if (result != null) {
```

```
        return buildResponse(result); // 已存在，直接返回原订单
    }
```

典型场景像电商系统里用 **Seata 的 AT 模式** 做分布式事务，库存预占作为其中一个分支，回滚时自动释放。或者自己基于消息队列实现延迟释放，比如 RabbitMQ 死信队列、RocketMQ 延时消息触发超时关单。

整个链路要短且快，token 校验压根不经过主业务逻辑，属于前置过滤器级别的操作，这样才能扛住刷单流量。

微服务中远程调用的超时时间应该设置为多少合适？

微服务的超时时间没有统一标准，得根据具体业务场景来定。一般内部服务之间的调用，响应在 100ms 内算正常，**超时时间设为 500ms 到 2s 比较常见**。如果是涉及复杂计算、批量处理或外部依赖（比如调第三方支付），可能需要 5s 甚至更长。

关键点在于不能无限等待。客户端必须设置 **connect timeout** 和 **read timeout**，避免线程被长时间占用。比如用 OpenFeign 时：

```
@FeignClient(name = "order-service", configuration = FeignConfig.class)
public interface OrderClient {
    @GetMapping("/orders/{id}")
    Order getOrder(@PathVariable("id") String orderId);
}

// 配置超时
public class FeignConfig {
    @Bean
    public Request.Options options() {
        return new Request.Options(
            Duration.ofMillis(1000), // connect timeout
            Duration.ofMillis(2000)  // read timeout
        );
    }
}
```

还要考虑链路叠加问题。A 调 B，B 又调 C，如果每层都设 2s 超时，整体可能卡住好几秒。这时候得做超时逐层递减，比如入口设 2s，下游调用留 1.5s，给自身处理留点缓冲。

另外，配合熔断器像 Sentinel 或 Hystrix 使用效果更好。短时间大量请求超时，直接熔断，防止雪崩。比如 Sentinel 可以按 RT（响应时间）规则触发降级。

最后，上线后一定要监控实际耗时分布。通过 Prometheus + Grafana 看 P99、P999 延迟，再反过来调整超时阈值，别靠猜。

1000 个任务，每个任务 0.1s，最大响应时间 1s，线程池参数怎么设置？

这类问题其实是在考察你对线程池设计背后权衡的理解，不是背公式。

1) 先算总工作量：1000 个任务 × 0.1s = 100 秒的 CPU 工作量。这些任务如果串行执行要 100s，但我们有最大响应时间 1s 的约束。

2) 响应时间卡在 1s，每个任务最长 0.1s，意味着 **最多容忍队列积压 9 个任务**（0.9s 等待 + 0.1s 执行）。再多就超时了。

3) 为了 1s 内处理完所有请求，最理想是让这 1000 个任务在 1s 内并行跑完。需要的并发数 $\approx 1000 \times 0.1 / 1 = 100$ 个线程。也就是说，**核心线程数设为 100 左右比较合理**。

4) 队列不能太大，否则会堆积导致超时。用 `SynchronousQueue` 最合适，不存储任务，直接交给线程，搞不定就新建或拒绝。

5) 拒绝策略建议用 `AbortPolicy`，直接抛异常，外部可以降级或熔断，比如结合 Sentinel 控制入口流量。

代码上长这样：

```
new ThreadPoolExecutor(  
    100, 200,  
    60L, TimeUnit.SECONDS,  
    new SynchronousQueue<>(),  
    new ThreadPoolExecutor.AbortPolicy()  
);
```

注意最大线程数别设太高，防止系统被瞬间打垮。如果任务是 IO 密集型，比如调远程服务，那线程可以多一点；但这里是纯计算 0.1s，更偏向 CPU 密集，所以线程数不宜远超 CPU 核数太多。

最终还得压测验证，理论只是起点。

有一张表里面有三个字段，分别是 (id, 开始时间, 结束时间)，表中数据量为 5000W，如何统计流量最大的时候有多少条数据？

流量最大的时刻，其实就是所有时间段中重叠最多的那个点。直接遍历 5000W 条记录的每个时间点去统计，肯定搞不定，时间和内存都扛不住。

正确思路是把“时间段”拆成“事件点”，用**扫描线算法 (Sweep Line)**。每条记录的开始时间看作 +1 事件，结束时间看作 -1 事件。把这些事件按时间排序，从左到右扫一遍，累加变化值，最大值就是峰值并发数。

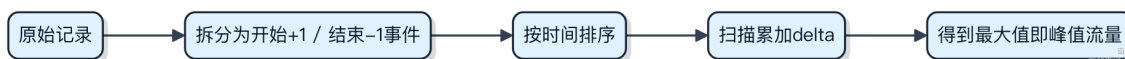
具体做法：1) 把每条记录拆成两个事件：{time: startTime, delta: +1} 和 {time: endTime, delta: -1} 2) 按 time 排序，如果时间相同，**结束事件优先**，避免把刚好接续的时间算成重叠 3) 顺序扫描，维护一个计数器，遇到 +1 加一，-1 减一，记录过程中的最大值

代码核心逻辑：

```
List<Event> events = new ArrayList<>();  
for (Record r : records) {  
    events.add(new Event(r.startTime, 1));  
    events.add(new Event(r.endTime, -1));  
}  
events.sort((a, b) -> {  
    if (a.time != b.time) return Long.compare(a.time, b.time);  
    return Integer.compare(a.delta, b.delta); // 结束事件 -1 在前  
});
```

数据量大可以分片处理。比如用 Spark 把所有事件拉过来做全局排序和扫描。或者先按小时分桶，找到可能出峰值的几个时间段再精细计算。

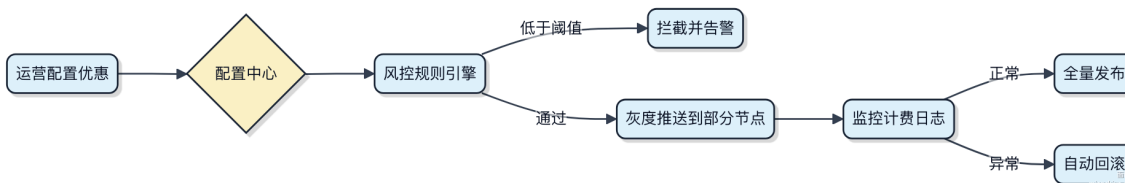
整个过程时间复杂度是 $O(n \log n)$ ，主要是排序开销。5000W 条数据在分布式环境下跑一次也就几分钟，能搞定。



针对支付宝最近出现的八折优惠事故，说说如何才能避免类似事件的发生？

支付宝的八折优惠事故，本质上是**配置变更**引发的生产故障。这类问题不靠“人肉审核”能兜住，得靠体系化的防护机制。

- 1) 所有关键参数必须走配置中心，比如 Apollo 或 Nacos，禁止硬编码。配置变更要有审计日志，谁改的、什么时候改的、改前改后值是什么，全部留痕。
- 2) 加**多级审批和熔断策略**。像优惠这种直接影响资损的功能，配置生效前需要多人复核，甚至自动触发风控校验。比如折扣低于 85 折，系统直接拦截并告警。
- 3) 灰度发布不可少。新配置先推给 1% 用户，跑通订单链路，监控无异常再逐步放量。Zookeeper 不适合做实时配置推送，优先选支持监听回调的中间件。
- 4) 核心业务要有**反向校验能力**。下单时服务端不仅要读取优惠规则，还得判断当前优惠力度是否在合理区间，超出阈值直接拒绝，而不是盲执行。

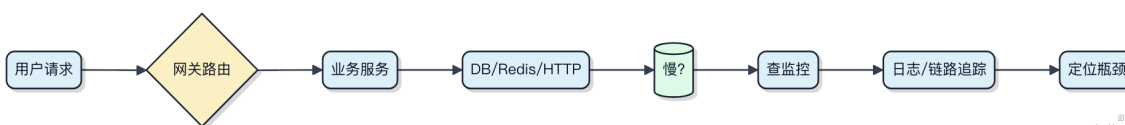


说白了，不能把业务安全压在一个操作员不犯错上。系统得有自保能力，变更只是输入，最终是否生效得由一整套流程说了算。

接口变慢了应该如何排查？导致接口变慢的原因有哪些？

接口变慢不是单一问题，是系统某个环节亮了红灯。排查得从链路一层层看，原因基本逃不开这四类：应用代码、外部依赖、中间件瓶颈、基础设施。

- 1) 先看监控大盘，比如 SkyWalking 或 Prometheus，定位是哪个服务、哪个接口、甚至哪一行代码耗时突增。如果发现数据库调用占了 80% 时间，那问题大概率在 SQL 慢查询。加个索引可能就解决了。
- 2) 外部依赖很常见，比如调第三方 HTTP 接口卡住。Feign 或 RestTemplate 默认超时不设，一旦对方抖动，线程池打满，整个应用就堵死。这类要配好超时和熔断，Hystrix 或 Sentinel 都能搞。
- 3) 缓存击穿也容易拖垮接口。Redis 挂了或 key 过期瞬间，大量请求直奔数据库。压测过就知道，MySQL 扛不住这种突发读流量。一般用空值缓存 + 本地缓存兜底。
- 4) JVM 老年代频繁 GC 也会导致接口毛刺。通过 `jstat -gc` 或 Arthas 查看 GC 日志，如果 Full GC 间隔小于 10 分钟，就得查内存泄漏。Arthas 的 `trace` 命令能定位到具体方法耗时。



别一上来就查代码。先看全局指标：CPU、RT、QPS、错误率。四个里有两个异常飙升，再往下钻。工具链得配齐，不然就是盲人摸象。

编写一段代码，使得这段代码必定会产生死锁，不能使用Thread.sleep()

死锁的产生需要四个条件同时满足：互斥、占有并等待、不可抢占、循环等待。我们只要设计一个程序，让两个或多个线程以相反顺序持有和请求两个锁，就能稳定触发。

最简单的办法是创建两个对象作为锁资源，两个线程分别先抢对方刚释放的锁，形成环路依赖。

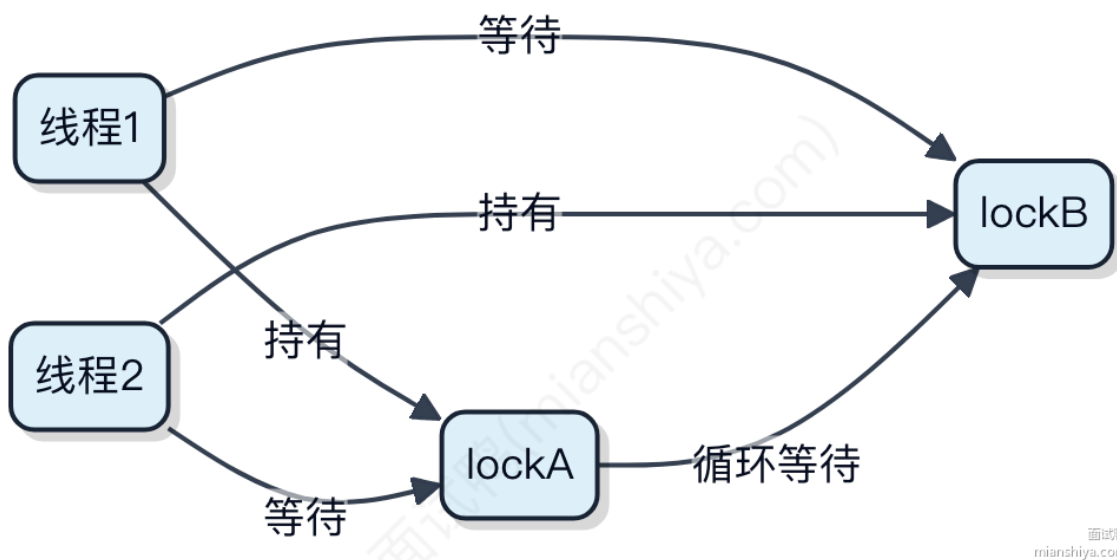
```
Object lockA = new Object();
Object lockB = new Object();

new Thread(() -> {
    synchronized (lockA) {
        System.out.println("线程1拿到lockA, 准备拿lockB");
        synchronized (lockB) {
            System.out.println("线程1拿到lockB");
        }
    }
}).start();

new Thread(() -> {
    synchronized (lockB) {
        System.out.println("线程2拿到lockB, 准备拿lockA");
        synchronized (lockA) {
            System.out.println("线程2拿到lockA");
        }
    }
}).start();
```

- 1) 线程1先拿到lockA，尝试获取lockB
- 2) 线程2此时已持有lockB，正等待lockA
- 3) 线程1拿不到lockB，线程2拿不到lockA，双方卡住
- 4) JVM不会主动干预这种等待，**死锁**就此发生

实际项目里，Spring 的事务代理或 Dubbo 的同步调用如果涉及嵌套远程服务+本地锁，也容易无意中搞出类似问题。排查时得靠 `jstack` 看到"Found one Java-level deadlock"提示。



在 Java 中，不使用锁如何实现一个线程安全的单例？

最稳的写法是**静态内部类**，也叫“懒加载登记式”。类加载的时候不初始化实例，只有第一次调用 `getInstance()` 时才会触发内部类加载，从而创建实例。JVM 保证类加载过程的线程安全，天然规避了多线程问题。

```
public class Singleton {  
    private Singleton() {}  
  
    private static class Holder {  
        static final Singleton INSTANCE = new Singleton();  
    }  
  
    public static Singleton getInstance() {  
        return Holder.INSTANCE;  
    }  
}
```

还有一种是枚举，Effective Java 里推荐的。枚举实例的创建默认就是线程安全的，而且防反射、防序列化破坏，代码还短。

```
public enum Singleton {  
    INSTANCE;  
}
```

饿汉式也行，但得在类加载时就创建，可能浪费资源。如果单例占用大、启动慢，会影响应用初始化时间。

有人提双重检查锁（DCL），但得配合 `volatile`，不然有对象未完全构造完成就被读到的风险。JDK 1.5 之前 DCL 是有问题的，之后靠 `volatile` 的禁止指令重排序才修掉。

至于 CAS 自旋？能做，但属于杀鸡用牛刀。像 `AtomicReference` 配合循环，适合复杂场景，单例这种一次性的没必要，反而增加复杂度。

总结：优先静态内部类，追求极致安全选枚举。别折腾锁，也别自己造 DCL 轮子。

如果项目需要你实现敏感词过滤功能，如何实现？

敏感词过滤看似简单，但真要搞稳定，得考虑性能、维护性和准确性。直接上正则替换是初级做法，数据一多就卡死。

1) 最核心的是匹配算法。**Trie 树**（字典树）是这类场景的标配，把所有敏感词构建成树形结构，文本只需遍历一遍就能完成全部匹配，时间复杂度接近 $O(n)$ 。比如 10 万个敏感词，也能在毫秒级完成匹配。

2) 工程落地时别自己从头写。直接用开源库更靠谱，像 `java-dump-filter` 或 `sensitive-word` 这类基于 DFA（确定有限状态机）优化的库，支持热更新词库、模糊匹配（如“和”谐“音”），还能插 Redis 做动态管理。

3) 高并发下别让词库拖后腿。把 Trie 树结构缓存在本地内存，配合定时任务拉取最新词库，避免每次查数据库。如果用 Spring Boot，可以封装成自动刷新的 Bean。

4) 实际处理策略要灵活。有些场景需要替换成 ***，有些要拦截请求，还有些只是打标记走审核流。定义好处理器接口，按业务分流：

```
public interface SensitiveHandler {  
    void handle(String text, List<String> matchedWords);  
}
```

5) 别忘了特殊 case。中文分词歧义、“f**k”这类变形、中英文混合，单纯靠字面匹配会漏。进阶方案可结合 NLP 模型做语义判断，但成本高，一般用正则预处理 + 白名单绕过即可。

要不要配规则引擎？除非策略特别复杂，否则压根不必要。Spring + 自研工具类完全能扛住日常需求。

如果现在有 500G 数据需要排序，但是你只有 4G 内存，如何实现？

外排序的核心是**归并排序**和分治思想。内存装不下全部数据，就得把大文件拆成能放进内存的小块，逐个处理。

1) 先把 500G 文件切分成多个 4G 左右的小文件，比如切成 125 个小文件。2) 每个小文件读进内存，用快排或归并排好序，再写回磁盘。这时候每个小文件内部有序。3) 然后开启多路归并，把这 125 个有序小文件当成输入源，每次从各个文件读一部分数据到内存缓冲区，用最小堆维护当前最小值，不断选出全局最小写入最终文件。

举个例子，像 Hadoop 的 MapReduce 排序就是这么干的。Map 阶段本地排序，Reduce 阶段做多路归并合并所有分区数据。

代码上不用自己实现底层，但得明白流程：

```
// 模拟多路归并中的比较单元  
class Element {  
    int value;  
    int fileIndex; // 来自哪个小文件  
}
```

整个过程磁盘 IO 是瓶颈，所以缓冲区管理很关键。可以加大读写缓冲，减少系统调用次数。



其实现代大数据系统都封装好了这套逻辑，但面试官想听的是你能不能把 **分而治之** 和 **外部排序** 这两个点说透。

项目上现在需要存储 IP 地址，数据库应该用什么类型来存储？

IP 地址存储其实分两种情况，一种是 IPv4，一种是 IPv6。别直接上 varchar(15) 或 varchar(39) 存文本，虽然能用但不是最优解。

MySQL 里有 **INET_ATON** 和 **INET_NTOA** 这两个函数，可以把 IPv4 字符串转成无符号整数存。比如 '192.168.1.1' 转成 3232235777，字段类型用 INT UNSIGNED 就行，占 4 字节，查询也快，索引效率高。

IPv6 长度更长，可以用 INET6_ATON / INET6_NTOA，对应字段类型得用 VARBINARY(16)，因为 IPv6 地址是 128 位，正好 16 字节。这样不管是 v4 还是 v6，都能统一用二进制方式处理。

实际项目里像 Nginx 日志分析系统或者风控系统，比如用 ClickHouse 做 IP 分析，人家原生就支持 IPv4 和 IPv6 类型，直接用就行。但在标准 MySQL 里就得靠函数转换。

代码层面查的时候：

```
SELECT INET_NTOA(ip_num) FROM user_log WHERE ip_num = INET_ATON('192.168.1.1');
```

有个坑要注意：PHP 或 Java 里如果用 int 接 MySQL 的 INT UNSIGNED，可能溢出，得用 long 或 BigInteger 处理。

总的来说，别存字符串，优先转整数或二进制存，省空间、提性能。

你说你项目上用了分布式锁，加锁后并发度不就降低了吗？

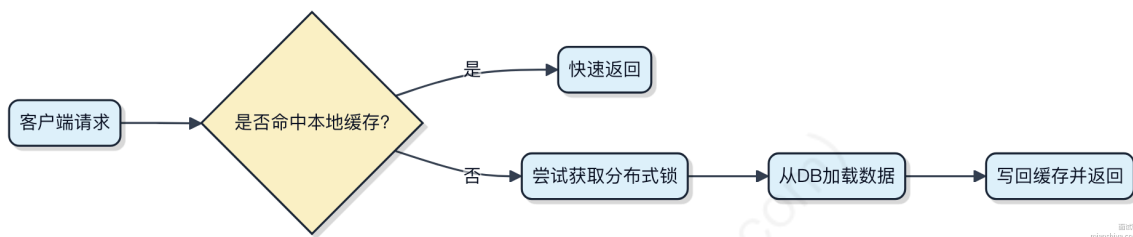
分布式锁确实会引入串行化操作，但它的目标不是提升并发度，而是保证关键路径的数据一致性。你得先想清楚，哪些场景必须用锁来保护共享状态。

比如在秒杀系统里，库存扣减就是典型的竞争点。Redisson 的 **RLock** 就是基于 Redis 的 **SETNX** 实现的可重入锁，加锁时设置过期时间防止死锁，解锁时通过 Lua 脚本保证原子性。

```
RLock lock = redisson.getLock("stock_lock");
lock.lock(); // 默认30秒过期，自动续期
try {
    // 扣减库存，DB 或缓存操作
} finally {
    lock.unlock();
}
```

这种设计下，并发度确实会下降，但这是为了防止超卖这种严重问题。实际中我们会尽量缩小锁的粒度，比如按商品 ID 分段加锁，而不是全局锁。像库存这种每秒几万请求的场景，通过分段锁 + 本地缓存计数，能把单点竞争降低到几百以内。

另一个思路是用无锁方案替代。比如用 Redis 的 **DECR** 原子操作直接扣库存，压根不经过应用层锁。或者用 TCC、消息队列削峰填谷，把强一致变成最终一致。

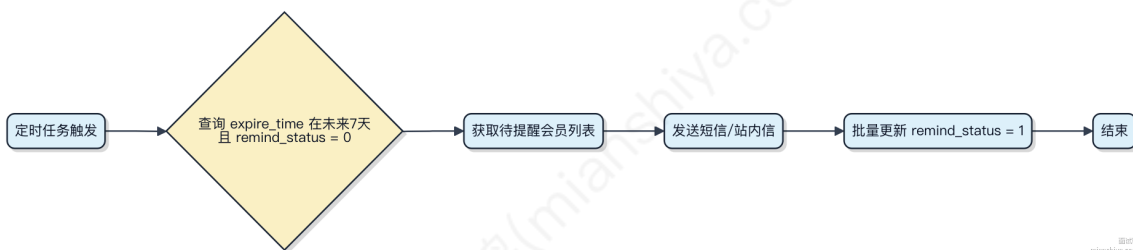


所以关键在于权衡。数据一致性要求高的地方，宁愿牺牲一点吞吐也要上锁；能容忍弱一致的，就用原子操作或异步解耦。

会员表中有500万条会员数据，有什么方式可以对即将过期的会员提前7天进行提醒？

定时扫描过期时间是常见做法，但500万数据量下直接全表扫会把数据库搞死。得用更聪明的办法。

- 1) 核心思路是**减少每次扫描的数据集**，别每次都查500万条。可以给会员表加个状态字段，比如 `remind_status`，标记是否已提醒。查询时只捞 `expire_time` 在未来7天内且未提醒的记录。
- 2) 索引必须建好，`expire_time` 字段上建B+树索引，这样范围查询快。配合 `remind_status = 0` 的条件，走索引过滤，实际扫描行数可能从百万级降到几千。
- 3) 任务调度用 xxl-job 或 Quartz，每天跑一次，查满足条件的数据。处理完一批就更新 `remind_status`，避免重复通知。
- 4) 如果数据量再大，可以按分片处理。比如用用户ID取模分10张表，每天只处理其中1-2个分片的数据，把压力摊开。
- 5) 极端情况还能引入延迟队列。用户开通会员时，算出过期前7天的时间点，往 RocketMQ 延迟队列发一条消息。到点自动触发提醒，完全不用查库。不过要注意MQ不能丢消息，适合对准确性要求高的场景。



其实大多数业务用方案1+2就够了，简单可控。延迟队列听着高级，运维成本也高，搞不定反而背锅。

调用第三方接口应注意哪些问题？

调用第三方接口不是发个 HTTP 请求就完事，背后要考虑的点其实不少。最怕的就是对方服务一抖，你整个系统跟着挂。

超时控制是第一道防线。很多开发者压根不设超时，等着对方返回，结果连接池被占满，自己先崩了。一般设置连接超时 13 秒，读取超时 510 秒比较合理。Spring Boot 里用 `RestTemplate` 或 `WebClient` 都得手动配。

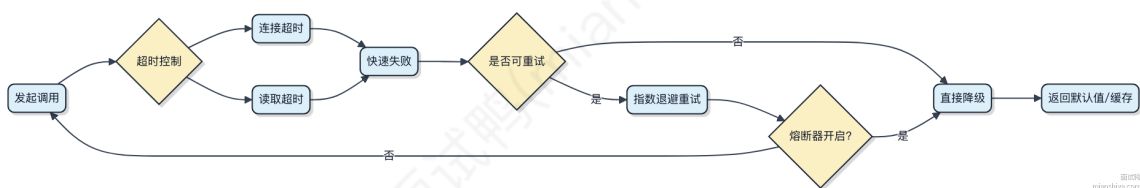
- 1) 重试机制不能滥用。网络抖动可以重试 2~3 次，但要加退避策略，比如第一次等 1 秒，第二次 2 秒。像支付类接口这种带副作用的操作，绝对不能自动重试，容易重复扣款。

2) 熔断降级得跟上。Hystrix 虽然停更了，但 Alibaba Sentinel 或 resilience4j 都能做。比如错误率超过 50% 就直接熔断几分钟，让请求走本地缓存或默认值，别把压力传给下游。

数据安全也得注意。敏感字段比如 token、用户 ID，尽量走 HTTPS。有些老系统还要求放 header 里传 secret key，记得别打日志，否则泄露了算事故。

幂等性设计要看场景。如果你是在订单创建后回调第三方物流，那得自己记录状态，避免重复触发。可以用 Redis 存个唯一任务 ID，10 分钟内重复来就直接忽略。

最后是监控和告警。每个外部调用都要打埋点，统计成功率、P95 耗时。Prometheus + Grafana 配个看板，再结合 AlertManager 发钉钉通知，出问题能第一时间感知。



让你设计一个购物车功能，怎么设计？

购物车功能看着简单，其实要考虑的点不少，尤其是高并发和一致性问题。

先说存储选型。用户未登录时用 LocalStorage 或 Cookie 存，体验顺滑，但换设备就丢。一登录，就得把本地数据同步到服务端。服务端一般存 Redis，key 用 `cart:userId`，value 用 hash 结构，field 是商品 ID，value 是数量和一些扩展信息。Redis 操作快，撑个几万 QPS 压力不大。

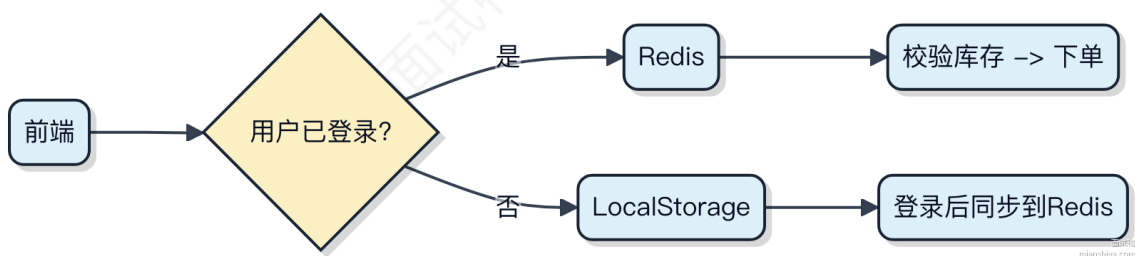
加减商品得防超卖。比如库存只有 10 件，你不能让 cart 加到 11。这个校验放在「下单时」做更合理，购物车本身允许临时多加，否则每次操作都查库存，系统早崩了。但数量为负？绝对不行，代码里直接卡住。

合并场景要处理。用户先加了商品 A 到本地购物车，登录后要和服务器上的记录合并，相同商品数量叠加，别覆盖。

接口设计几个核心就行：

- GET /cart：拉列表
- POST /cart：增删改一条记录
- DELETE /cart/{itemId}：删

性能方面，读多写少，可以考虑本地缓存一层，但 TTL 别设太长，避免和促销活动对不上。



重点其实是状态同步和边界控制，别在购物车上搞复杂事务，脏活累活留给订单系统去扛。

系统每天晚上都会有一小时左右的时间瘫痪，你觉得可能的原因是什么？有遇到过这样的情况吗？

这种问题在生产环境其实挺典型的，一般先别急着猜具体原因，得从时间规律和系统行为入手。每天固定时段瘫痪一小时，基本可以锁定是**周期性任务冲击**或者**资源耗尽型故障**。

1) 首先看是不是有定时批处理任务，比如用 xxl-job 或 Quartz 调度的夜间数据同步、报表生成、缓存预热这类操作。如果这些任务突然变重，比如数据量从 100 万涨到 1 亿，又没做分片，很容易把 CPU 打满、数据库连接池耗光，甚至引发雪崩。

2) 再查系统资源水位。有没有可能是 JVM Full GC 频繁？比如老年代堆积了大量对象，每晚触发一次长时间 GC，应用停顿几十秒到几分钟，看起来就像瘫痪。用 `jstat -gc` 或 APM 工具看一下 GC 日志就能确认。

3) 网络或依赖组件也有坑。比如每晚 2 点日志归档，NFS 挂载点卡住，导致所有写日志的线程阻塞，整个请求链路堵死。或者定时调用下游服务，对方限流了，重试机制又没控制好，线程池被耗尽。

代码上可以加个简单的防护：

```
// 定时任务加熔断和超时
@Scheduled(cron = "0 0 2 * * ?")
public void safeBatchJob() {
    if (circuitBreaker.isOpen()) return;
    try {
        Future<?> future = executor.submit(batchTask);
        future.get(10, TimeUnit.MINUTES); // 控制执行时间
    } catch (TimeoutException e) {
        circuitBreaker.trip();
    }
}
```

常见于传统金融、电商系统的夜间跑批场景，本质是**缺乏资源隔离和流量治理**。建议把重任务迁到独立集群，或者用 xxl-job 的分片广播能力拆解压力。

如果要实现一个抢红包的功能，红包金额是如何计算的？

抢红包金额的分配，关键在于保证总额不变的同时，让每个用户抢到的金额随机且合理。常见的有两种算法：线段切割法和二倍均值法。

1) 线段切割法

把总金额想象成一根绳子，抢红包的人就是在这根绳子上随机切几刀，每一段就是一个人抢到的金额。比如 100 元发给 10 个人，就在 [0,100] 区间里随机选 9 个点，排序后计算相邻间距。这种方式能保证所有人金额加起来正好是总额，而且每个人理论上均有均等的机会拿到大额或小额。

2) 二倍均值法（也叫期望值控制法）

每次抢的时候，当前可抢金额的上限是剩余人均金额的 2 倍。比如还剩 50 元、8 个人没抢，那下一个人最多能抢到 $(50/8) * 2 = 12.5$ 元，再在这个范围内随机。这样既能保持随机性，又不会出现前面抢太多后面不够分，或者最后几个人只能抢到一分钱的情况。

代码上其实很简单，核心就是控制随机范围：

```
Random random = new Random();
double leftAmount = total;
for (int i = 0; i < count - 1; i++) {
    double max = leftAmount / (count - i) * 2;
    double amount = Math.max(0.01, random.nextDouble() * max);
```

```
// 分配金额...
leftAmount -= amount;
}
// 最后一人拿完剩余
```

微信早期用的是线段切割，后来优化为改进版的期望控制策略，兼顾公平和体验。实际系统中还会结合 Redis + Lua 保证原子性，防止超发。

线上 CPU Load 飙高如何排查？

先用 `top` 命令看整体 CPU 使用情况，重点关注 load average 和占用最高的进程。把问题进程的 PID 记下来，下一步深入分析。

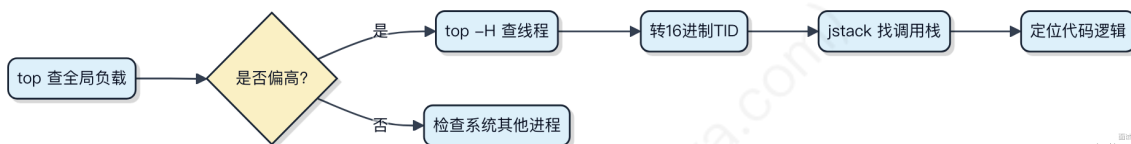
用 `top -H -p <PID>` 查看该进程下所有线程，按 CPU 使用率排序，找出最耗资源的线程。记下线程 ID，转成 16 进制（比如 `printf "%x\n" <TID>`）。

接着用 `jstack <PID> > jstack.log` 抓线程栈，搜索刚才的 16 进制线程 ID，定位到具体的 Java 线程状态和调用栈。常见罪魁祸首是死循环、频繁 GC、锁竞争或者正则匹配这种 CPU 密集操作。

如果是频繁 Full GC 导致的负载高，配合 `jstat -gcutil <PID> 1000` 观察 GC 频率和耗时。再用 `jmap -histo:live <PID>` 或生成堆 dump 分析内存泄漏点。

代码里常见的坑比如：用 `while(true)` 没有 `sleep`，或者用 `String.replaceAll` 走了正则引擎，正则表达式写得不好容易引发回溯爆炸，像 `(a+)+$` 这种输入恶意字符串能直接打满 CPU。

线上别乱执行 `jmap` 全量 dump，可能触发长时间 STW，服务直接不可用。优先用 `jcmd <PID> VM.native_memory` 或 `async-profiler` 这类低开销工具。



现在有 40 亿个 QQ 号，给你 1G 的内存，如何实现去重？

40亿个QQ号去重，内存只有1G，直接用HashSet肯定搞不定。一个int是4字节，40亿就是16GB数据，远远超过可用内存。

这时候得换思路，**布隆过滤器**就派上用场了。它用很小的空间判断元素是否存在，允许少量误判（比如把不存在的当成存在），但不会漏判。

我们来算笔账：1G内存是85亿bit。如果想存40亿个QQ号，每个元素分配约2.1个bit。查表可知，当哈希函数个数为2时，误判率大概在几%左右，能接受。

布隆过滤器底层就是一个大位数组 + 多个哈希函数。每次添加时，算出多个位置并置1；查询时所有位都是1才认为存在。

```
// 伪代码示意
class BloomFilter {
    private BitSet bits;
    private int[] hashSeeds;
```

```

public void add(long qq) {
    for (int seed : hashSeeds) {
        int index = hash(qq, seed);
        bits.set(index);
    }
}

public boolean mightContain(long qq) {
    for (int seed : hashSeeds) {
        int index = hash(qq, seed);
        if (!bits.get(index)) return false;
    }
    return true; // 可能存在
}
}

```

实际可以用Redis的 `RedisBloom` 模块，或者Google Guava的BloomFilter实现。

整个过程不需要把原始数据全放内存，压根不经过磁盘排序，效率高得多。不过最终精确去重还得结合外部存储做二次验证。

MySQL 里有 2000w 数据，Redis 中只存 20w 的数据，如何保证 Redis 中的数据都是热点数据？

这个问题其实在考察缓存淘汰策略和热点数据识别的结合机制。Redis 本身不会自动知道哪些是热点数据，得靠合理的过期策略和访问模式来驱动。

1) 设置合理的 TTL 过期时间

给 Redis 里的数据设置一个相对短的过期时间，比如 5 到 10 分钟。这样只有被频繁访问的数据才会在过期前被不断续命，不常访问的自然就被淘汰了。

2) 配合 LFU 淘汰策略

Redis 4.0 之后支持 **LFU** (Least Frequently Used) 模式，比起 LRU 更适合识别“高频访问”的数据。可以配置 `maxmemory-policy` 为 `volatile-lfu` 或 `allkeys-lfu`，让访问频率高的 key 更容易留下来。

3) 主动预热 + 延迟双删

上线前把已知的热门数据提前加载进 Redis，比如通过日志分析出过去 24 小时访问最多的 20w 商品，用脚本导入。后续靠实时访问维持热度。

4) 监控访问频次，动态调整

可以用旁路统计模块，比如用 Flink 实时分析业务日志，定期把访问 Top N 的 key 写回 Redis，同时淘汰低频 key。像微博热搜榜这种场景，就是靠实时计算驱动缓存更新。

代码上很简单，核心是读操作的封装：

```

String data = redis.get(key);
if (data == null) {
    data = db.query(key);
    redis.setex(key, 300, data); // 5分钟过期
}

```

整个过程其实是“TTL + 访问驱动 + 淘汰策略”三者配合的结果，压根不靠人工指定，而是让系统自己跑出热点。

系统上线后，发现某个接口响应很慢，你如何定位可能的原因，以及对应的解决思路？

接口变慢不是单一问题，得从链路一层层往下摸。先看是不是外部流量突增，比如被刷了，这种情况加个限流就行，像用 Sentinel 或者 Hystrix 挡住异常请求。

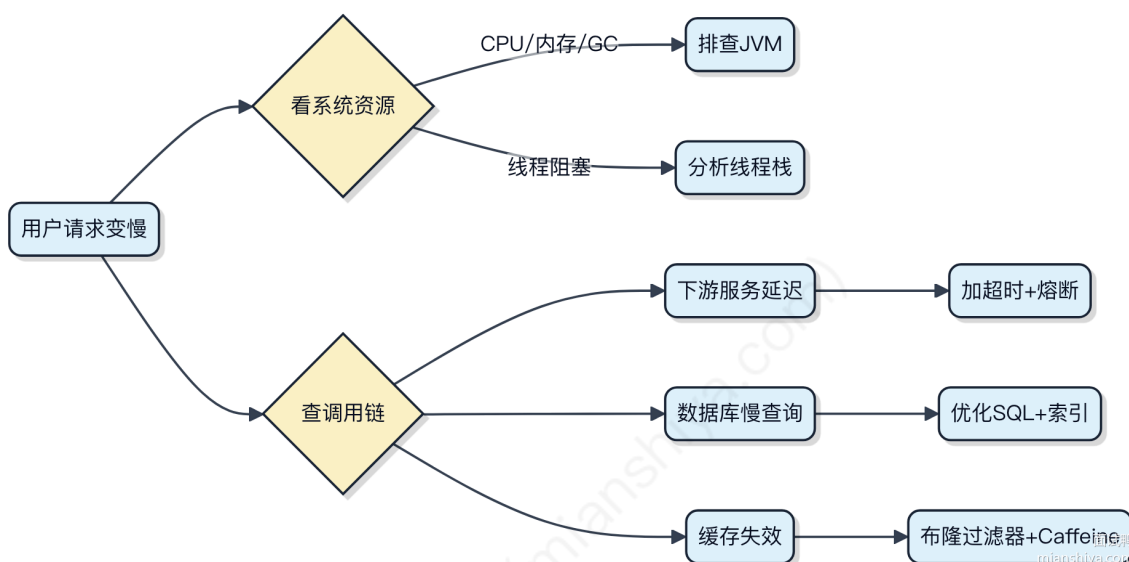
1) 先看监控大盘，CPU、内存、GC 频率有没有异常。如果 GC 太频繁，可能堆内存不够或者有对象泄漏，jstat 和 jmap 抓一下看看。线程数飙高的话，用 jstack 看是不是线程卡在某个方法上，比如数据库查询没索引，死锁，或者远程调用超时没降级。

2) 接着查依赖服务。你的接口可能调了下游的 HTTP 或 RPC，对方一抖你就跟着抖。用 SkyWalking 或 Prometheus 查调用链，看耗时是不是集中在某一个远程调用上。如果是，得和对方对齐 SLA，同时你这边要做超时控制和熔断。

3) 数据库也是重灾区。慢查询日志打开看看，有没有全表扫描。执行计划里 type 是 ALL 就得优化，加索引或者改写 SQL。连接池也得盯紧，HikariCP 的 maxPoolSize 别设太大，不然 DB 压力扛不住，一般 20-50 足够。

4) 缓存穿透、雪崩也会导致 DB 被打满。比如大量请求查不存在的 key，Redis 没命中直接压到数据库。布隆过滤器挡一下非法 key，热点 key 加本地缓存，比如 Caffeine。

最后别忘了应用本身的问题。比如循环里发 SQL，或者用了 synchronized 锁整块逻辑，QPS 上去就卡死。代码 review + APM 工具能快速定位这类低级错误。



现在手头上有一个单体项目，系统的整体 QPS 到了 1 万了，要微服务化拆分吗？

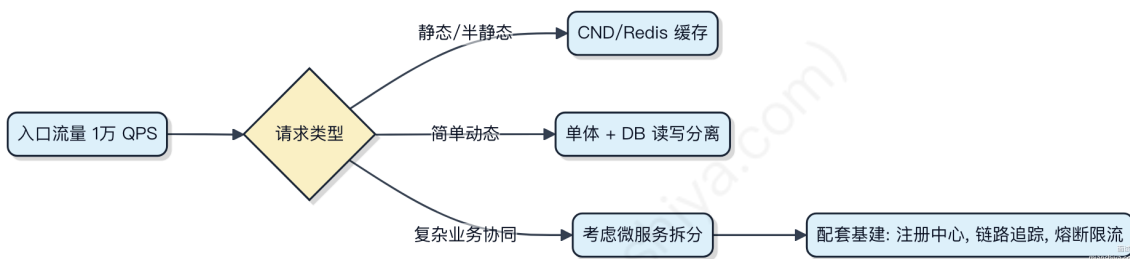
单体应用能不能扛住 1 万 QPS，关键看它干了啥。如果是简单的 CRUD，加个缓存、数据库读写分离，**垂直拆分**一下 web 和 db 层，很可能就扛住了。很多公司几十万 QPS 的流量，早期靠优化单体 + CDN + 缓存 + 异步化撑了好几年。

微服务拆分不是为了应对 QPS，而是为了解耦业务、提升迭代效率。当你发现团队协作开始打架，改一个功能要十几个人联调，发布周期越来越长，这时候才真正需要拆。

盲目拆分反而会引入新问题。服务一多，网络调用复杂度指数级上升，得立刻上链路追踪（比如 SkyWalking）、配置中心（Nacos）、服务治理（Sentinel），还有分布式事务的坑。这些中间件本身就要投入维护成本。

真要拆，建议先做 **逻辑拆分**：把代码按业务域划成模块，但还部署在一个进程里。通过包隔离和依赖管控，提前暴露调用关系，等设计清晰了再物理拆。

另外，1 万 QPS 如果集中在某个热点数据上，比如秒杀场景，那更该做的可能是本地缓存 + 分布式锁 + 热点探测，而不是拆微服务。像阿里早期“聚石塔”架构，就是靠单体极致优化扛下来的。

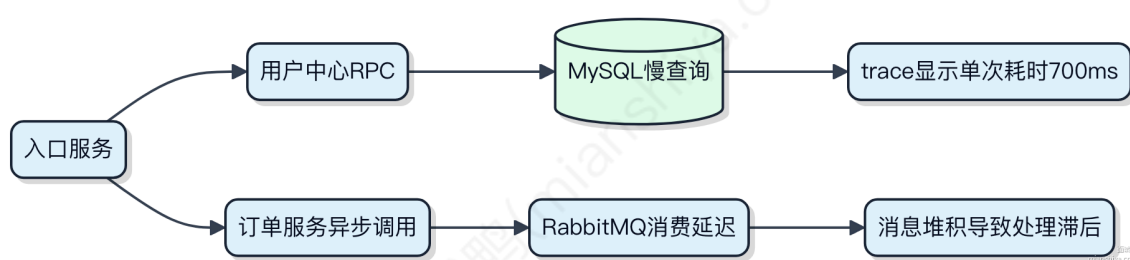


一句话：性能瓶颈优先用缓存、异步、读写分离解决，架构腐化才用微服务解耦。

横跨十几个分布式服务的慢请求要如何排查？

排查横跨多个服务的慢请求，关键在于**链路追踪**和分段耗时分析。靠日志时间戳对齐基本搞不定，尤其在调用深度大、异步任务多的场景下。

- 1) 先确认是否具备完整的分布式追踪能力。像 SkyWalking、Pinpoint 或 Zipkin 这类工具必须提前接入，它们能自动生成 traceId 并透传到下游服务。没有这个底子，排查就是盲人摸象。
- 2) 拿到慢请求的 traceId，在 UI 上直接查看全链路拓扑。重点看两个指标：每个 span 的耗时、是否存在串行等待或并行瓶颈。比如某个服务 A 调用 B 和 C，结果发现 C 耗时 800ms，B 才 50ms，那问题大概率出在 C 或其依赖方。
- 3) 结合 metrics 和日志下钻。如果发现是数据库慢，就去查 MySQL 的 slow log；如果是 RPC 慢，检查 Dubbo 或 Spring Cloud 的调用延迟监控。别忘了线程池满、连接池打满这些常见资源瓶颈。
- 4) 特别注意异步场景的上下文透传。CompletableFuture、MQ 消费等容易断链，需要手动传递 traceId，否则追踪图会断裂。



最终定位通常是某一个“高延迟节点”引发雪崩式影响。平时就得把 tracing + metrics + logging 三者打通，不然线上一出问题，根本扛不住排查压力。

项目用了 Redis 缓存来提升并发度，假设 Redis 挂了怎么办？

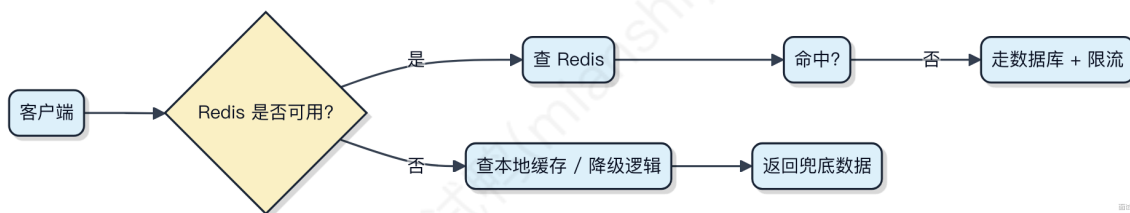
系统设计里引入 Redis，图的就是高性能和高并发。但一旦 Redis 挂了，直接打到数据库的请求可能瞬间涨 10 倍以上，MySQL 根本扛不住。

- 1) 最基础的是加 **降级开关**。比如用 Apollo 或 Nacos 配置中心设个开关，Redis 异常时，让接口直接返回缓存默认值或走本地缓存（Caffeine），先把核心链路保住了。

2) 缓存穿透保护不能少。Redis 挂了之后，所有查缓存的请求都会穿透到底层 DB。这时候布隆过滤器压根没用，因为数据根本没地方加载。得靠**请求合并**或者**信号量限流**，控制打到 DB 的并发数，比如用 Semaphore 控制最多 50 个线程能查库。

3) 持久化策略也得提前想好。如果用了 AOF everysec，宕机最多丢 1 秒数据，重启后还能恢复。但如果是纯内存模式，一挂全丢，恢复时间更长。

4) 架构上，可以考虑多级缓存。比如本地缓存 + Redis 集群。Redis 宕机时，短时间还能靠本地缓存顶一阵子，特别是热点数据。



别指望 Redis 永远在线，设计之初就要按“它随时会挂”来搞。抗住故障比追求性能更重要。

现在我让你把单体服务拆成多个微服务，这些服务之间如何自动发现彼此？具体原理是什么？

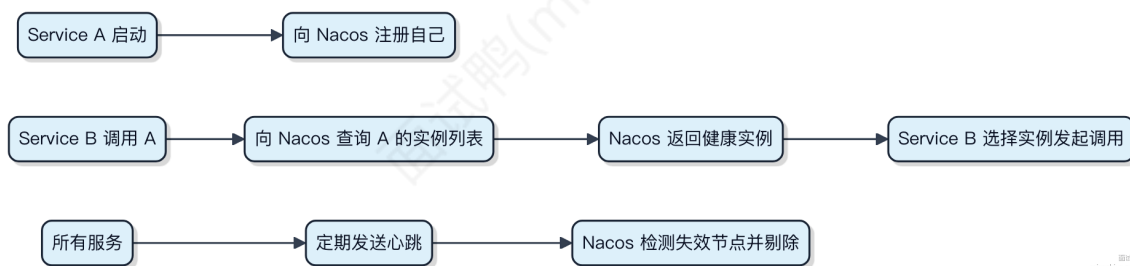
服务拆成微服务后，最头疼的就是服务之间怎么找到对方。以前单体里一个方法调用的事，现在变成跨网络调用了，IP 和端口随时可能变，靠写死配置根本搞不定。

核心是引入 **服务注册与发现** 机制。每个微服务启动时，主动去注册中心报到，把自己的服务名、IP、端口、健康状态这些信息写进去。这个过程叫服务注册。

别的服务想调用它，不去直接连 IP，而是先问注册中心：我要调用 order-service，现在哪个实例活着？注册中心返回可用的节点列表，调用方再用负载均衡策略选一个发请求。这就是服务发现。

关键是有心跳机制保活。注册中心不是静态存一下就完了，服务要定期发心跳证明自己还活着。超过一定时间没心跳，注册中心就把这个实例踢下线，避免别人调到宕机节点。

常见实现像 Eureka、Consul、Nacos 都是干这活的。比如 Nacos，你加个依赖，配个地址，启动自动注册，其他服务用 @LoadBalanced 的 RestTemplate 或 Feign 直接按服务名调用，背后的发现和负载均衡全透明了。

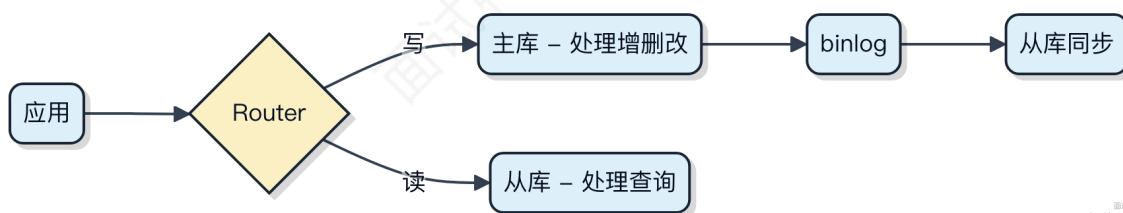


别忘了网络不是百分百可靠，所以客户端一般会缓存一份服务列表，就算注册中心挂了，还能靠本地缓存撑一阵子。

暂时先不考虑分库分表，如果单表的数据量很大，如何优化？

单表数据量大，本质是查询和写入的性能会随着数据膨胀而急剧下降。优化得从存储结构、索引策略、SQL 设计这几个层面入手。

- 1) 索引设计要精准。避免全表扫描，核心是让查询走索引。但索引不是越多越好，每多一个索引，写入时就要多维护一棵 B+ 树。像 MySQL 的二级索引带主键回表，如果主键太大，回表成本就高，所以主键尽量用自增整型。
- 2) SQL 要避免 select *，只查需要的字段，减少 IO 和网络传输。分页查询别用 `limit 1000000, 10`，这种偏移量大会导致 MySQL 扫描大量丢弃的数据。可以改成记录上次查询位置，用 `where id > last_id limit 10` 来翻页。
- 3) 大字段要拆出去。像用户表里放了个 text 存个人简介，这种字段会拖慢整表查询。应该单独建个扩展表，用主键关联，把大字段“拎出来”。
- 4) 冷热数据分离。比如订单表，最近 3 个月的查得多，一年前的基本不看了。可以把历史数据归档到另一个表，或者定期迁移到 Hive 这类大数据平台，主表只留热点数据。
- 5) 读写分离。写主库，读走从库，把压力分散开。像 MySQL 主从同步 + ShardingSphere 读写路由，能有效分担查询压力。



其实很多问题在数据量到百万行之前都看不出来，到了千万级就得动真格了。压根不经过规划的表结构，后期搞不定。

Java 应用的内存持续性增长，但是监控显示堆内存没有什么变化，可能的原因有哪些？

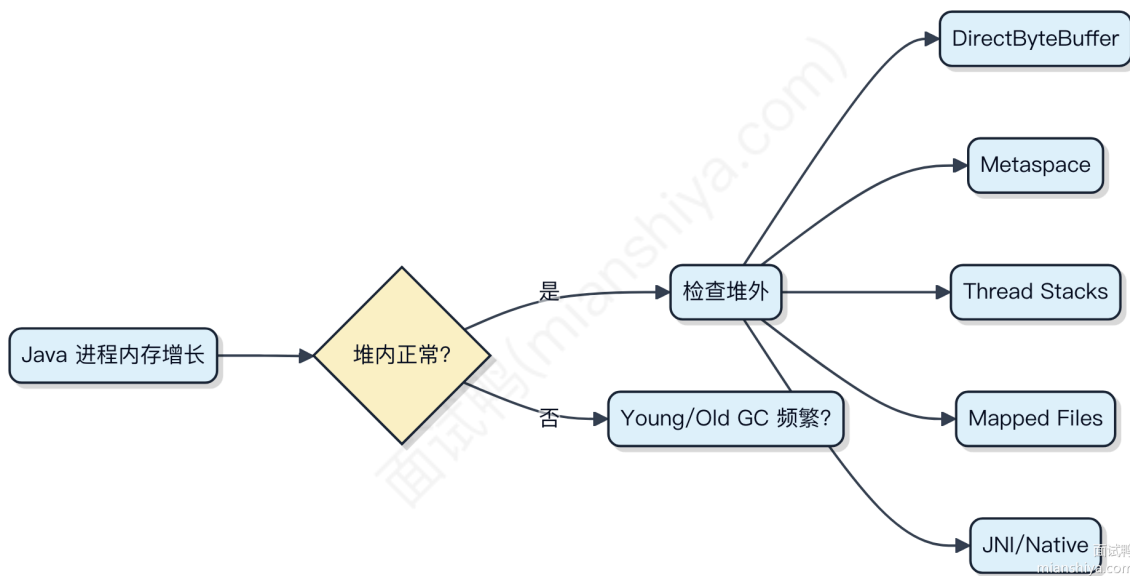
这种情况其实挺典型的，堆内存没涨，但整个进程的内存一直在涨，问题大概率出在堆外内存或者直接内存上。

- 1) 最常见的就是 **DirectByteBuffer** 的滥用。NIO 用得多的应用，比如 Netty、RocketMQ 这些，频繁申请堆外内存做网络传输，如果没控制好大小或回收不及时，Direct Memory 就会持续增长。虽然它不受 GC 直接管理，但会在 Old GC 或 Full GC 时触发释放，但如果 GC 不频繁，那这部分内存就一直占着。
- 2) JNI 调用或者本地库导致的内存泄漏。比如用了某些 native 的压缩库、图像处理库，它们在 C/C++ 层 malloc 了内存但没 free，Java 的监控压根看不到这部分。
- 3) 元空间（Metaspace）膨胀。动态生成类太多，比如大量使用 CGLIB、ASM、Groovy 脚本，每个类信息都存在 Metaspace，虽然默认无上限，但操作系统内存是有限的。可以加 `-XX:MaxMetaspaceSize` 控一下。
- 4) 线程导致的内存上涨。每个线程栈默认 1M（-Xss 控制），如果线程数飙到上千，光栈内存就能吃掉几个 G。常见于线程池配置不合理，比如用 `Executors.newCachedThreadPool()` 又不做限制。
- 5) 内存映射文件（MappedByteBuffer）也可能搞事情。用 `FileChannel.map()` 映射大文件后，这块内存不会轻易释放，而且算在虚拟内存里，top 看着很高，但 RSS 也可能跟着涨。

代码示例注意点：

```
// 避免频繁创建 DirectBuffer
ByteBuffer buffer = ByteBuffer.allocateDirect(1024 * 1024); // 每次都申请 1M 堆外
// 应该复用或者用池
```

排查建议：先用 `pmap -x <pid>` 看内存段分布，找增长的 anon 区；再用 Native Memory Tracking (NMT) 打开看看 Java 各部分内存明细。



用 Bitmap 来存储 100 个用户 ID，但是这个 ID 的范围很大，此时会有什么问题？如何解决？

ID 范围大但实际只存 100 个用户，直接用传统 Bitmap 会浪费严重。比如 ID 上到 10 亿，就得开 10 亿位的数组，哪怕只存 100 个，内存占用也是 GB 级，**空间效率极低**。

问题本质是稀疏数据场景下，稠密存储结构不适用。这时候得换数据结构。

一种解法是改用 Roaring Bitmap。它把大范围的整数拆成高 16 位作为 key，低 16 位存在对应容器里。如果某个区间只有几个值，就用短数组存；密集了再切到 bitmap。这样既能压缩空间，又能保持位运算优势。

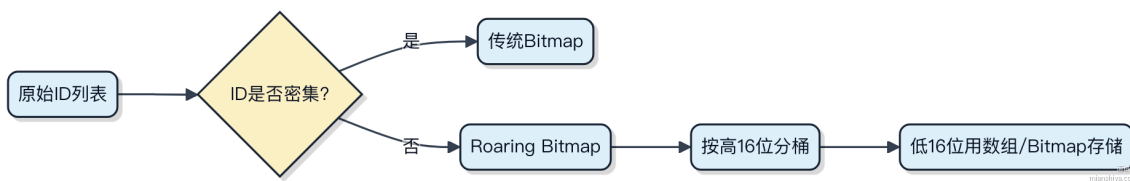
比如你存 ID 为 [1, 100_0000, 200_0000]，传统 Bitmap 得开 200 万 bit 以上，而 Roaring 只存三个 key-value 对，每个值是个小容器，总内存可能就几 KB。

代码上看，Java 里可以直接用 `org.roaringbitmap.RoaringBitmap`：

```
RoaringBitmap bitmap = RoaringBitmap.bitmapOf(1, 100_0000, 200_0000);
boolean contains = bitmap.contains(100_0000); // true
```

它在 Elasticsearch、Druid 这些系统里都用得很多，就是为了解决这种稀疏 ID 集合的高效存储和交并计算。

- 1) ID 稀疏时避免用原生 Bitmap
- 2) 优先考虑 Roaring Bitmap 这类动态压缩结构
- 3) 注意它对连续 ID 更友好，极端稀疏时可结合跳表或哈希



当前有个本地操作 A，远程操作 B，我需要保证 A 和 B 事务的一致性，你会如何实现？

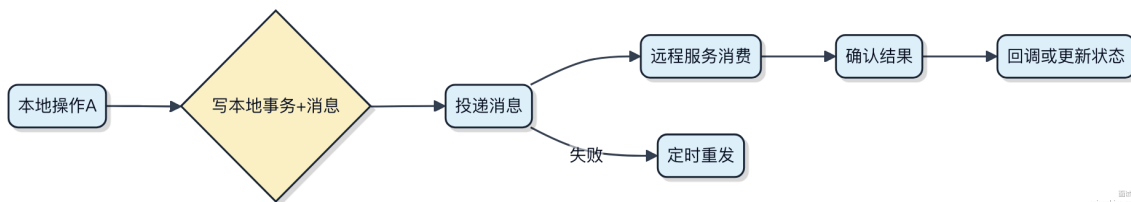
本地事务和远程操作要保证一致性，本质上是分布式事务问题。单靠本地数据库的事务压根不经过远程服务，没法直接回滚。

常见的做法是引入补偿机制或者使用最终一致性方案。TCC 是个典型解法，它分三步：Try 阶段先把 A 和 B 涉及的资源都预占住，比如扣库存加冻结标记；Confirm 阶段真正提交，把预占转正；如果中间出错就执行 Cancel，把预占释放掉。整个过程靠全局事务 ID 串联，状态机驱动。

但 TCC 对业务侵入大，写起来费劲。另一种更轻量的是基于消息队列做可靠事件。比如你先在本地数据库写一条事务记录，包含操作 B 的待发消息，然后通过定时任务或 binlog 监听把这个消息发到 RocketMQ 或 Kafka。消费端接到后执行远程调用，成功则标记完成。失败就重试，直到成功为止。这样本地事务和消息发送是一体的，只要消息发出去了，最终一定能触发 B。

- 1) 要么接受短暂不一致，用消息队列推着走
- 2) 要么上 TCC 强一致，但开发和回滚逻辑复杂得多

小公司搞不定 Seata 可以自己用数据库 + 消息表 + 扫描补发来实现，关键是幂等性必须做好，不然重复执行就炸了。



把单体项目进行了多机部署，多台服务器是如何共享用户登录信息的？

用户登录信息本质上是服务端生成的会话状态，多机部署后，请求可能打到任意一台机器，如果状态只存在本地，就会出现“登录了却提示未登录”的问题。解决思路就是把会话状态从本地剥离，集中管理。

- 1) 最直接的办法是使用 **Redis** 做分布式会话存储。用户登录后，把 session 数据写入 Redis，后续请求通过 Cookie 中的 sessionId 去 Redis 查找。这样任何一台服务器都能获取到登录状态。Spring Session + Redis 就是典型实现，应用服务器完全无状态。
- 2) 也可以用 JWT (JSON Web Token)，把用户信息签在 token 里，客户端每次请求携带。服务端无须存储，通过验签就能识别用户。但 token 一旦签发无法中途撤销，适合对实时权限控制要求不高的场景。
- 3) 负载均衡层做 IP Hash 转发也能保证同一个客户端固定访问一台服务器，会话保留在内存就行。但这破坏了负载均衡的随机性，某台机器挂了会导致一批用户会话丢失，一般只作为临时方案。

真正生产环境，尤其是微服务架构下，**Redis 集中存储**是主流选择，像淘宝、美团这种大流量系统都是这么扛住的。它既保证了高可用，又能配合过期策略自动清理无效会话，脏活累活都交给中间件处理。



一条 1s 请求的响应，怎么优化成 1ms?

把 1 秒的请求压到 1 毫秒，本质是**层层击破性能瓶颈**，不能只靠单一手段。关键是从网络、计算、存储、架构四个层面系统性优化。

1) 网络层：能省则省

跨机房 RTT 一般在 20~50ms，公网 HTTPS 握手要 3 次往返，光建连就几十毫秒。解决方案是接入 CDN 或边缘计算，比如用阿里云 EdgeRoutine 把静态内容推到离用户 10ms 内的节点。长连接复用、HTTP/2 多路复用也能减少频繁建连开销。

2) 计算层：别让 CPU 成短板

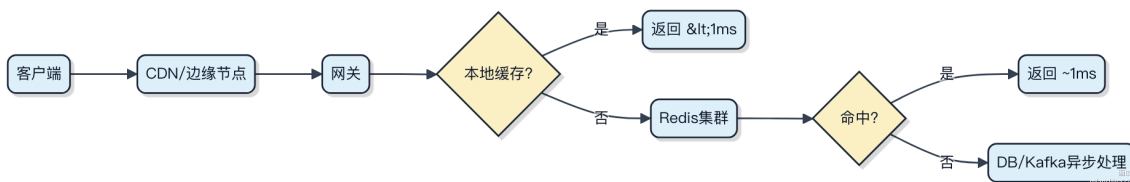
应用代码里避免同步阻塞调用，比如一个 for 循环查数据库 10 次，改成批量查询或异步并行。Spring WebFlux + Reactor 可以把吞吐从几百 QPS 提到几万。但要注意响应式编程的学习成本，不是所有场景都值得上。

3) 存储层：缓存为王

数据库一次主从延迟可能就 1050ms，Redis 集群 P99 延迟通常在 ±2ms。高频读场景必须上多级缓存，比如本地 Caffeine 缓存热点数据（访问速度 <100μs），穿透再走 Redis。写操作可考虑先写内存队列（如 Kafka），异步落库。

4) 架构层：预计算 + 服务治理

像推荐系统这种复杂逻辑，提前用 Flink 做好特征预计算，接口直接读结果。同时做好限流（Sentinel）、降级、熔断，避免雪崩拖慢整体响应。



WebSocket 知道吗？如果让你设计一个 IM 协议，你会考虑什么？

WebSocket 本质是 TCP 之上的全双工通信协议，和 HTTP 一样走 80/443 端口，但握手之后连接一直开着，客户端和服务端都能随时发数据。这特别适合需要实时双向通信的场景，比如聊天、直播弹幕、在线协作文档。

设计一个 IM 协议，光靠 WebSocket 连接还不够，得在它之上构建一整套消息体系。首先考虑**消息可靠性**，要支持 ACK 确认、重试机制，像微信用的 MMTP 就是基于自研协议保证不丢消息。其次要考虑**消息顺序**，同一个会话里的消息必须有序到达，可以给每条消息加 sequence ID，服务端做排序再投递。

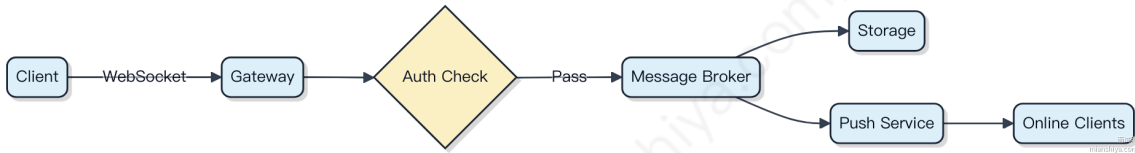
- 1) 连接管理要做心跳保活，一般 30 秒一次，超时 3 次就认为断开
- 2) 消息类型要分清楚：单聊、群聊、系统通知，带上 type 字段区分
- 3) 离线消息通过用户状态 + 消息持久化解决，用户上线后拉取未读
- 4) 安全方面必须有鉴权，比如连接时带上 token，防止伪造身份

代码上，服务端可以用 Netty 接 WebSocket，收到消息后封装成内部 Message 对象：

```
public class Message {
    String from;
    String to;
    String content;
    long timestamp;
```

```
int msgType;
}
```

然后交给消息队列（比如 Kafka）异步处理存储和路由，避免阻塞 IO 线程。



整个架构要能水平扩展，网关无状态，消息存储按用户 ID 分片，这样才能扛住百万级并发。

现在让你负责一个核心系统，需要实现核心数据的缓存预热，如何做？

缓存预热不是简单地把数据从数据库 load 一遍扔进 Redis 就完事，关键是要解决**时机**、**范围**、**一致性**这三个问题。

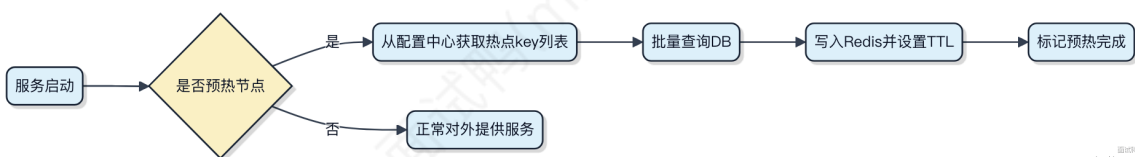
- 1) 时机上，一般在系统低峰期触发，比如凌晨发布后自动执行。可以用 Spring 的 `@PostConstruct` 或者结合 xxl-job 这类调度平台，在服务启动后异步加载。
- 2) 范围要精准，不可能全量加载。通常只加载高频访问的热点数据，比如电商里的爆款商品、用户中心的 TOP 1000 用户信息。可以基于历史请求日志做离线分析，生成热点 key 列表，预热时只 load 这部分。
- 3) 数据一致性得兜底。预热完成后不能放任不管，后续走正常的读写穿透逻辑。写操作依然要更新 DB 并失效缓存，避免预热后又被脏数据覆盖。TTL 也要合理设置，防止长期不更新变成“僵尸缓存”。

代码上很简单，核心就是个批量加载过程：

```
List<Data> hotData = dataMapper.selectByHotKeys(hotKeyList);
hotData.forEach(d -> redis.set(d.getKey(), d, 3600));
```

但真正难的是怎么确定哪些是 hot key。这块可以借助监控埋点 + Flink 实时计算热度，把结果存到配置中心，预热任务启动时先拉取最新热点清单。

整个流程可以用一个简单的状态流转来描述：



当客户端有重连机制，当服务意外宕机重新部署启动时，怎么避免流量洪峰？

服务重启后瞬时流量打满，很容易把刚启动的服务直接干趴，尤其客户端有自动重连机制时更明显。大量客户端在同一时间发起连接和请求，形成“雪崩式”重连风暴。

解决这个问题的关键不是让客户端不重连，而是控制**重连的节奏和请求的放行时机**。

- 1) 客户端做指数退避重连

别一断就连，第一次等 1 秒，第二次 2 秒，第三次 4 秒，最多退到 30 秒。像 Netty 的 `ExponentialBackOff` 就能

直接用。

2) 服务端启动后先不对外暴露

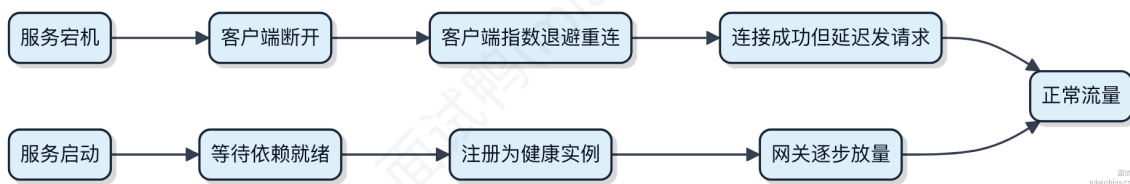
比如在 Nacos/Eureka 注册时, 先标记为非健康状态, 等内部组件 (数据库连接、缓存预热、配置加载) 都 ready 了再注册上线。Spring Boot 的 `/actuator/health` 配合就绪探针就能实现。

3) 网关层做全局限流缓冲

比如用 Sentinel 或 Hystrix, 在服务刚上线的前 30 秒只放行 20% 的请求, 逐步放开。也可以让网关临时返回 429 给部分重连请求, 让客户端错峰。

4) 客户端连接建立后别立马发业务请求

握手成功后先 sleep 一小会儿, 或者等收到服务端的“允许请求”信号再开始发数据。



这套组合拳下来, 基本能把尖刺流量削平, 让系统稳住。

第三方上游接口接入做异步处理, 是否可以使用 MQ? 主要考虑什么? 如果不用 MQ, 还有什么方式?

用 MQ 接第三方上游接口其实很常见, 关键是看场景是否匹配。异步解耦、削峰填谷是它的强项, 但不是所有情况都适合。

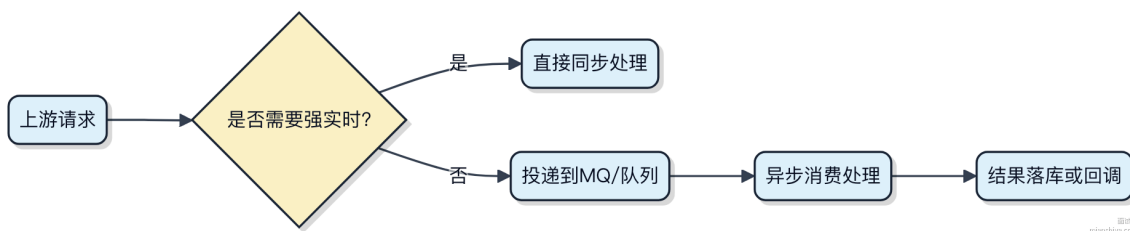
1) 用 MQ 的好处很明显。请求进来先发消息就返回, 后端慢慢消费处理, 系统能扛住突发流量。比如对接支付回调, 先把结果扔进 RabbitMQ 或 RocketMQ, 后续再核对订单状态, 避免对方重试压垮你。

2) 但得考虑几个事。首先是**一致性**, 发消息成功不代表处理成功, 得做好消费幂等和失败重试。其次是**延迟**, MQ 天然有积压风险, 如果对方要求秒级响应, 走消息队列可能搞不定。最后是运维成本, Kafka 这类中间件搭起来, 监控、堆积告警、扩容都得跟上。

3) 不用 MQ 也能做异步。最简单的是线程池 + 队列, 比如用 `ThreadPoolExecutor` 提交任务, 适合单体应用内部解耦。但挂了会丢数据, 可靠性差一截。另一个选择是数据库轮询, 把待处理任务写表, 定时任务扫着执行, 像 XXL-JOB 就这么干, 缺点是实时性差。

4) 还有种折中方案, 用 Redis 的 list 当轻量队列, 配合 blpop 和客户端轮询, 比数据库快, 又比正式 MQ 轻。不过得自己处理高可用和持久化问题。

要不要上 MQ, 本质是在**可靠性、复杂度和性能**之间权衡。小流量、低频调用, 别上来就整 Kafka, 容易杀鸡用牛刀。



4C8G和8C16G，这两个配置的服务器你们选哪个？为什么？

服务器选型不能光看核数和内存，得结合业务场景来定。如果是 CPU 密集型任务，比如做实时计算、数据编码、批量处理，那 8C16G 明显更扛得住高负载，尤其是并发线程多的时候，多核优势就出来了。

但很多 Java 服务其实是 **内存密集型** 的，比如跑 Spring Cloud 微服务，堆里塞了大量缓存、对象实例，GC 压力大。这时候 4C8G 虽然核少点，但只要并发不高，8G 内存也够用，反而资源利用率更均衡。盲目上 8C16G 可能造成浪费，尤其在容器化环境下，Pod 规格太大还会影响调度效率。

重点是看 JVM 参数怎么配。比如你用 G1 GC，一般建议堆外留 23G 给元空间和堆外内存，那 8G 机器最多给到 56G 堆，而 16G 可以轻松给到 10G 以上，大堆场景下 Full GC 频率会低不少。

代码上线前一定得压测。你可以先按 4C8G 部署，看看 CPU 使用率是不是经常飙到 70% 以上，或者 GC 停顿是否频繁。如果压测时 CPU 已经打满，说明需要更多核心；如果堆内存不够用，老年代增长太快，才考虑升到 8C16G。

简单说：

- 1) 业务轻、并发低、成本敏感 → 4C8G
- 2) 高并发、大数据处理、常驻内存大 → 8C16G

别一上来就上大配置，容易搞不定资源治理。

Agent 死循环问题有遇到过吗？如何解决？

Agent 死循环通常发生在字节码增强逻辑里，比如用 Java Agent 做监控或 APM 时，不小心把自己 instrument 的类又触发了新的 transform，结果无限加载 class，CPU 直接打满。

最常见的场景是，你在 ClassFileTransformer 里处理某个类的时候，执行了某些代码，这些代码又动态生成类或者触发类加载，而你的 transformer 没做过滤，就会重新进入 transform 方法，形成递归调用。特别是用到 ASM、CGLIB、Javassist 这些框架时很容易中招。

解决办法其实就两条：

- 1) 加 **类名过滤**，直接排除不必要的类，尤其是你 agent 自身的包路径，比如 `com.mycompany.agent.**` 全部 return null 不做处理。
- 2) 用 **本地线程标记** (ThreadLocal) 标记当前是否已经在 transform 流程中，如果是，直接跳过。这个特别关键，因为很多间接类加载是你控制不了的。

```
static final ThreadLocal<Boolean> transforming = new ThreadLocal<>();

public byte[] transform(ClassLoader loader, String className, Class<?>
    classBeingRedefined,
        ProtectionDomain protectionDomain, byte[] classfileBuffer) {
    if (transforming.get() != null) return null;
    try {
        transforming.set(true);
        // 执行实际的字节码修改逻辑
        return doTransform(loader, className, classfileBuffer);
    } finally {
        transforming.remove();
    }
}
```

另外，像 SkyWalking、Pinpoint 这些 APM 工具都遇到过这类问题，它们的源码里都有对应的 guard 机制防止重入。参考它们的实现能少踩很多坑。



如果一个GPU集群的LLM处理能力为1000tokens/s，那1000个用户同时并发访问，响应给每个用户的性能只有1 token/s吗？怎么分析性能瓶颈

不一定每个用户就只能分到 1 token/s，实际性能取决于调度策略、批处理能力和请求的负载特征。

大模型服务不像传统 Web 服务器那样“一个请求占一个线程”，它走的是 **动态批处理 (Dynamic Batching)** 路子。比如用 vLLM、TensorRT-LLM 这类推理框架时，多个用户的请求会被合并成一个 batch 一起跑，显存和计算单元利用率更高。

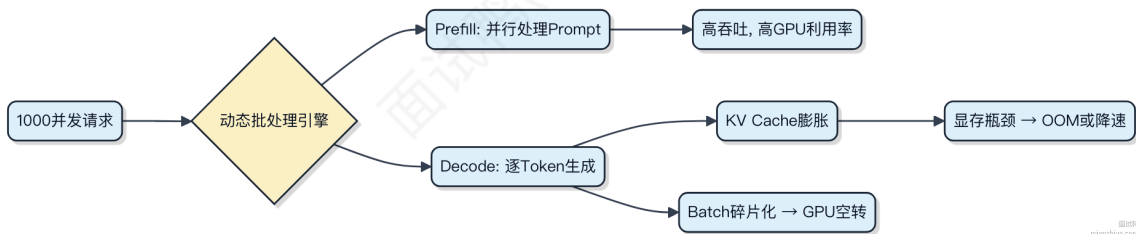
关键点在于：

- 1) 如果这 1000 个用户都是刚发请求，还没开始生成内容（prefill 阶段为主），那系统可以一次性把他们的 prompt 做 batch 处理，吞吐可能接近峰值 1000 tokens/s，平均下来每人几十甚至上百 token/s 都有可能。
- 2) 但到了 decode 阶段，每个请求要逐 token 生成，且长度不一，这时候 batch size 会慢慢被“拖慢”。长文本、高采样参数（如 temperature 高）的请求会拉低整体速度。
- 3) 真正的瓶颈往往不是算力，而是显存带宽和 KV Cache 管理。每个并发请求都要保留 KV Cache，1000 个并发意味着巨大的显存压力，搞不定就会 OOM 或被迫丢请求。

举个例子，A100 显存 80GB，跑 Llama3-8B，每个请求平均占 2GB KV Cache，理论上最多撑 40 个并发。超过就得靠 PagedAttention（像 vLLM 实现的）来分页管理，否则压根不经过计算就挂了。

所以性能瓶颈通常出现在：

- KV Cache 显存占用过大
- 小 batch 或不规则请求导致 GPU 利用率上不去
- 请求排队时间远超生成时间



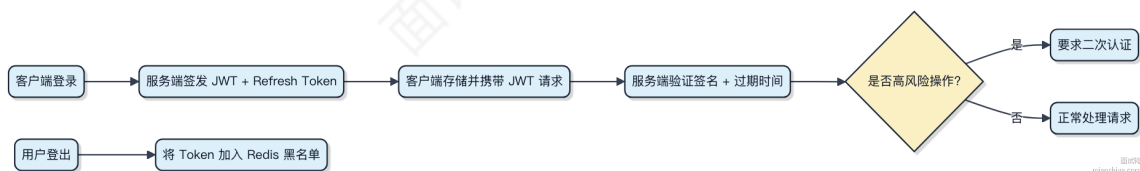
优化方向是：用 PagedAttention 降低显存压力，启用连续批处理（Continuous Batching），控制最大生长度，避免小请求拖垮整体延迟。

你项目用的 JWT，那如何保证安全性？别人拿到你这个 Token 不就可以直接登录你的账号了吗？怎么办？

JWT 本身是无状态的，把信息都放在客户端，所以一旦签发出去，服务端没法直接让它失效。这就带来一个风险，Token 被别人截获了，确实能冒用，而且过期时间之前一直有效。

解决这个问题不能靠 JWT 自己扛，得靠外围机制配合。

- 1) 首先，HTTPS 是底线，传输过程必须加密，防止中间人偷。明文传 Token 压根不经过脑子。
- 2) Token 的过期时间要合理控制，别设成 7 天、30 天这种长周期。一般建议 15 分钟到 2 小时，结合刷新 Token (refresh token) 机制。刷新 Token 存在服务端，可以随时废掉。
- 3) 关键操作比如改密码、删账号，不能只认 JWT，得再做一次认证，比如输入密码或短信验证。相当于高风险动作加一道闸。
- 4) 如果真需要主动让某个 Token 失效，就得引入黑名单机制。可以用 Redis 记录已注销的 Token 或用户 ID，每次请求校验 JWT 同时查一下 Redis。虽然破坏了无状态性，但换来了可控的安全兜底。
- 5) JWT payload 别放敏感信息，比如密码、手机号。签名算法也别用 none，老老实实用 HS256 或 RS256。



总的来说，JWT 安全不是靠它自己，而是组合拳：短过期、HTTPS、刷新机制、敏感操作加固、必要时用 Redis 控制生命周期。

[我们线上软件安装包地址，这几天访问量暴增，现在 CDN 流量一直异常，这是什么原因？如何解决？](#)

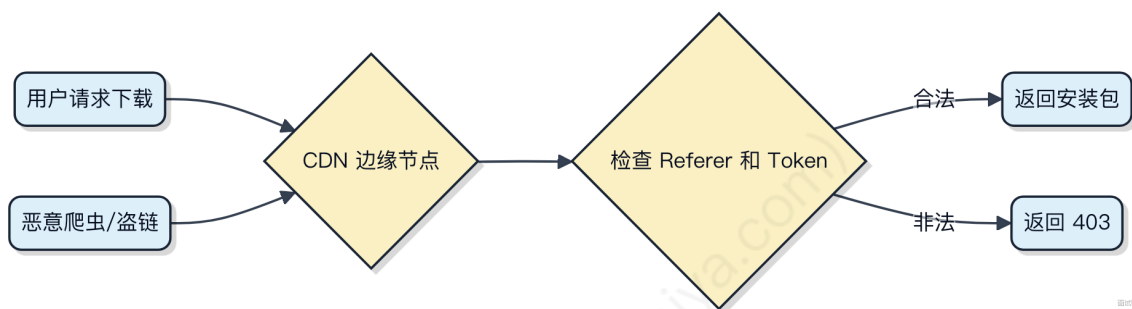
某个软件安装包突然访问量暴增，CDN 流量飙升，这种情况其实挺常见的。先别慌，得搞清楚是“真用户需求”还是“异常流量”。

一般来说，这种突增要么是被大量真实用户传播下载，比如上了热搜或者被大 V 推荐；要么就是被盯上了，比如被恶意爬虫批量拉取，或者被当成了资源站的源进行镜像分发。还有一种可能是盗链，别的网站把你的 CDN 地址直接嵌在页面上当免费图床或下载源用。

排查第一步，看访问日志。重点看 User-Agent、Referer 和 IP 分布。如果发现大量请求来自同一个 IP 段，或者 Referer 是某个陌生域名，基本可以确定是盗链或爬虫。User-Agent 是空或者明显伪造的，也大概率是机器刷的。

解决办法有几个方向：

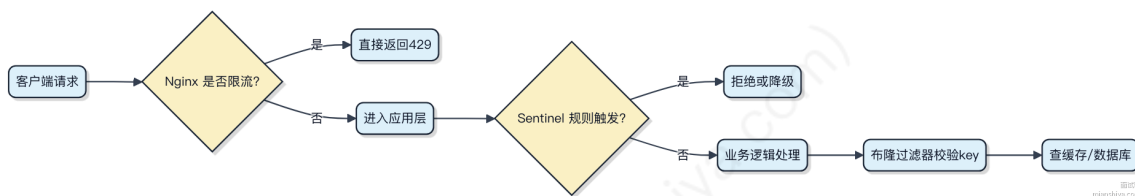
- 1) 如果是盗链，最直接的是在 CDN 侧配置 **Referer 白名单**，只允许你自己的域名发起请求。简单有效，几分钟就能生效。
- 2) 加 **Token 鉴权**，给下载链接加上时效性签名。比如 `https://cdn.example.com/app.apk?token=xxx&expire=1723456789`，过期失效，防止链接被长期传播滥用。
- 3) 对高频 IP 做限速或拦截，CDN 平台一般都支持基于 IP 的速率限制，比如单 IP 每秒超过 10 次就返回 403。
- 4) 如果确认是热点资源，也可以接受流量成本，但建议把静态包挪到独立二级域名，比如 `dl.yourapp.com`，和主站流量隔离，避免影响其他资源。



如果线上接口被恶意刷流量了，要如何解决？

恶意刷流量本质上是系统在短时间内被大量非正常请求冲击，目标是保护接口可用性，同时尽量不影响真实用户。

- 1) 最直接的手段是**限流**，把瞬时流量峰值控制在系统能扛住的范围内。比如用 Redis + Lua 实现分布式令牌桶，单个用户每秒最多放行 5 个请求。超过的直接拒绝，Nginx 层也可以加 limit_req 模块做前置拦截。
- 2) 识别恶意行为得靠**风控规则**。常见的是基于 IP、设备指纹、用户 Token 做频次统计。像 Sentinel 这类组件就内置了热点参数限流，能自动发现异常参数并拦截。如果发现某个 IP 一分钟内调用登录接口超过 100 次，直接拉黑。
- 3) 复杂一点的情况要结合**行为分析**。比如爬虫可能模拟正常访问路径，但节奏特别规律。可以用滑动窗口计算请求熵值，或者引入简单机器学习模型判断是否为真人操作。极验、阿里云风险识别这些服务就是干这个的。
- 4) 底层存储和缓存也得防穿透。恶意请求往往查不存在的数据，压垮数据库。布隆过滤器可以挡掉 90% 以上的非法 key 查询，Redis 缓存也要设好空值过期时间。



关键是分层防御，网络层、应用层、数据层都要有对应策略。单纯靠某一种手段很难搞定高强度攻击。

项目中同样的功能，可能是环境或其他要求原因，需要适配多种数据库，如何实现？

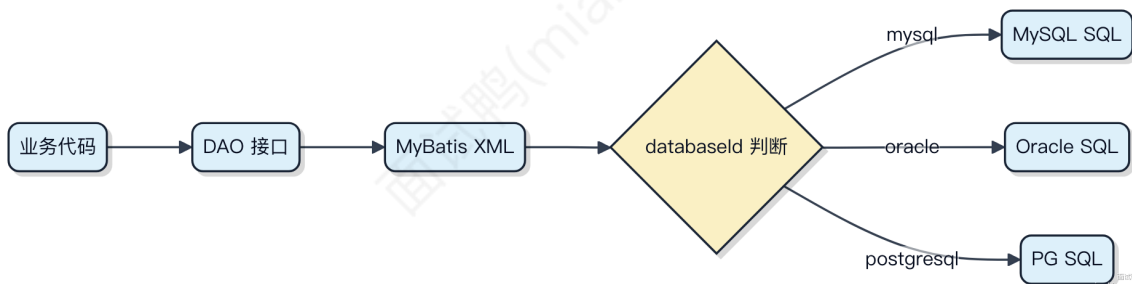
不同数据库之间语法、函数、数据类型这些细节差异不小，硬编码肯定搞不定，后期维护会直接崩。想灵活应对，得靠**抽象隔离和分层设计**。

- 1) 最稳的路子是用 ORM 框架，比如 MyBatis 或 Hibernate。MyBatis 里可以用 `<databaseId>` 标签做方言适配，不同的数据库执行不同的 SQL 片段。Hibernate 更省心，它通过 Dialect 自动处理底层差异，你写 HQL 就行，剩下的交给它翻译。
- 2) 如果绕不开原生 SQL，那就自己封装一层。定义统一的数据访问接口，每个数据库实现各自的 SQL 构造器。比如 MySQLImpl、OracleImpl 都实现 SqlBuilder 接口，运行时根据配置加载对应实现。Spring 的 @Profile 或工厂模式都能搞定这个路由逻辑。
- 3) 别忘了连接池和驱动也得可插拔。用 HikariCP 配合 `spring.datasource.driver-class-name` 和 URL 动态切换，应用启动时自动装对应数据源。

代码示例就一句话体现核心：

```
@Mapper
public interface UserMapper {
    @Select("SELECT * FROM users WHERE id = #{id}")
    User findById(@Param("id") Long id);
}
```

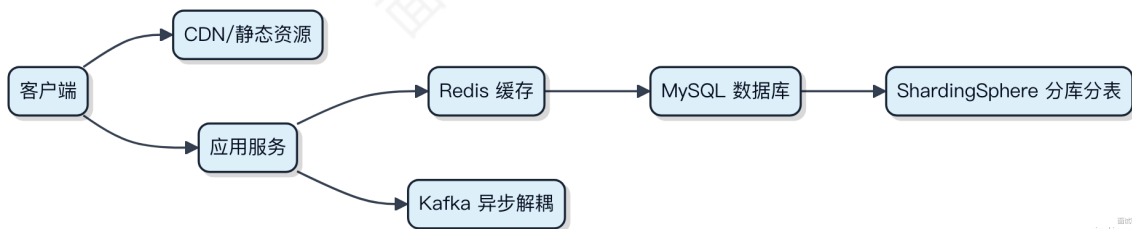
这玩意在 MySQL 和 PostgreSQL 都能跑，前提是 SQL 写的是标准语法。真碰到分页这种坑货，MySQL 用 LIMIT，Oracle 得用 ROWNUM，就得拆成两个 SQL 文件按库加载。



除了代码，还有哪些地方可以优化性能？

性能优化从来不只是改几行代码的事，系统层面、架构设计、资源调度都得拉进来一起看。

- 1) 网络传输能省则省。接口返回别一股脑塞一堆字段，用 Protobuf 代替 JSON 可以让数据体积直接砍掉一半以上，像 gRPC 就是这么干的。CDN 把静态资源甩到离用户最近的节点，首屏加载从 2 秒压到 300 毫秒很常见。
- 2) 数据库慢查询是大头。加索引不是万能药，InnoDB 的聚簇索引结构决定了主键查询最快，二级索引回表代价得掂量。分库分表用 ShardingSphere 或 MyCat 解决单库撑不住的情况，但事务和跨表查询立马变复杂。
- 3) 缓存用好了是神器，用不好就是定时炸弹。Redis 做热点数据缓存，QPS 能扛上万，但得防雪崩、穿透、击穿。本地缓存比如 Caffeine，访问延迟压到微秒级，适合放不变的配置类数据。
- 4) 异步化能扛住突发流量。用户下单后发个消息到 Kafka，后面慢慢扣库存、发短信，响应立马回来。线程池参数调不好反而拖垮服务，核心线程数、队列长度得结合业务 TP99 来算。
- 5) JVM 层也得盯住。Young GC 频率太高说明对象分配过猛，可能是日志打太多或者临时对象没回收。老年代突然涨，十有八九是内存泄漏，MAT 分析 dump 文件才能定位根因。



新项目要上线了，你会关注哪些指标，为什么？

系统上线后得盯着几类关键指标，不然出问题了压根不知道。先说最要命的三个：错误率、延迟和吞吐量。

1) 错误率超过 1% 就得警惕，特别是 HTTP 5xx 或 RPC 调用失败。像用 Spring Cloud 的服务，如果 Feign 调用突然报一堆熔断或超时，说明下游扛不住或者网络有问题。

2) 延迟看 P99 和 P95，别只盯平均值。比如订单接口平均耗时 50ms，但 P99 到 2s，说明有慢请求堆积。配合 SkyWalking 或 Prometheus + Grafana 能定位到具体方法或 SQL。

3) 吞吐量 QPS 得跟压测基线对比。如果比预期低一半，可能是线程池没调好，或者数据库连接不够。Kafka 消费者组 lag 突增也属于这类，消息积压了。

JVM 层面也不能漏。老年代每分钟都在 Full GC，那肯定有内存泄漏。Young GC 频率突增可能对象创建太猛，配合 Arthas 看下堆栈。

最后是业务指标联动。比如支付成功率掉 5%，技术侧发现是调第三方接口超时，这时候不能光等对方恢复，得加降级策略，像走缓存默认值。



面试鸭
mianshiya.com

如何设计一个 OAuth2.0 授权服务？Token 如何管理？

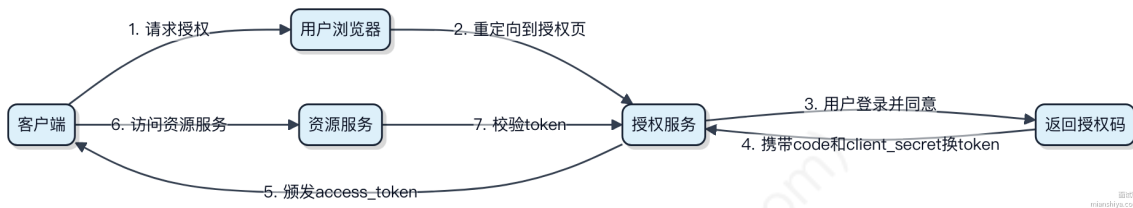
设计一个 OAuth2.0 授权服务，关键在于理清四种角色的协作流程：资源所有者、客户端、授权服务、资源服务。授权服务的核心是安全地颁发 Token，并保证整个流程符合规范。

1) 授权码模式是最常用的流程。用户在授权服务上登录并同意授权，服务会重定向带回一个临时的 authorization_code。客户端拿到这个 code，用它加上自己的 client_secret 去换取 access_token。这个过程里，access_token 不会暴露给浏览器，安全性高。

2) Token 管理方面，一般用 JWT (JSON Web Token)。好处是无状态，把用户信息和过期时间都签在 token 里，资源服务自己就能校验，不用每次都查数据库。但问题在于无法主动失效，所以得配合 Redis 记录黑名单或短期 token + 频繁刷新。

3) 如果用自管理的 token，比如存到数据库或 Redis 的随机字符串 token，则可以随时删除来实现登出或禁用，但每次请求都要查一次存储，压测下来 1k QPS 就可能成为瓶颈。

代码层面，Spring Security OAuth2 或 Spring Authorization Server 能帮你搭起骨架，但生产环境得自己加细节：比如 client_id/client_secret 的安全管理、token 的加密存储、速率限制防 brute-force。



别忘了刷新 token 的机制，避免用户频繁登录。整个链路必须走 HTTPS，否则全白搭。

有人恶意攻击我们的注册功能，伪造手机号大量下发验证码，除了限流之外，如何解决这个问题？

设备和行为分析是关键，光靠限流只能拦住简单攻击。

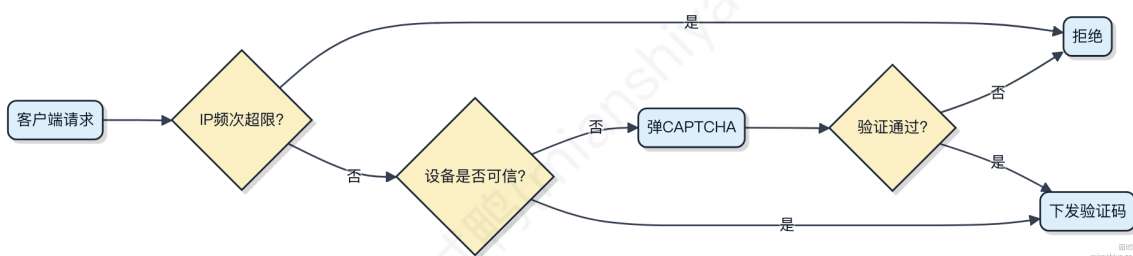
- 1) 同一个 IP 短时间内请求验证码超过 5 次就触发拦截，但攻击者可能用代理 IP，所以不能只看 IP。
- 2) 接入设备指纹，比如前端埋 SDK 收集设备 ID、浏览器特征、网络环境，同一个设备反复请求直接标记。像极验、数美这些风控平台就是干这个的。
- 3) 加入机验证，比如滑动拼图或点选文字，注册前先过一道 CAPTCHA，机器脚本基本搞不定。阿里云、腾讯云都有现成服务。

代码层面可以结合 Redis 记录状态：

```
// 伪代码：基于设备ID+手机号做双重校验
String key = "reg:verify:" + deviceId;
redis.setex(key, 300, "1"); // 5分钟内不允许再发
```

- 4) 用户行为异常也要监控，比如页面停留时间低于 2 秒就提交验证码，大概率是脚本。后端记录埋点日志，交给风控系统打标签。

真正扛住攻击的方案都是组合拳，**设备指纹、行为识别、人机挑战**三者配合，才能让恶意请求压根不经过业务逻辑。



数据库表上新增一个字段，如果这个表正在进行读写操作，如何处理才能不影响现有读写操作？

给线上数据库表加字段，关键是要减少锁表时间，避免把数据库搞崩了。MySQL 5.6 之前 `ALTER TABLE` 基本是全表拷贝，写操作会阻塞，读也可能被拖慢。现在一般用 **Online DDL** 和工具配合来搞定。

- 1) MySQL 5.6+ 支持 `ALGORITHM=INPLACE, LOCK=NONE` 的 DDL 操作，新增字段如果是非空且有默认值，其实是能走 inplace 的，不会重建表。但要注意，即使不重建，加字段本质还是修改表结构元数据，过程中仍可能短暂加 MDL 锁。
- 2) 真正稳妥的做法是结合工具。比如用 `pt-online-schema-change` 或 `gh-ost`，它们原理都是创建一个影子表，加好字段后再逐步把原表数据同步过去，最后原子性替换。整个过程原表可读可写，业务完全无感。

举个例子，用 gh-ost 给 user 表加字段：

```
gh-ost \  
  --host=localhost \  
  --database="test" \  
  --table="user" \  
  --alter="ADD COLUMN ext_info JSON" \  
  --execute
```

它会在后台异步迁移，你可以控制速率、暂停、切流，特别适合大表变更。

小表可以直接走 Online DDL，大表或者核心业务表，建议上 gh-ost 这类工具。别忘了提前在测试环境压测一把，确认不影响性能。

如何快速定位到五分钟内重复登录了两次QQ号，用什么数据结构？

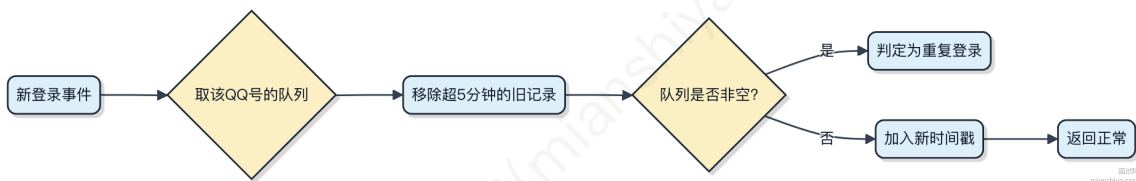
要快速找出五分钟内重复登录的 QQ 号，关键在于**高效记录和查询时间窗口内的行为**。直接用 HashMap 存 QQ 号和登录时间，每次来一个新登录，就去查有没有历史记录在 5 分钟内，这种做法看似可行，但数据会无限增长，也不好清理。

其实更优解是用 **滑动时间窗口 + 双端队列**。每个 QQ 号维护一个双端队列，存它最近几次的登录时间戳。当一个用户再次登录时，先把队列里超过 5 分钟的旧记录从队头踢掉，然后检查队列是否还有元素。如果有，说明他在 5 分钟内登过，触发重复判定；最后把新时间从队尾加进去。

```
Map<String, Deque<Long>> userLogins = new HashMap<>();
```

这种方式每个用户的存储只保留有效时间内的数据，空间可控，判断也快。平均下来，一次登录的处理是 $O(1)$ 。

如果量特别大，比如每天几亿次登录，可以用 Redis 的 Sorted Set，把 QQ 号作为 member，时间戳作为 score，用 ZREMRANGEBYSCORE 清理过期记录，再用 ZCOUNT 判断区间内数量。像 Kafka 登录风控就有类似实践。



公司目前有一些海外业务，从国内将数据发送到海外延迟比较大，如何处理这个问题？

海外链路延迟大，本质是物理距离和网络跳数导致的 RTT 高，TCP 传输效率下降。解决思路不是单一技术，而是分层优化。

1) 应用层做聚合与压缩

减少请求次数比降低单次延迟更有效。比如把多个小数据包合并成批次发送，配合 Gzip 压缩，能显著减少总耗时。像 Kafka 生产者就支持 `linger.ms` 和 `batch.size` 控制批量发送。

2) 传输层选型优化

传统 HTTP/HTTPS 走 TCP，受握手和拥塞控制影响大。可以考虑 QUIC 协议，基于 UDP 实现快速连接建立和多路复用，抗丢包能力强。Cloudflare、Google 已大规模使用。

3) 部署边缘节点或中继服务

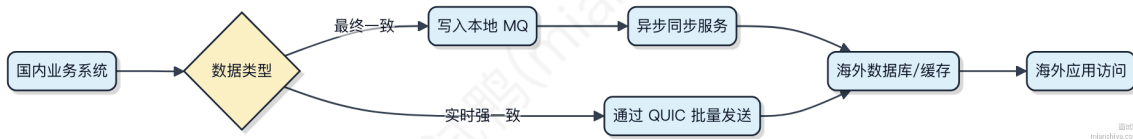
在海外就近部署反向代理或同步服务，比如用 Nginx 或 Envoy 做中转，数据先发到离海外近的区域（如新加坡、东京），再转发目标地，缩短最后一公里。

4) 异步化 + 最终一致性

不是所有数据都要实时同步。引入消息队列，比如 RabbitMQ 或 RocketMQ，先把数据落进本地队列，由后台消费者异步推送，系统间解耦，也能扛住网络抖动。

5) 缓存热点数据

如果存在重复读场景，可以在海外前置 Redis 或 Memcached 缓存副本，减少回源频率。阿里云、AWS 都提供跨域复制方案。



现在有一天内的大量日志，每条日志记录了用户id, 登陆时间, 登出时间 {userid, login_time, logout_time}, 时间单位是秒。怎么求出一天内的最大在线人数？以及维持最大在线人数的最长持续时间？

这个问题本质是区间叠加问题，求的是所有时间点中同时在线人数的峰值，以及这个峰值能维持多久。

- 1) 把每个用户的登录登出事件拆成两个时间点：登录 +1 人，登出 -1 人。
- 2) 按时间顺序排序所有事件点，如果时间相同，先处理登出再处理登录（避免同一秒进出被重复计算）。
- 3) 扫描一遍这些事件，用一个计数器模拟实时在线人数，记录过程中出现的最大值和它连续维持的时间。

```
List<Event> events = new ArrayList<>();
for (Log log : logs) {
    events.add(new Event(log.loginTime, 1));
    events.add(new Event(log.logoutTime, -1));
}
events.sort((a, b) -> {
    if (a.time != b.time) return Long.compare(a.time, b.time);
    return Integer.compare(a.delta, b.delta); // 先-1后+1
});

int online = 0, maxOnline = 0;
long maxStart = -1, maxDuration = 0, currentMaxStart = -1;

for (Event e : events) {
    online += e.delta;
    if (online > maxOnline) {
        maxOnline = online;
        currentMaxStart = e.time;
    } else if (online < maxOnline && currentMaxStart != -1) {
        maxDuration = Math.max(maxDuration, e.time - currentMaxStart);
        currentMaxStart = -1;
    }
}
```

```
}  
}
```

关键在于事件驱动扫描，把二维的“时间段”压平成一维的时间流。整个过程 $O(n \log n)$ ，主要是排序开销。像 Flink 处理窗口聚合时也常用这种思想，把时间切片变成可累加的状态。



主线程如何得知线程池中多个并行任务的完成并收集它们的结果？

主线程要等一堆并行任务跑完再拿结果，靠的是 **Future** 这个契约对象。你往线程池 submit 一个 Callable 或 Runnable，它就返回一个 Future，相当于开了一张“结果欠条”。

1) 用 `ExecutorService.submit()` 提交任务，拿到 `Future<T>` 集合 2) 遍历 Future 调用 `get()` 方法，它会阻塞直到对应任务完成 3) 收集返回值，统一处理

```
List<Future<String>> futures = tasks.stream()  
    .map(executor::submit)  
    .toList();  
  
List<String> results = futures.stream()  
    .map(f -> {  
        try { return f.get(); }  
        catch (Exception e) { throw new RuntimeException(e); }  
    })  
    .toList();
```

但这样是串行 `get()`，前一个卡住后面的也动不了。想一起等，得用 `ExecutorCompletionService`。它内部维护一个完成队列，哪个任务先做完，就先 put 到队列里，主线程 `take` 就行。

```
CompletionService<String> cs = new ExecutorCompletionService<>(executor);  
for (Callable<String> task : tasks) cs.submit(task);  
  
for (int i = 0; i < tasks.size(); i++) {  
    String result = cs.take().get(); // 按完成顺序拿  
}
```

或者直接上 `CompletableFuture.allOf()`，更现代。所有任务包装成 `CompletableFuture`，`allOf` 返回一个总 future，`join` 等待全部完成，再挨个 `get` 结果。

场景选型：

小批量固定任务用 Future + 循环 `get`；

追求响应速度、任务耗时不一，用 `CompletionService`；

复杂编排比如还要做 `thenCombine`、异常处理，直接上 `CompletableFuture`。

在 Linux 命令行敲下一行命令，会进行哪些事情？

你敲下一条命令，比如 `ls -l /home`，系统背后其实走完了一整套流程。整个过程从用户输入到结果输出，涉及 shell 解析、进程创建、系统调用和硬件交互。

shell 先对命令行做分词，拆出程序名 `ls`、参数 `-l` 和 `/home`。然后它去 `$PATH` 环境变量列出的目录里找 `ls` 的可执行文件，一般在 `/bin/ls`。

找到后，shell 调用 `fork()` 创建子进程。这个子进程再调用 `execve()` 把自己替换成 `ls` 程序，加载 ELF 二进制并初始化内存布局。老的 shell 进程就等着它结束。

`execve()` 触发系统调用，进入内核态。内核为新程序设置虚拟内存、堆栈、文件描述符。标准输入输出默认继承自 shell，通常连到终端。

程序运行过程中，如果要读目录，会通过 `getdents()` 系统调用进入内核，VFS 层再转发给 ext4 或 XFS 这类具体文件系统驱动。数据最终通过块设备层到达磁盘或 SSD。

命令执行完，进程调用 `_exit()`，内核释放其资源，父 shell 收到 `SIGCHLD` 信号，调用 `wait()` 获取退出码，然后打印新的提示符，等待下一条命令。

设计数据库表时，关联表和在表中加冗余字段关联各有什么优势？

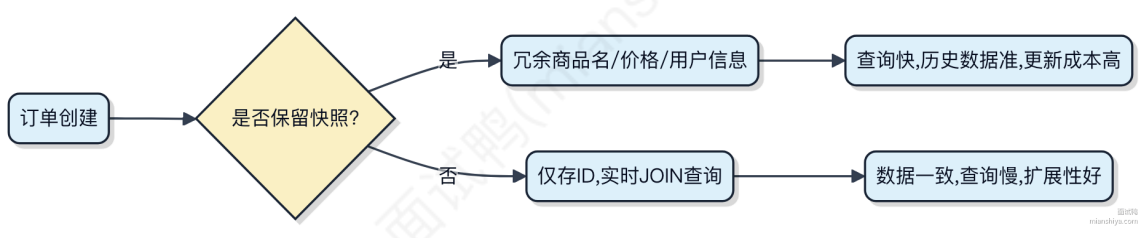
设计表结构时，关联表和冗余字段本质是在**一致性和性能**之间做权衡。

用关联表，比如用户和角色的关系，单独建 `user_role` 表，通过外键关联。这种方式数据完全正范式，改一个角色名，所有地方自动生效，维护起来省心。但查用户带角色信息时就得 join，要是关联三四张表，查询性能明显下降，尤其数据量上来后，索引压得数据库喘不过气。

冗余字段就是反范式做法。比如订单表直接存商品名称，而不是只留个商品 ID。好处是查订单列表时，不用连商品表也能展示名称，一次查询搞定，响应快，数据库压力小。但问题是一旦商品名称改了，所有相关订单里的冗余字段都得同步更新，否则就出现数据不一致。要真这么干，得配合 mq 或 binlog 去异步刷，系统复杂度上来了。

1) 需要强一致性、数据变动频繁的场景，优先用关联表，比如财务、权限体系。2) 读多写少、对响应要求高、且冗余字段变更不频繁的，可以加冗余，比如电商订单里的收货地址快照。

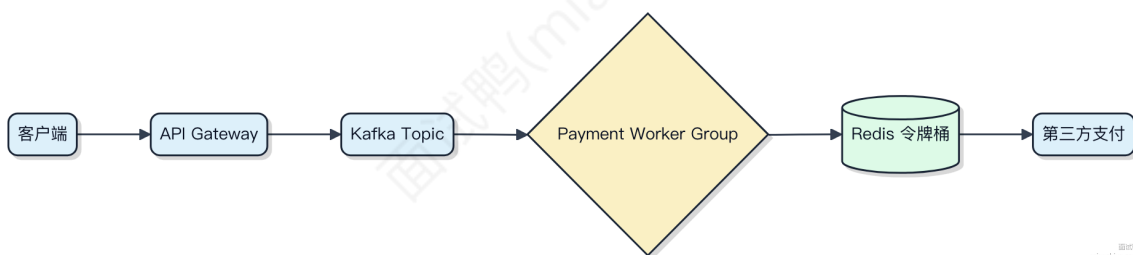
实际项目里，像美团下单、淘宝订单，都会把关键信息做快照冗余，避免事后查数据时因为基础信息修改而对不上账。这种设计不是为了省 join，而是业务逻辑本身要求“记录历史状态”。



每秒 200 笔订单请求到支付服务，支付服务需调用第三方支付，限流 100 笔每秒。请你设计支付服务，要求分布式的，并且是 FIFO，不超过第三方限流量，但是要用满第三方100 笔每秒的量，请问如何设计，需要考虑哪些方面？

系统设计这种题，关键不是一上来就画高大上的图，而是先把约束条件吃透。每秒 200 请求进来，第三方只能扛 100 笔/秒，说明入口和出口不匹配，必须引入缓冲。要求 FIFO、分布式、还要把 100 笔用满，那其实就是个削峰填谷 + 流量整形的场景。

- 1) 核心思路是加个消息队列做缓冲。比如用 Kafka 或 RocketMQ, 订单请求进来直接发到队列, 然后后端起一批支付 Worker 消费, 控制消费速度在 100 条/秒。这样既保证了顺序性 (单个分区内 FIFO), 又不会超限。
- 2) 分布式环境下, 多个 Worker 怎么协同控速? 不能每个都拉 100 条, 那样就 $100 \times N$ 超了。得有个统一的速率控制器。可以用 Redis 实现令牌桶, Worker 每次取任务前先拿令牌, 成功才去队列取数据调第三方。令牌生成速度设为 100/s, 就能精准控流。
- 3) 队列本身要持久化, 避免宕机丢单。Kafka 本身支持多副本, 能扛住节点故障。Worker 做幂等处理, 防止重复支付——毕竟网络可能超时重试。
- 4) 监控必须跟上。实时看队列积压情况、出队速率、失败率。如果发现长期积压, 说明 100/s 是瓶颈, 可能需要申请提额或降级策略。



注意点: 别让 Worker 拉得太猛, 哪怕令牌拿到了也得匀速发, 避免瞬间冲击。还有就是时间窗口统计要用滑动窗口, 别用简单的计数器导致毛刺。

MySQL数据库一亿条数据, 怎么快速加索引?

加索引这事, 关键不是怎么加, 而是怎么**不影响线上服务**。一亿条数据直接 `ALTER TABLE`, 那表锁住几个小时, 业务直接瘫痪。

MySQL 5.6 以后其实有在线 DDL, 像加索引这种操作默认走 `INPLACE` 模式, 不重建表, 只记录变更, 最后再合并。但即使这样, 大表加索引依然耗时, 因为要扫描聚簇索引构建二级索引。

真正靠谱的做法是分步来:

- 1) 用 `pt-online-schema-change` 或 `gh-ost` 这类工具。它们原理差不多, 新建一个影子表, 加好索引后, 通过 binlog 同步原表的变更, 等追平了再切换。整个过程对业务基本无感。
- 2) 如果你自己搞, 也得模拟这个流程: 建新表、加索引、导数据、双写同步、切流、下线旧表。脏活累活多, 不如直接上工具。

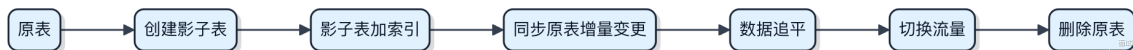
代码示例? 其实一条命令就行, 比如用 `gh-ost`:

```
gh-ost \
  --host=localhost \
  --database="test" \
  --table="large_table" \
  --alter="ADD INDEX idx_name(user_id)" \
  --execute
```

它会在后台默默干活, 你可以看进度、暂停、限速, 控制得很细。

还有一点, 别一股脑加一堆索引。先分析慢查询, 加最急需的。索引不是越多越好, 写入性能会崩。

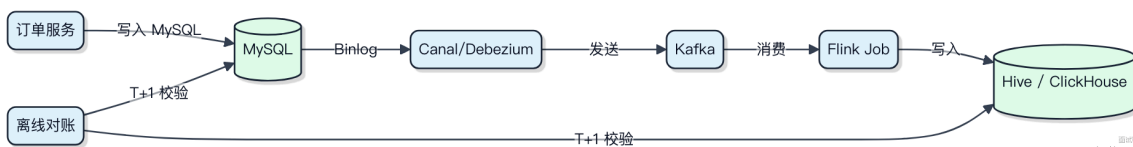
最后提醒，生产操作一定先在备库或测试环境演练一遍。这种事出一次问题，运维能记你一年。



怎么设计电商系统的订单数据同步方案（同步到数仓）？要求数据准确、性能高

电商订单数据同步到数仓，关键是在高并发写入和复杂业务逻辑下保证**最终一致性**，同时不拖垮在线业务。

- 1) 优先用 **binlog + 消息队列** 方案。MySQL 的 binlog 记录了所有变更，通过 Canal 或 Debezium 捕获，发到 Kafka。这样订单库的写操作不会额外负担，消费端可以异步处理，削峰填谷。Kafka 能扛住大流量，比如双 11 订单洪峰，积压几十万消息也能慢慢消化。
- 2) 数仓侧用 Flink 消费 Kafka 数据。Flink 做实时计算很稳，支持 exactly-once 语义，能去重、合并更新，再写入 Hive、ClickHouse 或 Iceberg 表。比如一个订单从“创建”到“支付”到“退款”，Flink 可以按 order_id 聚合，输出最终状态，避免数仓里出现多条中间态脏数据。
- 3) 加一层校验机制。每天跑一次离线对账任务，对比源库订单总数和金额与数仓是否一致。如果有偏差，触发补偿同步。这层兜底能发现极端情况下的数据丢失。



别用定时全量拉取，性能差还容易把数据库搞挂。也别让业务代码直接发消息，事务没提交就发出去，容易丢数据。

设计一个实时数据同步系统，将 MySQL 数据实时同步到数据仓库？

先说整体思路，这种系统一般走 **binlog** 搞订阅消费，不能用定时扫表，延迟高还压垮数据库。

MySQL 开启 binlog，格式设成 ROW 模式，这样每一行数据变更都会记录前后镜像。同步服务比如用 **Canal** 或 **Flink CDC** 去伪装成从库，连接 MySQL，拉取 binlog 解析出增删改事件。

这些事件通过消息队列中转，比如发到 Kafka。好处是解耦、削峰，数据仓库那边消费可以慢一点，也不怕瞬时写入压力。Kafka 一个 topic 按表或按业务域分区，保证顺序性。

数据仓库端比如是 ClickHouse、StarRocks 或者 Hive，用消费者程序读 Kafka，把数据写进去。如果要支持精确一次语义，可以用 Flink 做 checkpoint，结合 Kafka 的 offset 和目标库的事务提交做两阶段控制。

- 1) 网络隔离：同步服务和 MySQL 一般得在同一个内网，避免延迟和安全问题
- 2) DDL 处理要小心：binlog 里 DDL 是 statement 格式，解析麻烦，大字段、枚举变更更容易翻车
- 3) 初始快照：第一次全量同步不能直接扫表，要用快照读，避免锁表，Canal 支持 dump + binlog 续接

代码上核心就是注册一个 binlog 回调：

```
canalConnector.subscribe(".*\\..*");
while (true) {
    Message msg = connector.getWithoutAck(1024);
    for (RowData row : msg.getEntries()) {
        // 发送到 Kafka
    }
}
```

```

    }
    connector.ack(msg.getId());
}

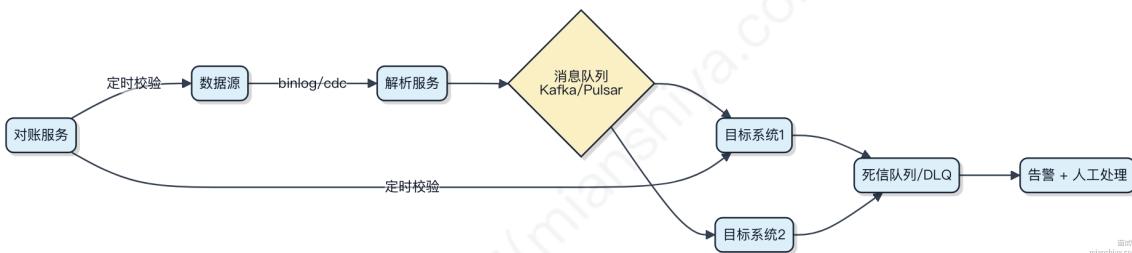
```



如何设计一个高可用的数据同步系统？需要考虑哪些容错机制？

设计高可用数据同步系统，本质是解决**数据一致性**和**系统可靠性**在异常情况下的保障问题。重点不在功能实现，而在容错兜底。

- 1) 数据传输链路必须支持断点续传。源端读取位置（比如 MySQL 的 binlog offset、Kafka 的 consumer offset）要和数据变更一起做原子性持久化。像 Canal 或 Flink CDC 都会把位点存到外部存储，重启后从最后确认点恢复，避免重复消费或丢数据。
- 2) 网络抖动和下游不可用时，要有缓冲层。直接写失败不能丢消息，一般靠消息队列扛住。比如通过 Debezium 抽取变更发到 Kafka，目标系统异步消费。这样即使 ES 或 HBase 挂了几个小时，数据也不会丢，等它恢复后慢慢重放。
- 3) 写入目标失败得有重试策略。简单重试可能压垮下游，要用指数退避 + 死信队列。比如第一次 1 秒后重试，第二次 2 秒、4 秒，最多 5 次，还不行就进 DLQ，人工介入。RocketMQ 和 Pulsar 都内置了这类机制。
- 4) 双向同步或环形同步容易出循环复制，必须打标记。比如在消息头加 sync-id，中间件看到自己发过的就丢弃。不然 $A \rightarrow B \rightarrow A$ ，数据会无限刷。
- 5) 最终一致性场景下，定期对账补数是最后一道防线。每天跑一次全量比对，发现差异就触发修复任务。滴滴的 DTS 和阿里的 Otter 都带这个能力。



使用 LIMIT OFFSET 进行分页同步时，发现数据丢失了，可能是什么原因？如何解决？

分页查询时用 `LIMIT offset, size` 看似简单，但在数据同步或实时查询场景下，**offset 变化会导致重复或丢失数据**。根本原因在于：**数据的物理顺序可能因并发写入而变动**。

比如你查第一页 `LIMIT 0, 1000`，刚查完，前面插入了一条新记录。这时第二页 `LIMIT 1000, 1000` 实际上会跳过一条旧数据——它已经被挤到前一页了，结果这条数据就“丢了”。

这种情况在使用 MySQL 的 InnoDB 表时特别常见，尤其是按非唯一索引排序时，相同 key 的记录顺序不稳定。

解决办法不是优化 offset，而是**放弃 offset**。

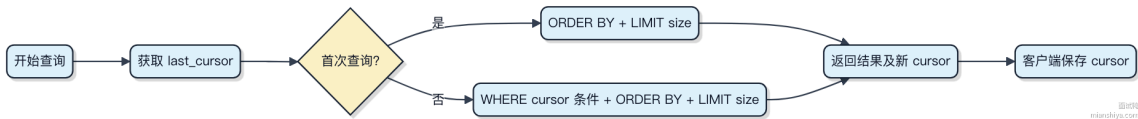
推荐用**基于游标的分页**（cursor-based pagination）。假设按时间倒序分页，维护一个上次查询的最大时间戳和主键：

```
SELECT id, ts, data
FROM t
WHERE (ts < last_seen_ts) OR (ts = last_seen_ts AND id < last_seen_id)
ORDER BY ts DESC, id DESC
LIMIT 1000;
```

每次把最后一条记录的（ts，id）作为下一次查询起点。这样每条数据只会被遍历一次，不会因为插入新数据而错位。

这种方式要求排序字段具备**单调性和唯一性组合**，通常用时间+主键就能搞定。像 Kafka、Elasticsearch 的 scroll 或 search_after 本质也是这个思路。

如果你非得用 offset，那就得加锁或者快照读，但性能压根扛不住，不现实。所以真要保证一致性，游标分页是唯一靠谱的选择。



数据同步过程中出现 OOM 内存溢出，应该如何排查和解决？

数据同步出 OOM，基本是内存里攒了太多对象没释放。排查第一步看堆栈，用 `jmap -histo:live` 或 `jstat -gc` 看老年代是不是快满了，再配合 `jstack` 看有没有线程卡在数据拉取或转换环节。

常见原因是批量拉数据没做分页，比如一次查 10 万条全 load 到 List 里，每个对象几百字节，光对象头+实例数据就吃掉上百 MB。更糟的是中间还套个转换层，比如 JSON 序列化反序列化，临时对象成倍涨。

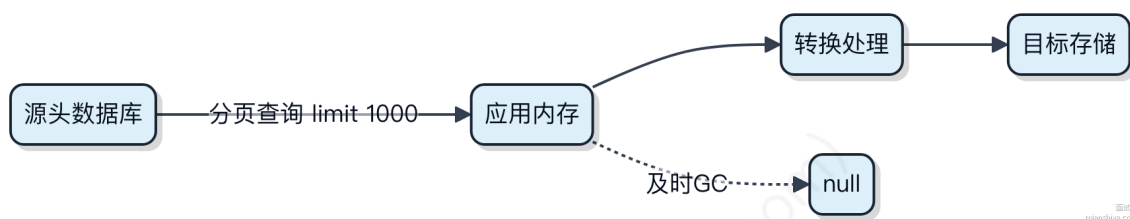
解决思路就三个：1) 控制批次大小，单次同步限制在几千条以内，MySQL 可以用 limit offset，ES 用 scroll 或 search_after 2) 流式处理代替全量加载，ResultSet 设置 fetchSize，或者用 Cursor 遍历，处理完一批马上丢掉引用 3) 必要时上磁盘缓冲，超大数据集用 MapDB 或 RocksDB 做外部存储中转

代码上注意别写这种：

```
List<Data> all = jdbcTemplate.query("SELECT * FROM huge_table", ...);
all.stream().map(...).collect(toList());
```

改成 while 循环 + 分页查，或者用 Spring Batch 的 `JdbcCursorItemReader`，它底层就是游标流式读。

如果用了 Kafka 做中转，消费者也别一口气拉 1MB 消息不提交，容易堆积。调小 `max.poll.records` 和 `fetch.min.bytes`，让每次 poll 回来几千条就行。



压根不经过大对象堆积，才是根治 OOM 的路子。

双十一期间订单量暴增，实时数据同步系统出现消息堆积，如何应对？

消息堆积本质是消费速度跟不上生产速度，双十一这种场景下，核心得从**扩容**、**降级**、**削峰**三个层面快速响应。

- 1) 先看能不能横向扩容消费者。比如用 Kafka 的话，增加 consumer 实例数能立刻提升并行度，前提是 topic 分区数够多。分区数不够就得提前做 re-balance，但线上改分区有风险，一般大促前就该规划好。
- 2) 如果资源到顶了还是扛不住，就得做降级。把非核心数据同步走异步队列延迟处理，比如用户行为日志可以暂时存本地文件，等高峰期过了再补传。核心订单数据必须保，可以用 RocketMQ 的事务消息保证最终一致。
- 3) 源头上也要削峰。前端加个限流网关，比如用 Sentinel 按用户维度限流，防住刷单类流量。也可以把部分写请求转成 MQ 异步化，数据库压力小了，整个链路更稳。

代码上注意消费幂等，重试多了容易重复消费：

```
@KafkaListener(topics = "order")
public void listen(ConsumerRecord<String, Order> record) {
    if (isProcessed(record.key())) return; // 幂等判断
    processOrder(record.value());
    markAsProcessed(record.key()); // 记录已处理
}
```

最后留一手：监控告警要配齐，消费延迟超过 5 分钟就得自动触发预案，人工介入太慢搞不定大促流量。

使用消息队列实现数据同步时，发现订单状态更新的顺序错乱了，如何解决？

消息顺序错乱在跨系统数据同步里很常见，根本原因在于消息队列默认不保证全局有序，只保证单个 producer 到单个 partition 的顺序。

要解决这个问题，关键是把**有顺序依赖的消息路由到同一个分区**。比如订单状态变更，所有关于 order_id=1001 的消息必须发到同一个 topic partition。Kafka 就靠这个机制实现局部有序，RocketMQ 也一样，关键是你得指定正确的分区键。

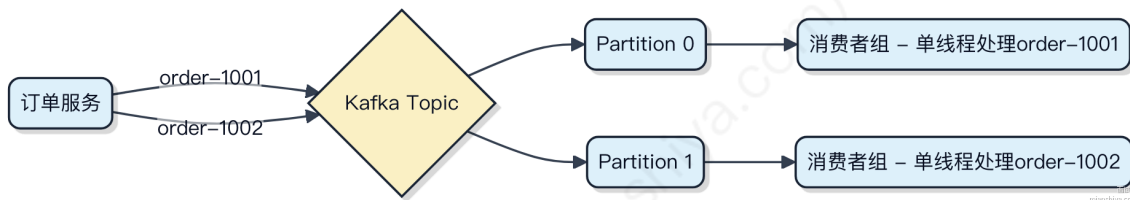
- 1) 发送端按订单 ID 做 hash 或直接作为 key，让 MQ 路由到固定 partition
- 2) 消费端确保单线程处理同一个订单的消息，避免并发导致乱序

代码上很简单，比如 Kafka 生产者：

```
ProducerRecord<String, String> record =
    new ProducerRecord<>("order-topic", "order-1001", "status=shipped");
producer.send(record);
```

只要 key 相同，Kafka 会发到同一 partition，保持写入顺序。消费者用单线程拉取该 partition 数据，自然按序处理。

如果多个订单之间也有强时序要求，那就得引入外部版本号或时间戳，在消费端做排序缓冲，但性能开销大，一般不这么做。大多数场景下，**分区内有序 + 单线程消费**就能搞定。



别指望全局有序，成本太高。抓住“**局部有序**”这个核心思路，按业务主键分流，问题就解了。

数据同步任务执行到一半失败了，如何保证数据的一致性？

数据同步任务中途失败，一致性不能靠“重试”这种粗暴方式硬扛，得从设计上解决。

1) 首先明确场景：是单表同步、跨库迁移，还是 CDC 增量同步？比如用 DataX、Flink CDC 或 Canal 做数据迁移时，策略完全不同。

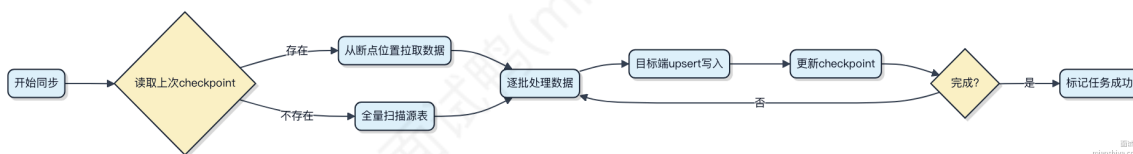
2) 核心思路是**幂等 + 状态追踪**。每次任务启动前先检查上次的执行状态，比如把任务进度写入外部存储（MySQL 的 checkpoint 表或 Redis），包含 source_position、target_offset、时间戳和状态字段。重启时根据状态决定是从头跑还是断点续传。

3) 目标端写入必须保证幂等。常见做法是：

- 按主键或业务唯一键做 upsert，而不是 insert
- 用 merge into (Oracle/PostgreSQL) 或 ON DUPLICATE KEY UPDATE (MySQL)
- 在 Flink 中使用 UpsertKafkaTableSink 或 JDBC Upsert Sink

4) 如果是批量同步，不要一次性提交 10 万条，拆成 1000 条一批，每批独立事务提交。这样即使失败，最多丢一批，恢复后也能跳过已成功批次。

5) 对于强一致性要求的场景，可以加一层校验任务，对比源库和目标库的 checksum（如使用 pt-table-checksum），发现不一致自动触发修复。



最终一致性方案最实用，强一致代价太高，一般用在金融级对账场景。

API 网关如何实现接口版本管理？如果要同时支持 v1、v2、v3 多个版本的接口，你会怎么设计？

接口版本管理本质是路由分流，把不同版本的请求精准打到对应的后端服务。关键在于**路由规则的设计**和**版本隔离策略**。

一般有三种常见方式：

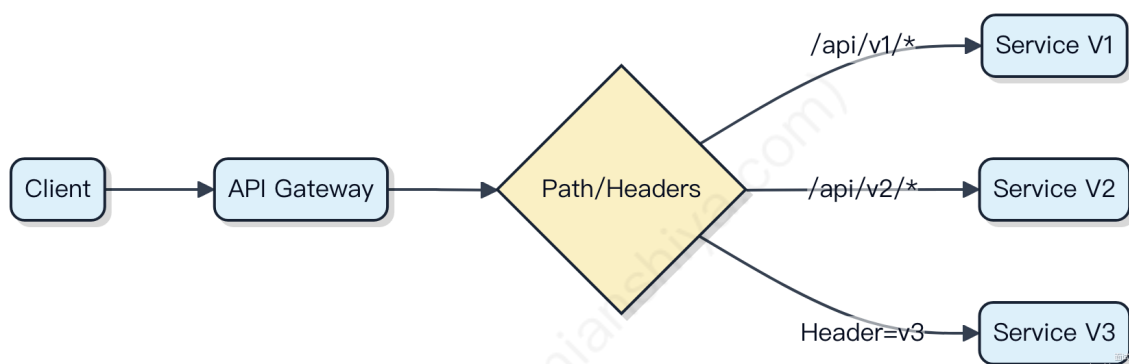
- 1) 路径路由：最直观，比如 `/api/v1/users` 走 v1 服务，`/api/v2/users` 走 v2。Nginx、Spring Cloud Gateway 都支持基于 path 的匹配转发，配置简单，前端也容易理解。
- 2) Header 路由：通过 `Accept-Version: v2` 或自定义 header 来区分。适合不想暴露版本在 URL 的场景，比如对外 SDK 接口。网关解析 header 后动态路由，对客户端透明性强。
- 3) 域名路由：`v1.api.example.com` 指向老版本集群，`v2.api.example.com` 走新架构。适合大版本彻底重构，甚至技术栈都不一样的情况，隔离最彻底。

代码层面，在 Spring Cloud Gateway 可以用 Predicate 实现路径匹配：

```
@Bean
public RouteLocator versionRoutes(RouteLocatorBuilder builder) {
    return builder.routes()
        .route(r -> r.path("/api/v1/**")
            .uri("lb://service-v1"))
        .route(r -> r.path("/api/v2/**")
            .uri("lb://service-v2"))
        .build();
}
```

灰度发布时还能结合权重，比如先把 10% 流量导到 v3，观察稳定后再全量。这时候路由规则要配合注册中心元数据，用 Nacos 或 Eureka 标记版本标签做条件路由。

版本多了之后，别忘了统一文档聚合，可以用 Swagger + Gateway 做集中 API 文档入口，按版本分组展示。



网关层如何防止接口重放攻击？如果攻击者抓取了请求包进行重放，你如何识别和拦截？

接口重放攻击的本质是同一个合法请求被多次提交，比如支付请求被重复发送。网关层要识别这种行为，关键在于判断请求的**唯一性**和**时效性**。

一般用 **时间戳 + 唯一令牌 (Nonce)** 的组合来解决。客户端发起请求时带上当前时间戳和一个一次性随机串，网关收到后先校验时间戳是否在允许的时间窗口内（比如 5 分钟），超出直接拒绝。这一步能挡住大部分延迟重放。

接着检查这个随机串是否已经处理过。可以用 Redis 记录已处理的 Nonce，设置过期时间略长于时间窗口，比如 6 分钟。每次请求到来先查 Redis，存在就说明是重放，拦截；不存在就写入，继续处理。

```
// 伪代码示意
if (Math.abs(now - request.timestamp) > 300) {
```

```

        throw new InvalidRequestException();
    }
    String nonceKey = "nonce:" + request.nonce;
    if (redis.set(nonceKey, "1", 360, NX)) { // NX+过期时间实现分布式原子写
        proceed();
    } else {
        reject();
    }
}

```

注意 Nonce 生成要足够随机，避免碰撞。像 Spring Cloud Gateway 或 Kong 这类网关，可以在全局过滤器里统一实现这套逻辑，所有流量都走一遍校验。

整个过程压根不经过业务系统，脏活累活由网关扛住。只要 Redis 不挂，几万 QPS 的去重校验完全没问题。

如果配置中心挂了，各个微服务应该如何保证可用性？你会设计什么降级策略？

配置中心挂掉时，微服务不能跟着一起瘫，得靠本地缓存和降级策略扛住。

- 1) 启动时把配置拉下来写到本地文件，比如 Apollo、Nacos 都支持持久化到 `config-cache.json` 这种文件。哪怕配置中心炸了，下次重启还能从本地加载上一次的配置，至少能起来一半。
- 2) 运行中走内存+定时刷新，客户端每隔 30 秒轮询一次，有更新就拉下来，同时更新本地文件。这样即使中间断个几十秒，内存里还有旧配置可用，服务不会立刻崩。
- 3) 加一层兜底开关，比如用 Spring Cloud 的 `@RefreshScope`，配合 `@Value("${xx:default}")` 写默认值。一旦远程拿不到，直接用默认值顶上，保证不空指针。

代码示例：

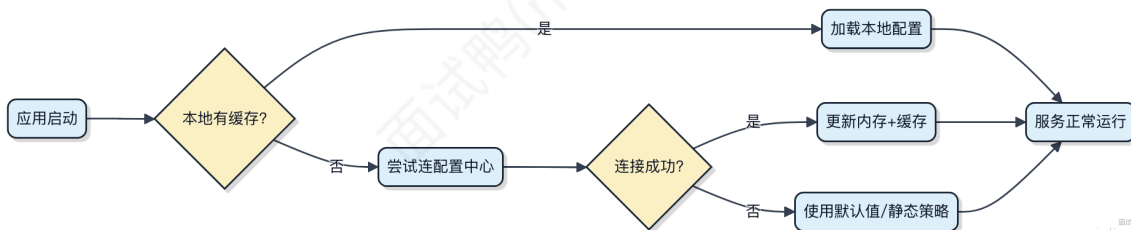
```

@Value("${timeout.ms:500}")
private int timeout;

```

这个 500 就是保命值，配置中心没了也压根不经过它。

- 4) 极端情况做熔断式降级，比如 Hystrix 或 Sentinel 拦住配置获取请求，失败超过阈值就直接返回缓存或静态策略，避免雪崩。

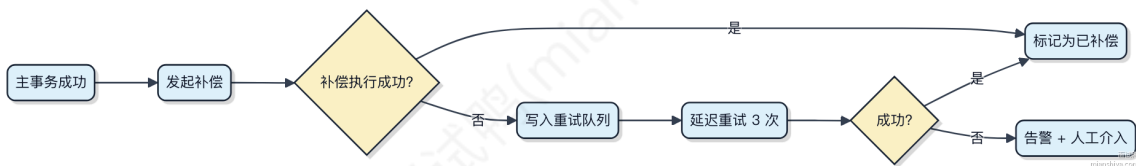


核心是 **本地缓存 + 默认值 + 熔断** 三件套，让服务在没配置的情况下也能跑起来，只是功能可能弱一点，但别死。

如果让你实现一个分布式事务的补偿机制，应该注意哪些问题？补偿失败了怎么办？

补偿机制本质是靠**幂等性**和**状态机**来兜底，不是简单回滚就完事了。分布式环境下网络抖动、任务重复、执行顺序错乱太常见，设计时得默认“补偿一定会失败”。

- 1) 每一步操作必须记录全局事务ID和分支状态，像 Seata 那样维护一个事务日志表。补偿动作不是无脑反向操作，而是先查当前状态，只有“已提交未补偿”才允许执行，避免重复扣款这类问题。
- 2) 补偿逻辑本身得异步重试，但不能死循环。一般用消息队列+延迟重试，比如 RocketMQ 的事务消息或者 Kafka 重试主题。第一次补偿失败，发个延迟消息5秒后重试，最多3次，再失败就进人工干预队列。
- 3) 系统要预留对账和修复入口。哪怕设计得再周全，极端情况还是会出问题。每天定时跑对账任务，比方说订单状态和账户余额不一致，就得能手动触发修复或导出异常单给运维。
- 4) 不要把补偿链拉得太长。超过5个服务的长事务，建议拆成多个短事务，每个阶段独立补偿，否则一环出问题整条链都卡住。



最关键的是别指望一次搞定，接受“最终一致性”，把监控、日志、重试、对账这一套基建配齐，才能扛住真实场景的冲击。

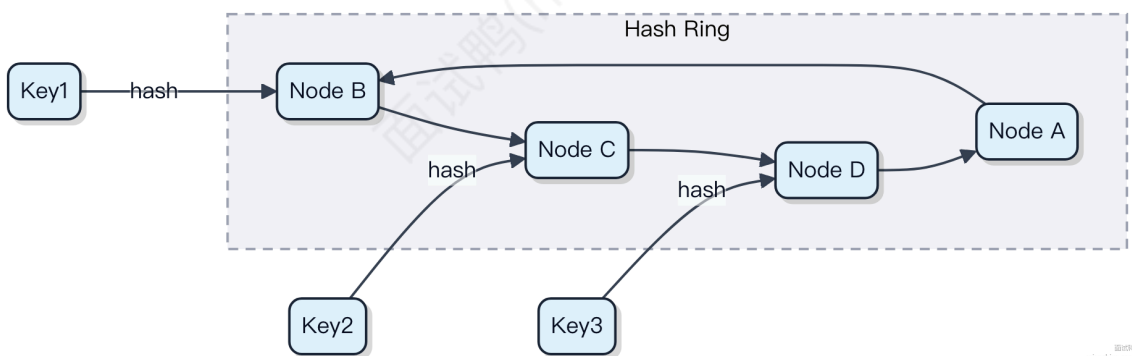
分布式缓存中，如何实现一致性 Hash 算法？相比普通 Hash，它解决了什么问题？

一致性 Hash 算法主要解决的是分布式缓存扩容缩容时，数据大规模迁移的问题。普通 Hash 做分片通常是 $\text{hash}(\text{key}) \% N$ ，当节点数 N 变化时，几乎所有 key 的映射关系都会失效，导致缓存集体失效，后端数据库直接被击穿。

一致性 Hash 把所有节点和 key 都映射到一个 0 到 $2^{32}-1$ 的环形空间上。先对节点的标识（比如 IP+端口）做哈希，放在环上。请求到来时，对 key 哈希，顺时针找到第一个大于等于该哈希值的节点，就是目标节点。

这样做的好处是，增删节点时只会影响相邻的一小段区间。比如加一个节点，只接管它前一个节点的一部分数据，其他节点基本不受影响。

为了进一步避免数据倾斜，还会引入**虚拟节点**。每个物理节点在环上生成多个虚拟节点，分散位置，让分布更均匀。Redis Cluster 虽然不用一致性 Hash，但类似的逻辑体现在它的槽位分片设计里，而 Memcached 客户端广泛采用这种算法来实现弹性伸缩。



- 1) 节点变化时，只有部分 key 需要重定位
- 2) 虚拟节点提升负载均衡效果
- 3) 实际应用如 Twemproxy、Ketama 等客户端库都基于此实现

相比普通 Hash，它让系统在扩缩容时能平滑过渡，压根不经过大规模数据迁移，扛住流量冲击。

灰度发布过程中，如何保证新旧版本数据的一致性？如果新版本有数据库结构变更怎么办？

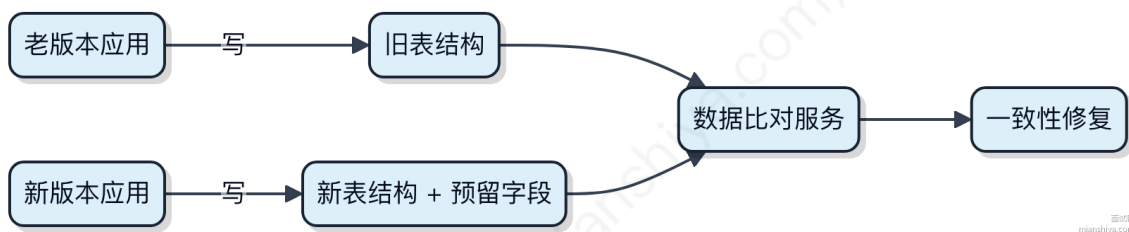
灰度发布时数据一致性最怕新旧版本对数据的理解不一致，尤其涉及数据库变更。核心是让**数据结构演进对读写流量透明**，整个过程要分阶段控制。

- 1) 先做写兼容：如果新版本要加字段，提前在老版本中以“预留字段”方式写入，但不使用。老版本忽略这些字段，新版本能读到就行。删除字段更简单，先别删，改成可空，逐步下线引用。
- 2) 读写分离过渡：用中间件比如 ShardingSphere 或自研路由，把写请求按规则分流。比如 95% 老版本写旧表，5% 新版本写带新结构的表或新字段。读的时候根据来源决定查哪边，或者双写后统一读新表。
- 3) 双写迁移阶段：新旧版本同时写两套结构，用异步任务比对并补齐差异。等数据追平，切全部写入新结构，旧逻辑只读不写。
- 4) 数据库变更走低风险方案：加索引用 Online DDL，像 MySQL 的 `ALGORITHM=INPLACE`；改列类型先加新列，程序双写，迁移完成再下线旧列。

代码层面，DAO 层抽象出版本适配器，不同版本走不同映射逻辑：

```
if (version.equals("new")) {  
    saveWithNewField(user);  
} else {  
    saveLegacy(user);  
}
```

最后靠监控和熔断兜底，一旦数据校验失败快速回滚。整个过程宁可慢，也不能让脏数据扩散。



如何实现一个单点登录（SSO）系统？如果有多个子系统，用户只需登录一次即可访问所有系统

单点登录的本质是**身份信息的跨系统共享**。用户在一处认证后，其他系统通过某种机制确认其身份，不再重复输入账号密码。

最常见方案是基于**中心化的认证服务**，比如用 OAuth2 + JWT 实现。用户首次访问子系统 A，发现未登录，跳转到统一的 SSO Server。登录成功后，Server 生成一个全局 Token（比如 JWT），并重定向回 A 系统。这个 Token 可以存 Cookie（主域名下）或前端存储。

后续访问子系统 B 时，B 会检查本地是否有有效 Token，没有就再跳 SSO Server 验证。Server 发现用户已登录，直接签发新 Token 给 B，完成无感登录。整个过程依赖一次浏览器重定向完成身份同步。

- 1) SSO Server 负责认证和签发 Token
- 2) 各子系统信任 SSO Server 的签名，只做校验不处理密码
- 3) Token 过期时间要合理设置，一般比 session 稍长

代码上, Spring Cloud Gateway + Spring Security OAuth2 是成熟组合。关键在于资源服务器配置:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(authz -> authz
        .anyRequest().authenticated()
    );
    http.oauth2ResourceServer(oauth2 -> oauth2.jwt(Customizer.withDefaults()));
    return http.build();
}
```

注意安全: Token 必须 HTTPS 传输, 防止泄露; 支持主动登出时, 需要广播通知所有子系统清理会话。



SSO 系统中, 如果认证中心挂了怎么办? 如何保证已登录用户不受影响?

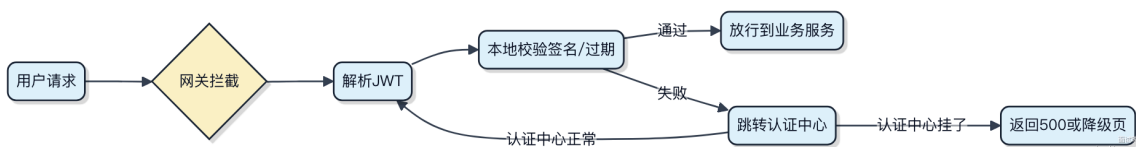
认证中心挂了, 最怕的是用户正在用的系统突然要你重新登录。其实关键在于, 会话状态不要全靠认证中心实时验证。

- 1) 用户登录后, 认证中心发的不是一纸空文, 而是带签名的 Token, 比如 JWT。服务端拿到 Token 后, 能自己校验真伪, 压根不经过认证中心。这样就算它挂了, 已登录用户访问业务系统照样能过。
- 2) 本地缓存一层用户凭证信息也挺常见。像用 Redis 存一份 session 映射, TTL 设置成和 Token 过期时间一致。认证中心出问题时, 靠这个缓存扛一阵, 避免所有请求都堵住。
- 3) 高可用层面, 认证中心本身得做集群+主备切换。用 Keepalived 搞 VIP, 或者上 Kubernetes 的 readiness probe 自动摘除异常实例。别忘了数据库也得高可用, 不然主从切不动一样歇菜。

代码上简单说就是:

```
// 校验 JWT 签名, 不依赖远程调用
if (jwtUtil.validate(token) && !jwtUtil.isExpired(token)) {
    String userId = jwtUtil.parseUserId(token);
    // 放行
}
```

图示一下用户访问流程:



重点是: 在线验证只发生在登录和 Token 刷新, 日常访问靠的是 自包含 Token + 本地校验。

如何设计一个接口签名验证机制？防止参数被篡改和重放攻击？

接口签名和防重放，本质是解决两个问题：**数据完整性和请求时效性**。

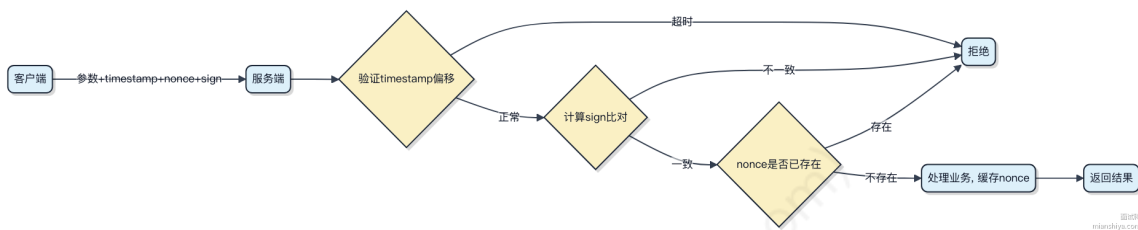
1) 签名保证参数没被改过。客户端把所有请求参数按字典序拼接，加上 secretKey 做一次 HmacSHA256 或 MD5，生成 signature 放在 header 或参数里。服务端收到后，用同样的方式重新计算，对不上就直接拒绝。注意 secretKey 绝不能走网络，一般通过平台分配 accessKey/secretKey 对，双方离线持有。

2) 防重放靠时间戳 + 临时令牌 (nonce)。每个请求带上 timestamp 和唯一 nonce，服务端校验 timestamp 与当前时间差不能超过 5 分钟，同时把 nonce 存进 Redis 做去重，有效期设为 5 分钟。只要重复的 nonce 在窗口期内出现第二次，立马判定为重放攻击。

3) 典型流程像支付宝、微信支付的 API 调用就是这么干的。你自己做系统间对接，比如订单中心对外暴露创建接口，就必须加这层保护，不然别人抓个包改个价格就能下单。

```
// 签名示例：Map 已按 key 排序
String sign = HmacUtils.hmacSha256Hex(secretKey, sortedParams.entrySet().stream()
    .map(e -> e.getKey() + "=" + e.getValue())
    .collect(Collectors.joining("&")));
```

4) Redis 去重建议用 setNx 加过期时间，别用 setEx，避免原子性丢失。如果 QPS 高，可以考虑布隆过滤器本地缓存一层已见 nonce，减少 Redis 查找压力，但要注意误判和节点间状态同步问题。



接口中的敏感数据（如身份证号、手机号）应该如何加密传输和存储？

接口传输和存储敏感数据，不能只靠前端加密，得端到端一起上措施，否则容易出事。

1) 传输层必须用 HTTPS，这是底线。TLS 加密能防止中间人窃听，所有包含身份证、手机号的请求都得走 https://，HTTP 直接干掉。

2) 敏感字段在数据库里不能明文存。一般做法是 **加密存储** 或 **脱敏存储**。如果是必须还原的场景，比如实名认证，可以用 AES 对称加密，密钥交给 KMS 管理，应用不硬编码密钥。像身份证号这种，也可以只存哈希（如 SHA-256 + salt）用于比对，但注意防彩虹表。

3) 接口返回时做逻辑脱敏。比如手机号显示成 138****1234，后四位保留或全掩码，前端不处理，由后端统一过滤。Spring 可以用 Jackson 的 @JsonSerialize(using = SensitiveDataSerializer.class) 拦一下。

```
public class SensitiveDataSerializer extends JsonSerializer<String> {
    @Override
    public void serialize(String value, JsonGenerator gen, SerializerProvider sp)
        throws IOException {
        if (value == null) return;
        if (value.length() == 11) {
```

```

        gen.writeString(value.replaceAll("(\\d{3})\\d{4}(\\d{4})", "$1****$2"));
    } else {
        gen.writeString(value);
    }
}
}
}

```

4) 日志里也别打明文。AOP 拦参数，把 idCard、phone 这类字段自动脱敏再写入 ELK。

整个链路：HTTPS 传 → 后端解密 → 业务处理 → 存数据库前加密/哈希 → 查询时按需解密或比对 → 返回前脱敏 → 日志再过滤。每一步都得卡住，少一个环节都可能泄露。

加密后的数据怎么支持模糊搜索？

加密数据要支持模糊搜索，本质是在不解密的前提下对密文做部分匹配，这事儿本身就挺拧巴。因为标准加密算法要求密文不可预测，相同明文每次加密结果都不同，直接查等于都不行，更别说模糊了。

想搞这个，得换思路。常见做法是引入 **确定性加密** 或 **可搜索加密** 机制。

- 1) 确定性加密：相同明文 + 相同密钥 → 相同密文。这样你查 "张%" 就能转成对密文前缀的匹配。但代价是泄露了数据模式，比如两个用户姓一样，攻击者能看出来。MySQL 的字段级加密如果配成 deterministic mode 就是这么干的。
- 2) 可搜索加密 (Searchable Encryption) 更高级点。比如用 **陷门函数** 生成关键词的“搜索令牌”，数据库拿着令牌去查预先构建的加密索引。像 CryptDB 就用了多层加密，把 =、<、LIKE 操作对应的字段分别加密，让查询能在特定条件下穿透。
- 3) 实际落地更多是妥协方案：把高频搜索词做哈希或加密后存个副本来建索引，或者干脆把模糊查的场景放应用层解密后处理。Elasticsearch 配合 Vault 做字段加密时，有些公司就选择只对精确查询字段加密。

代码上没统一 API，但逻辑类似：

```

String encrypted = encrypt("张三", key); // 确定性加密
String token = buildSearchToken("张%"); // 生成前缀令牌
// 查库：WHERE encrypted_name LIKE (token)

```

总之，真要端到端安全，模糊搜很难做；大多数系统在性能和安全之间折中，挑一部分字段降级保护来换取可用性。

如何防止接口被恶意刷量？除了限流，还有哪些防护手段？

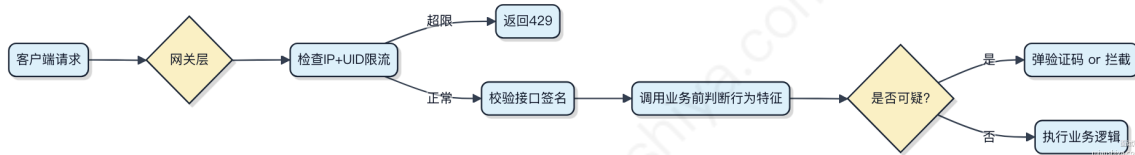
接口防刷不只是加个限流就完事，得层层设防。限流确实是最基础的一环，比如用 Redis + Lua 实现滑动窗口，每秒限制单用户最多 10 次调用，超过直接拒绝。

但攻击者会绕过，比如换 IP、模拟合法请求头，这时候就得上别的手段。

- 1) 身份识别要更细粒度。别只靠用户 ID 或 IP，IP 容易伪造。可以把设备指纹、User-Agent、Token 行为模式一起打标，用 Redis 做短期行为记忆。异常设备直接进黑名单。
- 2) 验证码是终极拦截。高风险操作比如登录、注册、下单，触发阈值后弹验证码，用极验或阿里云人机验证，机器脚本基本搞不定。
- 3) 业务逻辑加反常判断。比如一个用户 1 秒内连点 5 次提交，但每次间隔都精确到毫秒级，这人肯定不是人。后台可以标记为可疑，甚至延迟响应。

4) 接口签名防重放。所有请求带签名, timestamp + nonce 防止请求被重复发送。服务端校验时间戳偏差不能超过 5 分钟, 老请求直接丢。

5) 前端埋点辅助风控。通过 JS 埋点收集鼠标轨迹、点击时序, 上传到风控系统做模型判断。像字节的“风控中台”就这么干。



如何实现一个滑动验证码功能? 如何防止被机器识别破解?

滑动验证码的核心是生成一个带缺口的背景图和一个滑块, 用户拖动滑块到缺口位置完成验证。关键点在于**行为特征**和**服务端校验**。

1) 前端生成一张图片, 用 Canvas 画出一个不规则的缺口 (比如半圆形), 同时生成对应的滑块图。这个缺口形状每次都要随机, 不能固定模板。

2) 用户拖动滑块时, 记录鼠标移动轨迹, 包括时间戳、坐标点、加速度等。这些数据会一起提交给后端, 不是只传最终位置。

3) 后端不光看位置对不对, 还要分析行为。真实用户会有犹豫、抖动、变速, 而机器往往是匀速直线拖动。可以用简单规则判断, 比如:

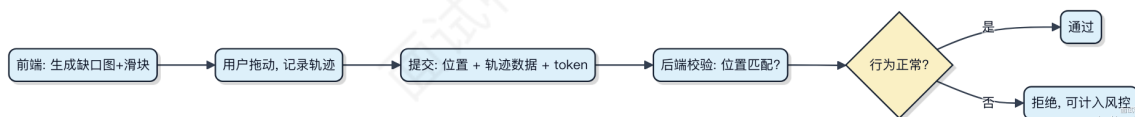
- 轨迹点少于 10 个, 大概是机器
- 平均速度超过 50px/100ms, 可疑
- 轨迹过于平滑, 不符合人类操作

4) 增加干扰项, 比如背景图加入噪声点、扭曲变形, 滑块边缘做模糊处理, 防止图像识别直接匹配。

5) 服务端每次生成的图都绑定一个一次性 token, 用完即废, 防重放。token 和轨迹数据一起存 Redis, 过期时间设为 5 分钟。

6) 可以引入风控系统, 比如同一 IP 短时间内失败超过 5 次, 触发二次验证或封禁。

整个流程里, 前端负责采集行为数据, 后端做综合决策。像极验、腾讯云滑块都是这么干的, 只是他们模型更复杂, 会用机器学习判断真人概率。



短信验证码如何防止被恶意轰炸? 如果有人不断请求发送验证码, 你会如何限制?

防止短信验证码被恶意请求, 关键在于**限流**和**增加攻击成本**, 单纯靠业务逻辑拦不住高频打码。

1) 接口层做基础限流。比如用 Redis 记录 IP + 手机号的请求频次, 同一 IP 短时间内请求超过 5 次就封 10 分钟。Spring Cloud Gateway 或 Nginx 都能做这层拦截, 压根不经过后端服务。

2) 增加前置验证步骤。用户点“发送验证码”前，先过一道图形验证码（reCAPTCHA 或阿里云滑块），机器脚本很难批量通过。这招直接卡住 80% 的自动化工具。

3) 手机号维度限制更关键。同一个手机号，1 分钟内只能发 1 次，1 小时最多 5 次，一天不超过 10 次。这些规则全用 Redis 维护计数，key 设为 `verify_code:send_count:{phone}`，带 TTL 自动清理。

4) 加点行为分析。比如新设备首次请求、短时间内切换多个手机号，这些异常行为可以触发风控模块（类似网易易盾），直接拦截或人工审核。

代码上核心是原子操作：

```
String key = "sms:count:" + phone;
Long count = redisTemplate.opsForValue().increment(key, 1);
if (count == 1) {
    redisTemplate.expire(key, 1, TimeUnit.HOURS);
}
if (count > 5) {
    throw new RuntimeException("发送太频繁");
}
```

真正扛住轰炸的，其实是把这些策略组合起来，单点防御很容易被绕开。

如何用消息队列实现延迟任务？比如订单30分钟未支付自动取消？

用消息队列做延迟任务，关键不是消息队列本身有多“聪明”，而是怎么设计流程让它能“卡住”消息到指定时间再处理。

一般有三种做法，最常用的是**延迟消息 + 死信队列**这套组合拳。比如你用 RabbitMQ，发个订单未支付的消息，设置 TTL 为 30 分钟。这期间消息压根不经过消费者，就躺在队列里“睡觉”。TTL 到了自动过期，被转发到绑定的死信交换机，再路由到真正的取消订单队列，这时候消费者才拿到消息去执行关单逻辑。

```
// 发送一条带TTL的消息，30分钟后过期进入死信队列
AMQP.BasicProperties props = new AMQP.BasicProperties.Builder()
    .expiration("1800000") // 30分钟毫秒
    .build();
channel.basicPublish("order.exchange", "order.unpaid", props, messageBodyBytes);
```

第二种是用 **RocketMQ 的原生延迟消息**，它支持 18 个延迟等级，比如 1s、5s、1m 直到 2h。你发消息时指定 `setDelayTimeLevel(6)` 就对应 30 分钟，Broker 会自动帮你调度。

第三种是**外接调度系统**，比如把任务存数据库，配合 xxl-job 每分钟扫一次要触发的任务。但这已经不算纯靠消息队列了，适合复杂定时场景。

要注意的是，RabbitMQ 的 TTL 是“最多等这么久”，如果队列积压，前面的消息没处理完，后面即使过期了也得排队，可能不准。真要高精度，得上时间轮算法，或者用 Redis ZSet 按时间戳存，定时轮询。



什么是死信队列？在什么场景下会用到？如何设计死信队列的处理机制？

消息消费失败了不能一直卡着，系统需要一种机制来处理这些“处理不了”的消息，死信队列就是干这个的。当消息在正常队列中被拒绝、超时或重试次数超过阈值后，会被自动投递到一个特殊的队列，这就是**死信队列（DLQ）**。

- 1) 消息变成死信的常见情况：消费者显式拒收（NACK）、消息过期、队列满溢、重试达到上限。比如用 RabbitMQ 时，可以配置 `x-dead-letter-exchange` 来指定死信转发规则。
- 2) 典型使用场景包括：订单支付超时后的异常处理、数据清洗时遇到格式错误的消息隔离、防止恶意消息导致消费者崩溃。像 Kafka 的重试机制配合独立 topic 做死信记录就很常见，RocketMQ 也有类似机制。
- 3) 死信队列本身不解决业务问题，它是个“缓冲池”。真正的处理机制要靠后续流程。一般做法是启动专门的 DLQ 消费者，把这些消息落库或发到监控平台，人工介入或自动分析原因。也可以结合定时任务做周期性重放尝试。
- 4) 设计时注意三点：死信消息要保留原始上下文（比如头信息），避免丢失关键数据；要有告警联动，比如接入 Prometheus + Alertmanager 实时通知；定期清理已处理的死信，防止堆积。



让你设计一个 HashMap，怎么设计？

设计一个 HashMap，重点是解决两个问题：**散列均匀和冲突处理**。

- 1) 先定结构：数组 + 链表 + 红黑树。数组是主干，每个桶存一个链表，当链表长度超过 8 且数组长度大于 64，就转成红黑树，避免查找退化到 $O(n)$ 。
- 2) 散列函数不能直接用 key 的 `hashCode()`，得二次扰动。比如 Java 里把高 16 位和低 16 位异或一下，让高位也参与计算，减少碰撞概率。
- 3) 容量必须是 2 的幂。这样取模操作可以变成位运算 `hash & (length - 1)`，快得多。扩容时也是翻倍增长，老数据要么原位置，要么高位多 1，迁移起来也快。
- 4) 负载因子默认 0.75 是个经验值。太小浪费空间，太大碰撞多。如果业务知道数据量，最好初始化时指定容量，避免频繁扩容。
- 5) key 必须不可变，尤其是 String 和 Integer 这种天然不可变类最合适。要是 key 变了，`hashCode` 结果变了，再 `get` 就找不到，数据就“丢了”。

代码核心就是 put 流程：

```
int hash = hash(key);
int index = hash & (table.length - 1);
// 走 table[index] 查找，有就覆盖，无就插入
// 链表转树、扩容都在这判断
```

常见坑是自定义对象做 key 没重写 `hashCode` 和 `equals`，结果放进去拿不出来。这两个一定得同时重写，不然逻辑乱套。

让你设计一个线程池，怎么设计？

设计线程池关键不是堆代码，而是理清楚任务生命周期和资源控制的边界。你得先想明白几个事：任务来了放哪？线程怎么创建？满了怎么办？

线程池本质是生产者-消费者模型。任务提交进来不直接开线程，而是放进一个**阻塞队列**。核心线程数（corePoolSize）就像固定班底，一直活着；超过这部分叫非核心线程，空闲时间长了就回收。

- 1) 任务来了，优先用核心线程处理
- 2) 核心都忙，任务进队列排队
- 3) 队列也满，才开非核心线程顶上
- 4) 总线程数到上限（maximumPoolSize），执行拒绝策略

拒绝策略别忽略，常见有四种：抛异常、丢任务、调用者自己处理、丢掉最老的任务重试。

```
new ThreadPoolExecutor(  
    2,           // corePoolSize  
    4,           // maxPoolSize  
    60L,         // 空闲存活时间  
    TimeUnit.SECONDS,  
    new LinkedBlockingQueue<>(100), // 队列容量  
    Executors.defaultThreadFactory(),  
    new ThreadPoolExecutor.CallerRunsPolicy() // 拒绝策略  
);
```

队列选型很重要。ArrayBlockingQueue 有界防爆内存，LinkedBlockingQueue 容量大但可能堆积过多。线上一般不用 Executors 工厂方法，容易踩坑，比如 newFixedThreadPool 用的无界队列，任务太多直接把堆撑爆。

拒绝策略和队列容量是线上调参的重点，得结合业务 TP99 和峰值流量来定。

让你设计一个消息队列，怎么设计？

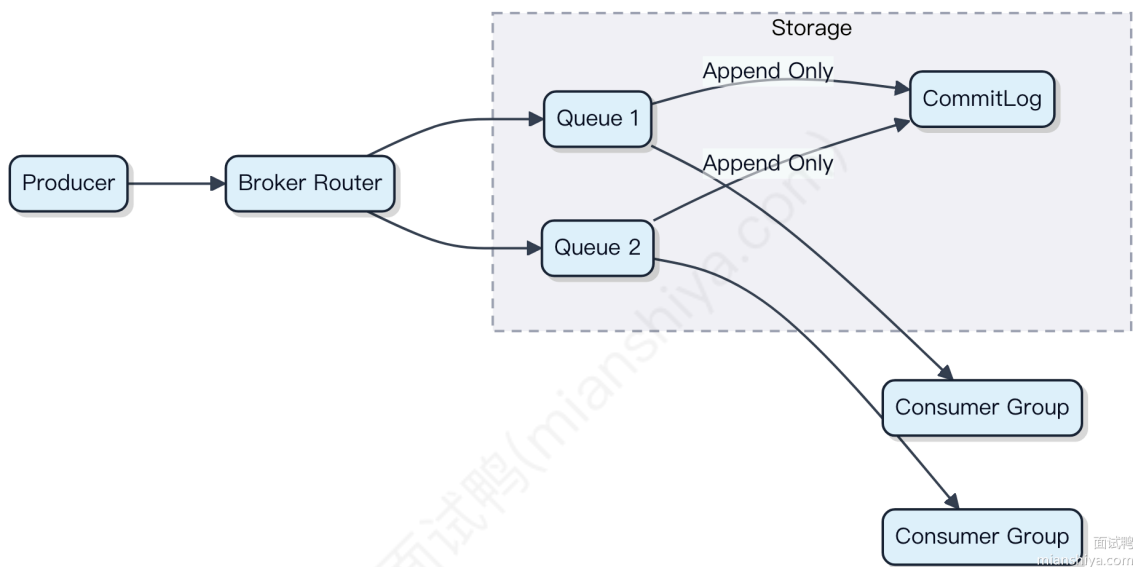
消息队列设计得先想清楚它的核心职责：收、存、发。重点是这三个环节怎么做到高吞吐、低延迟、不丢消息。

- 1) 接收端要扛住大量生产者写入，可以用多分区（Partition）机制，像 Kafka 那样把一个 topic 拆成多个 queue，分散到不同节点上并行写入。网络层用 Netty 做 Reactor 多线程模型，避免 IO 阻塞。
- 2) 存储不能直接写数据库，太慢。得先写内存缓存加 WAL（Write Ahead Log），比如 mmap 文件映射，数据先刷到 PageCache，异步批量落盘。这样写入几乎压根不经过磁盘，99% 的请求都在内存完成。
- 3) 消费模型通常用拉取（Pull）而不是推送（Push），让消费者自己控制节奏，避免被压垮。配合 offset 机制记录消费位置，支持重试和回溯。
- 4) 如果要保证不丢消息，就得在写入时做副本同步，比如双写或 Raft 协议，等至少两个节点落盘才返回成功。但这样延迟会上去，所以是否开启看业务需求。
- 5) 路由元信息可以交给外部组件管理，比如用 ZooKeeper 或 Nacos 维护 broker 和 topic 的映射关系。

代码层面，核心是一个队列数组 + 原子指针：

```
class SimpleQueue {  
    private final Message[] buffer;  
    private volatile long writePos, readPos;  
}
```

整个链路的关键路径必须无锁化设计，比如用 RingBuffer 或 CAS 操作来支撑十万级 QPS。



让你设计一个 RPC 框架，怎么设计？

先想清楚 RPC 的本质，就是让远程调用像本地调用一样简单。关键在于把“调哪儿”、“传什么”、“怎么传”、“结果怎么拿”这四件事搞定。

1) 服务暴露与发现

服务启动时得把自己的接口、IP、端口注册到注册中心，比如 ZooKeeper 或 Nacos。消费者去上面拉取可用的提供者列表，缓存下来，避免每次调用都查。

2) 动态代理拦截调用

你在代码里写 `userService.getUser(1)`，其实是个接口，没有实现。框架通过 JDK 动态代理或 ByteBuddy 生成一个代理对象，一调用就进到 `InvocationHandler`，把方法名、参数抓出来。

3) 网络通信选型

一般用 Netty 做传输层，性能好，异步非阻塞。序列化协议可以选 Protobuf、Hessian 或 JSON，Protobuf 体积小、速度快，跨语言也方便。

4) 请求唯一标识与异步等待

每个请求发出去前打上唯一 ID（比如原子递增），放进 `ConcurrentMap`，线程用 `CountDownLatch` 挂着等结果。响应回来时根据 ID 找到对应的等待线程，唤醒并返回结果。

5) 容错与负载均衡

如果节点挂了，得有重试机制，配合负载均衡策略（随机、轮询、一致性 Hash）选下一个节点。超时时间必须设，不然线程全卡死。

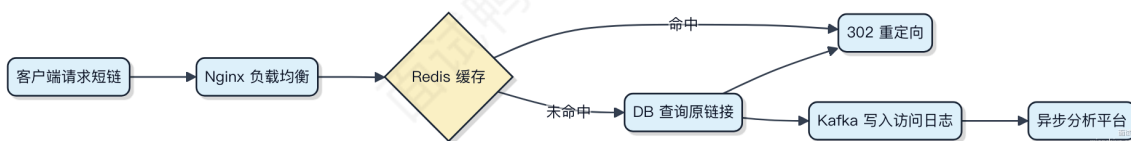


别忘了埋点监控，比如用 SkyWalking 记录调用耗时。一开始不用搞得太重，先把“能通”做到位，再加路由、限流、熔断这些脏活累活。

让你设计一个短链系统，怎么设计？

短链系统本质是把一个长 URL 映射到一个短字符串，核心挑战是怎么生成这个短字符串，并且能反向查回去。

- 1) 生成策略上，一般不用自增 ID 直接转成 62 进制 (a-z, A-Z, 0-9)，因为容易被遍历猜出。更稳妥的是用 **分布式唯一 ID** 生成，比如 Snowflake 算法生成 64 位 long，再转成 62 进制字符串，长度通常 6~8 位，够用还不重复。
- 2) 存储结构就是个 key-value，短码做主键，存原始 URL 和过期时间。读请求远多于写，所以得靠缓存扛压。Redis 是首选，先查缓存，没命中再去 MySQL 或 TiDB 查。写的时候记得双写一致，删缓存而不是更新，避免脏数据。
- 3) 跳转接口要快，302 重定向就行，响应头 Location 填原链接。注意短码大小写敏感问题，建议统一转小写处理。
- 4) 防刷和限流不能少。同一个 IP 短时间内频繁请求不同短链，可能是爬虫，可以用 Redis 记录访问频次，结合滑动窗口限流。
- 5) 扩展功能比如统计点击量，别直接改主表，会锁行。用 Kafka 把点击事件异步发出去，后端消费落 HBase 或 ClickHouse 做分析。



整个系统最脏活累活在 ID 生成和缓存一致性，其他都是常规操作。美团、微博的短链都这么搞。

假设有一个 1G 大的 HashMap，此时用户请求过来刚好触发它的扩容，会怎样？让你改造下 HashMap 的实现该怎样优化？

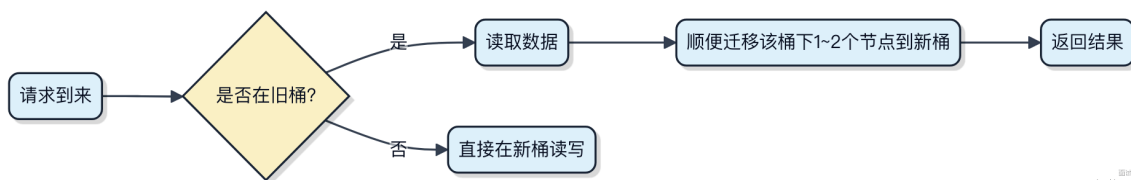
1G 的 HashMap 已经远超常规使用场景，这时候扩容带来的问题会非常致命。HashMap 扩容时需要重新计算每个元素的桶位置，也就是 rehash，并且要申请一块更大的内存空间来复制所有数据。

这个过程是**全量同步**的，意味着整个 Map 在扩容期间基本不可用，CPU 会飙高，GC 压力剧增。一次扩容可能持续几百毫秒甚至更久，对于在线服务来说就是雪崩前兆。而且 1G 数据很可能导致 Young GC 晋升失败或直接触发 Full GC，系统停摆。

再者，这么大的单机缓存其实违背了分布式缓存的设计原则。你应该考虑用 **Caffeine** 或者本地缓存+Redis 集群模式来拆分数据，避免单点压力。

如果非得优化 HashMap 本身，可以参考 **ConcurrentHashMap** 的思想：把一个大的 hash 表拆成多个 segment，扩容时只扩压力大的 segment，实现渐进式扩容。或者像 **Go map** 那样支持增量扩容，老表和新表共存，通过指针标记逐步迁移，请求落到老表就顺手搬几个 entry 过去，这样每次操作只承担一点点迁移成本，而不是一次性搞死。

另一个思路是引入**外部存储分级**，比如用 off-heap 内存（类似 Ehcache 的堆外模式）或者 mmap 文件映射，减少对 JVM 堆的压力。



这种惰性迁移机制能让扩容从“集中风暴”变成“细水长流”。

让你实现一个订单超时取消功能，怎么设计？

订单超时取消本质是解决“状态定时变迁”问题，关键在于**高效触发**和**幂等处理**。直接轮询数据库会把系统搞死，得靠异步化+延迟消息。

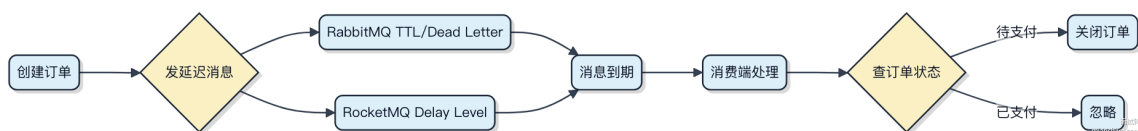
1) 最稳的方案是用 **RabbitMQ 死信队列**。订单创建时发个普通消息到 TTL 队列，设置过期时间比如 30 分钟。时间一到自动转移到 DLX 绑定的业务队列，消费者接到消息就查订单状态，如果是待支付就关单。这个机制利用中间件保底，哪怕服务重启也不丢事件。

2) 如果公司用阿里云技术栈，**RocketMQ 4.3+ 的延迟消息**更直接。支持 18 个级别延迟（1s 到 2h），下单成功后发个 level=6（默认 10min）的消息，消费端统一处理超时逻辑。注意别用早期版本，老 RocketMQ 延迟等级少得可怜。

3) 纯内存场景可以用 **Redis + Lua 脚本**。订单写入时 `zadd orders_timeout timestamp order_id`，后台起个线程每秒 `zrangebyscore` 拿到期任务。但要注意 Redis 故障可能丢数据，适合对一致性要求不高的场景。

4) 千万别用 JDK Timer 或 ScheduledExecutorService 做分布式定时任务。节点一多直接重复执行，而且挂了就没人干活。

代码上记得加分布式锁，防止超时消息被多次消费导致重复关单。处理前先判断当前状态是不是“未支付”，是的话才更新为“已关闭”，保证操作幂等。



让你实现一个分布式单例对象，如何实现？

分布式环境下不存在传统意义上的单例，因为 JVM 隔离了内存空间。要实现一个全局唯一的对象访问，本质是保证某个资源或行为在集群中被唯一控制。

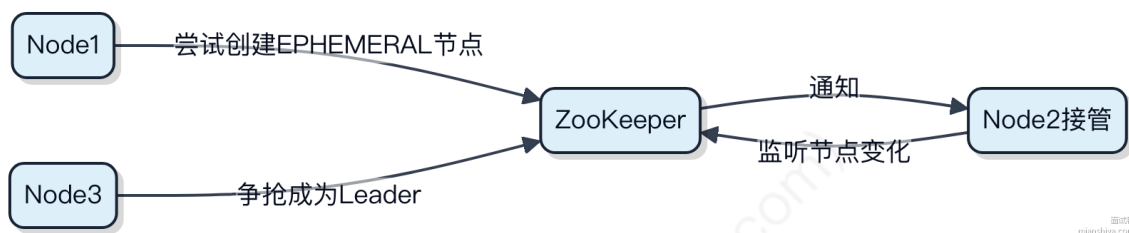
1) 最直接的思路是借助外部存储做协调，比如用 Redis 的 `SET key value NX` 命令抢锁，抢到的节点初始化对象并对外提供服务，其他节点作为备用或只读消费者。这种方式简单可靠，适合配置中心、定时任务调度这类场景。

2) ZooKeeper 也是常见选择，利用临时顺序节点和 Watcher 机制选举出主节点。像 Dubbo 的注册中心、Kafka 的控制器（Controller）都这么干。一旦主节点宕机，Watcher 会通知从节点重新竞选，实现高可用的“伪单例”。

3) 注意这种模式不是真的让对象跨进程共享，而是通过协调机制确保同一时间只有一个实例在工作。如果业务需要状态同步，还得配合消息队列比如 RocketMQ 广播状态变更。

代码上不用自己轮子，可以直接用 Curator 的 LeaderLatch 或 LeaderSelector 实现主从选主：

```
LeaderSelector selector = new LeaderSelector(client, "path", (leader) -> {
    // 成为 leader 后执行的逻辑，相当于单例初始化
    initSingletonService();
});
selector.start();
```



如果没有内存限制，如何快速、安全地将 1000 亿条数据插入到 HashMap 中？

压根不可能把 1000 亿条数据塞进一个 HashMap，哪怕内存无限也不该这么干。原因很简单，HashMap 是单机内存型数据结构，维护这么大的哈希表，光是扩容时的 rehash 就能把 CPU 搞死，更别说并发访问时的锁竞争和 GC 压力。

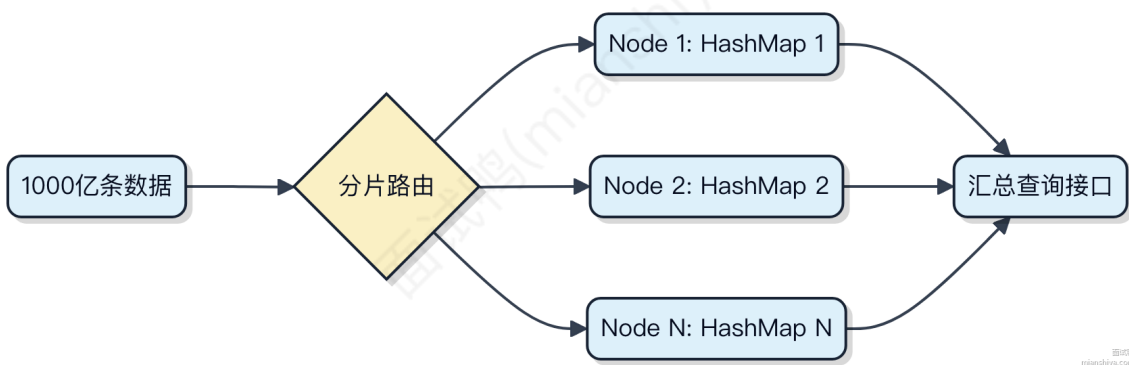
真正能扛住这种规模的，得靠分布式存储。比如用 **一致性哈希** 把数据分散到多个节点，每个节点只存一部分，像 Redis Cluster 或者 Cassandra 就是这么设计的。你要是非得“插入”，其实是在往分布式集群里写入，背后是分片、路由、持久化一套机制在兜底。

再退一步，如果只是临时处理这批数据，可以考虑外部排序或分批处理。比如用 MapReduce 模型，先把数据按 key 分段，每段本地建小 HashMap 处理，最后合并结果。Google 的 Bigtable 和 Hadoop 都是这么玩的。

代码上你不会看到 put(1000亿次)，而是类似这样的逻辑：

```
// 伪代码：分片写入
for (DataBatch batch : hugeDataSet) {
    Node node = router.route(batch.key);
    node.send(batch); // 发送到对应节点处理
}
```

整个过程根本不会在一台机器上维护全量映射，否则任何操作都会变成 $O(n)$ 的灾难。



让你设计一个文件上传系统，怎么设计？

文件上传系统看着简单，但真要扛住大流量、大文件、高并发，得把每个环节都抠细了。

先说整体思路，别一上来就写代码。用户上传一个文件，从浏览器点“提交”开始，这个请求会经过 Nginx 做前置处理，比如限制大小、防恶意请求。Nginx 可以直接存小文件到磁盘，或者用 proxy_pass 转发给后端服务。大文件的

话，前端最好做切片，比如每个 5MB，这样断点续传才有戏。

后端接收到分片后，不能直接写本地磁盘，因为服务一扩容就挂了。得统一存到对象存储里，比如 **MinIO** 或者阿里云 OSS。分片上传完，再通知服务端合并。合并这一步其实可以由对象存储自己完成，比如 OSS 的 multipart upload 接口，你只要传完所有分片，发个 complete 请求就行。

数据库里只存元信息：文件名、大小、存储路径、分片总数、MD5 校验值。MD5 很关键，上传完比对一下，能发现传输错误。如果要秒传，就先查一下这个 MD5 有没有人传过，有就直接复用。

断点续传的实现也不难。前端上传前先问服务端：“我这个文件 ID 的第 3 片传了吗？” 服务端查一下已上传的分片列表，告诉前端哪些没传，接着传就行。

整个流程其实是异步的。你可以把上传任务丢进消息队列，比如 Kafka，让下游消费去做病毒扫描、转码、缩略图生成这些脏活累活。



别忘了限流和安全。同一个用户单位时间上传次数要控，文件类型要白名单过滤，防止上传 .jsp 这种脚本。

让你设计一个分布式 ID 发号器，怎么设计？

分布式 ID 发号器的关键是生成全局唯一、趋势递增、高可用且低延迟的 ID。一般来说，会结合多种策略来满足不同场景。

1) 最简单的方案是用数据库自增主键，但单点瓶颈明显，扛不住高并发。可以改用分段预分配，比如每个应用实例从数据库一次性拿 1000 个 ID，本地自增，用完再取，减少 DB 压力。像美团的 Leaf 就是这么干的。

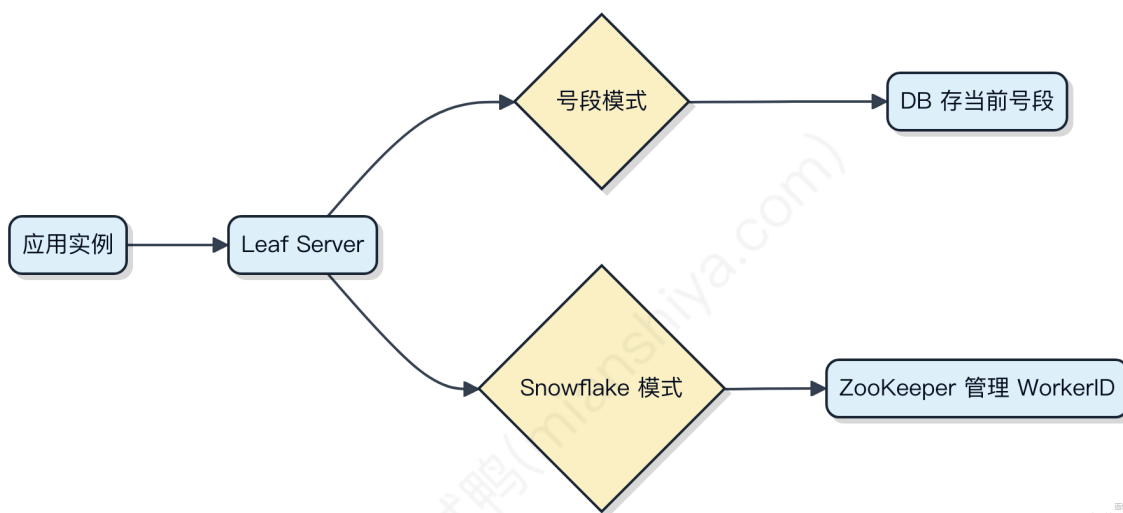
2) 纯内存生成常见的是 **雪花算法** (Snowflake)，64 位 Long 型，包含时间戳、机器 ID、序列号。时间戳保证趋势递增，机器 ID 区分节点，序列号解决毫秒内并发。压根不经过数据库，性能极好，TPS 能到几十万。

```
// 简化版 Snowflake 核心逻辑
long id = ((timestamp << 22) | (workerId << 12) | sequence);
```

但要注意时钟回拨问题，如果系统时间往回调，可能产生重复 ID。一般做法是等待或抛异常。

3) 如果对 ID 无序性有要求，可以用 UUID，但字符串太长，存储和索引效率差，不适合做数据库主键。

4) 更高阶的做法是混合型，比如 Leaf 同时支持号段模式和 Snowflake 模式，通过 ZooKeeper 或 Etcd 做 worker ID 分配和高可用协调。

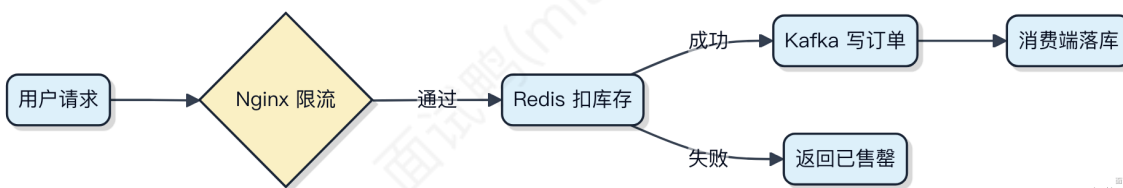


选哪种方案，得看你的 QPS、是否允许间隙、对单调性的要求。大部分业务系统用号段 + 分布式协调服务就能搞定。

如何设计一个秒杀功能？

秒杀本质是用技术手段把瞬时超大并发转化成系统能处理的量级，关键在于“削峰”和“前置”。

- 1) 流量层直接拦截无效请求。Nginx 用 Lua 做本地限流，同一用户每秒只放行 1 次请求，再多直接 503。Redis 集群预热商品库存，key 设成 `seckill:stock:{itemId}`，值为原子整数，前端按钮灰化后仍可能被绕过，所以每扣减都必须走 `DECR` 并判断非负。
- 2) 异步化减少数据库冲击。用户中签后写 Kafka，消息消费端才真正落订单到 MySQL。这样数据库只扛住最终一致性写入，峰值写入从 10w+/s 降到几千/s，MySQL 配合分库分表就能扛住。
- 3) 缓存热点数据。商品详情、活动规则全放 Redis，页面静态化，CDN 托管前端资源。下单接口不做任何远程调用，所有校验在本地完成。
- 4) 避免超卖的核心是库存扣减原子性。Redis 扣库存 + 消息队列异步下单，两个动作必须绑定成一个逻辑单元。可以用 Lua 脚本保证原子性，脚本返回成功才发 MQ。



压测要覆盖真实场景，比如用 JMeter 模拟 5w 并发抢 100 件商品。上线前做全链路压测，重点看 Redis 和 Kafka 的负载。一般系统按 3 倍冗余设计容量，但秒杀这种极端场景，前端拦截掉 90% 以上请求才是正解。

怎么分析 JVM 当前的内存占用情况？OOM 后怎么分析？

分析 JVM 内存占用，第一步是看整体分布。用 `jstat -gc <pid>` 能实时看到年轻代、老年代、元空间和 GC 执行次数、耗时，一般观察 30 秒以上，判断是否有频繁 GC 或内存缓慢增长。

接着用 `jmap -heap <pid>` 查看堆内存的详细配置和使用情况，比如初始堆、最大堆、各区实际使用量。如果发现老年代使用率长期超过 70%，就得警惕了，可能快 OOM 了。

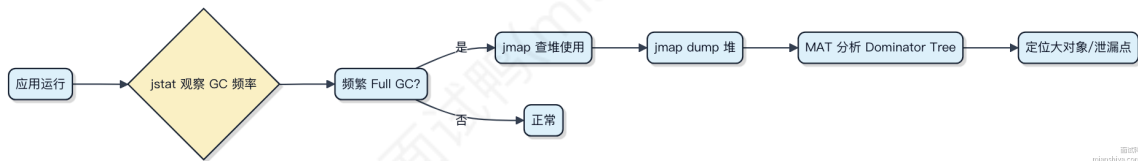
怀疑有对象泄漏时，`jmap -dump:format=b,file=heap.hprof <pid>` 抓取堆转储文件，然后用 Eclipse MAT 或 JVisualVM 分析。MAT 可以快速找出“主导集”（Dominator Tree），定位谁占了最多内存，比如某个缓存 Map 持有上百万对象。

OOM 后分析也靠 dump 文件，但得先确保加了启动参数：

```
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/path/to/dumps
```

这样 OOM 时会自动生成 dump，不然是事后诸葛也搞不定。

代码里也可以主动触发：`jcmd <pid> GC.run_finalization` 或生成 dump 辅助排查。



线上服务建议加上 `-XX:+PrintGCDetails -Xlog:gc*:file=gc.log`，GC 日志能帮你回溯 OOM 前的征兆，比如连续几次 Full GC 后内存还是下不来，基本就是泄漏了。

如果发现 Redis 内存溢出了？你会怎么做？请给出排查思路和解决方案

发现 Redis 内存溢出，别慌，先搞清楚是物理内存撑爆了，还是达到了 `maxmemory` 配置上限。这两个情况表现不一样，处理方式也不同。

1) 先看现象

如果 Redis 开始频繁写 swap，响应变慢甚至卡死，基本是物理内存耗尽。如果是开始大量返回 `OOM command not allowed`，那就是触达了 `maxmemory` 限制，内存淘汰策略开始干活了。

2) 查当前用了多少

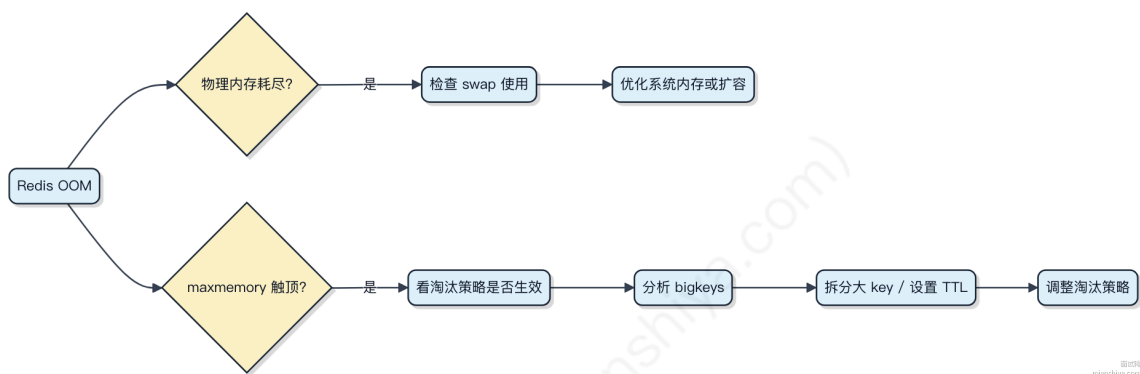
上机器跑 `info memory`，重点关注 `used_memory_rss` 和 `used_memory`。前者是系统实际占用，后者是 Redis 内部统计。差太多可能有内存碎片，比如 `mem_fragmentation_ratio > 1.5` 就得注意了。

3) 谁占的内存？

用 `redis-cli --bigkeys` 快速扫一遍，看看有没有超大 hash、zset 或 list。也可能某个 key 的过期时间没设，一直堆积，比如日志类数据往里塞。监控平台像 **Prometheus + Grafana** 搭配 Redis Exporter 能帮你盯住内存增长趋势。

4) 解决方案分三层

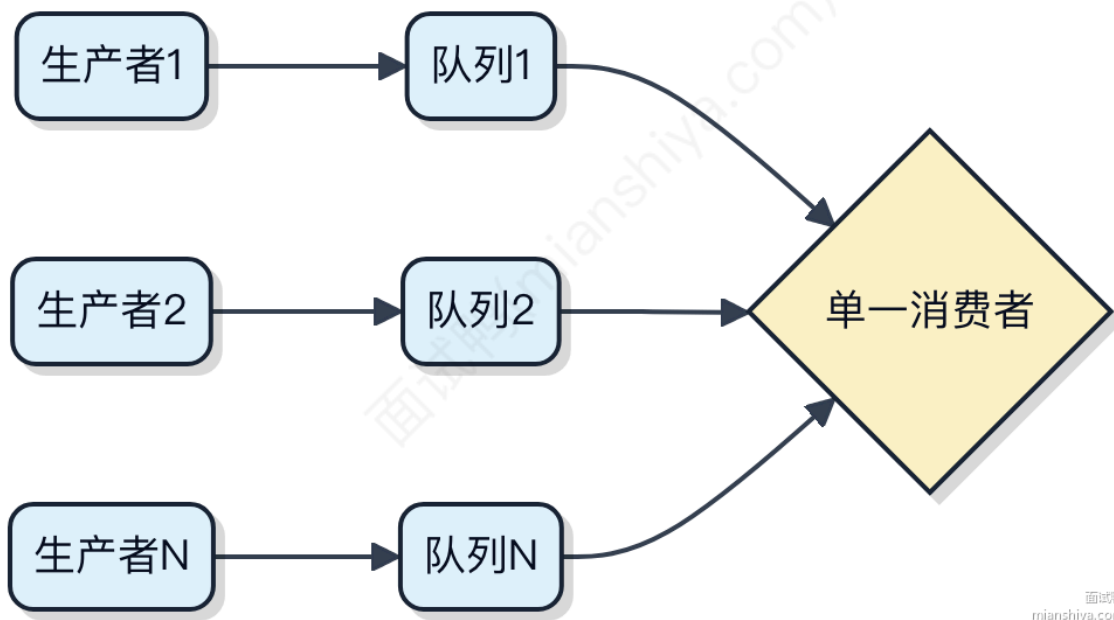
- 紧急：调大 `maxmemory`（临时顶住），或者切到新实例做数据迁移。
- 治本：开启 **LFU/LRU 淘汰策略**，比如 `allkeys-lfu`，让冷数据自动滚蛋。
- 长期：数据结构优化，大 key 拆分，加二级缓存（比如本地 Caffeine），或者把部分数据往 **Codis** 或 **Tendis** 这类分布式方案挪。



两百万个生产者发送消息，仅一个消费者，如何高效设计锁？

这种场景下，锁的设计压根不是重点，因为两百万生产者竞争同一把锁，系统早就崩了。关键在于**避免让所有生产者直接抢资源**，得靠无锁化设计和队列缓冲来扛住。

- 1) 生产者端必须做分片。把两百万生产者按某种规则（比如 hash 或轮询）分散到多个独立的队列里，每个队列自己处理自己的消息。像 Kafka 就是靠 Partition 分摊压力，RocketMQ 用 MessageQueue 做类似的事。
- 2) 队列内部用无锁结构。比如使用 Disruptor 框架里的环形缓冲区，或者 ConcurrentLinkedQueue 这种基于 CAS 的线程安全队列。这些结构能让多个生产者并发写入不阻塞，性能比 synchronized 或 ReentrantLock 高好几个数量级。
- 3) 消费者从所有队列里拉数据。可以单线程轮询多个队列，也可以用反应式模型统一消费。重点是消费速度要跟上，否则队列会堆积。如果实在跟不上，就得考虑扩容消费者，哪怕题目说“一个消费者”，也得评估是否真能搞不定。



其实大多数高并发消息系统都不靠“锁”解决问题，而是把锁的影响降到最低。Disruptor 甚至能做到无锁发布，靠的是内存屏障 + 序号标记。所以这题的破局点不在锁怎么设计，而在**如何让锁压根不需要出现**。

如何在附近 100w 的商户中，快速找到离你最近的 5 家商户？

要解决在 100 万商户中快速找出最近 5 家的问题，关键在于**空间索引**和**距离计算优化**。直接遍历全量数据算距离肯定搞不定，响应时间会直接崩。

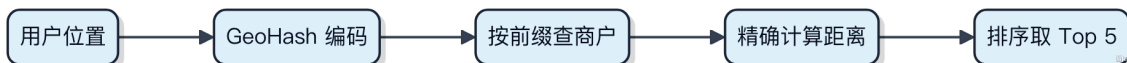
一般做法是先用 **GeoHash** 把二维经纬度编码成一维字符串，利用前缀匹配缩小搜索范围。比如你当前坐标生成的 GeoHash 是 `wx4g0`，那么系统优先查前缀为 `wx4g` 的商户，这些大概率就在你附近。这步能过滤掉 90% 以上的无效数据。

接着在筛选出的小范围内，用球面距离公式（比如 Haversine）精确计算实际距离，排序取 Top 5。这个计算可以放在 Redis 或数据库里完成，Redis 的 `GEOSEARCH` 命令原生支持这类操作。

```
// Redis 示例：查找指定坐标附近 10km 内最多 5 家商户
List<String> nearby = jedis.geosearch("merchants",
    new GeoCoordinate(116.4, 39.9),
    new GeoRadiusArgs().count(5).sortAscending());
```

如果数据量更大或者需要更复杂的空间分析，可以用 **PostGIS** 或者 **Elasticsearch**，它们对地理空间查询的支持更完整。Elasticsearch 的 `geo_distance` 查询配合 `size=5` 就能直接返回结果。

整个流程其实是“粗筛 + 精算”两步走，GeoHash 负责扛住第一波流量冲击，脏活累活交给空间数据库或搜索引擎兜底。



MySQL 中如果我 `select * from` 一个有 1000 万行的表，内存会飙升么？

这个问题得看你怎么执行这条 SQL。很多人以为 `SELECT * FROM 表` 会把 1000 万行数据全捞到内存里，其实不一定。

MySQL 的查询结果默认是**流式返回**的。Server 端一边查存储引擎，一边往客户端发数据，不会一次性把所有结果塞进内存。所以服务端的内存压力其实是可控的，关键在于客户端怎么处理。

但问题出在客户端。比如你用的是 MySQL Workbench、Navicat 这类图形工具，默认会把整个结果集全拉到本地内存里渲染表格。1000 万行宽表可能直接把你的机器干趴，内存飙升的就是你本地这头。

程序里也一样。Java 用 JDBC 的 `Statement.executeQuery()`，默认行为是把结果全 fetch 回来存到 `ResultSet` 里。除非你设置了 `fetchSize(Integer.MIN_VALUE)`（MySQL 驱动特殊支持），让它用游标模式流式读取。

所以真正要关注的是：**结果集是否缓冲**。

- 1) 如果你只是想数一数，写成 `SELECT COUNT(*) FROM huge_table`，这个当然没问题，聚合后就一行。
- 2) 如果是导出数据、做迁移，建议加主键范围分页，或者用 `WHERE id > ? LIMIT 10000` 一批批拿。
- 3) 别在生产连上直接敲 `SELECT * FROM 超大表`，尤其是图形化工具，压根不经过脑子。

代码示例：

```
stmt.setFetchSize(Integer.MIN_VALUE); // MySQL 游标式读取
ResultSet rs = stmt.executeQuery("SELECT * FROM big_table");
```

```
while (rs.next()) {  
    // 流式处理，内存不爆炸  
}
```

让你设计一个 API 网关，需要考虑哪些核心功能？如何实现动态路由和限流？

API 网关本质是所有请求的统一入口，得先把核心职责理清楚。路由、鉴权、限流、监控这些是基本盘，动态配置能力则是灵活性的关键。

- 1) 动态路由的核心在于把路由规则从代码里剥离出来。一般做法是用 Nacos 或 ZooKeeper 存储路由表，比如路径 `/user/**` 转发到 `user-service` 的 8080 端口。网关启动时拉取配置，监听变更事件实时更新内存中的路由表。Spring Cloud Gateway 的 `RouteDefinitionWriter` 就是干这个的，配合 Redis 或数据库也能做持久化。
- 2) 限流要防突发流量打垮后端。常见的是 **令牌桶算法**，Guava 的 `RateLimiter` 是单机版，生产环境通常用 Redis + Lua 实现分布式限流。比如按用户 ID 或 IP 做 key，每秒放 100 个令牌，超了就返回 429。Sentinel 和 Hystrix 也内置了限流策略，集成起来省事。
- 3) 路由和限流规则最好能热更新。比如通过 OpenAPI 提供管理接口，改完配置推送到配置中心，网关监听到变化后 reload。整个过程不需要重启服务。



压根不经过业务服务的逻辑，都应该在网关层兜住。像认证可以放 JWT 校验，日志打点记录响应时间，这些脏活累活让网关扛住，后端才能轻装上阵。

微服务架构中，如何设计一个配置中心？配置变更后如何实时通知各个服务？

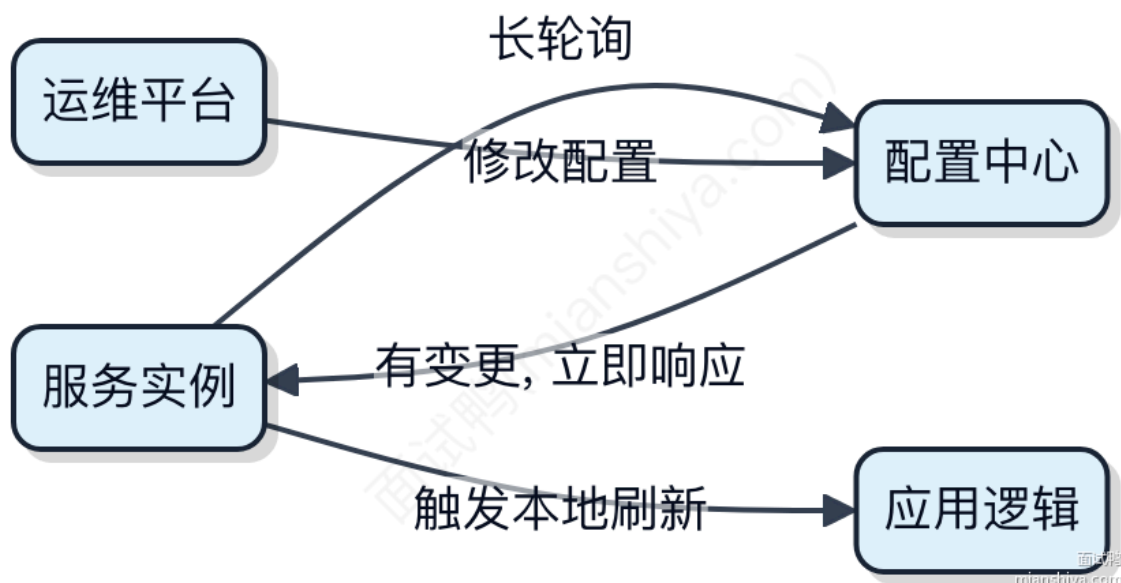
配置中心本质是解决分布式环境下配置统一管理的问题。你得先想清楚，服务启动要拉配置，运行中配置变了还得实时感知，这两件事一个都不能少。

- 1) 配置存储一般用 **ZooKeeper**、**etcd** 或 **Nacos** 这类支持监听的中间件。比如 Nacos 用 MySQL 做持久化，内存里维护一份最新配置，读写分离扛住高并发。
- 2) 服务启动时从配置中心拉取全量配置，这步通常是 HTTP 请求，简单直接。关键在运行时更新——不能轮询，延迟大还费资源。主流做法是长轮询（long polling），客户端挂住请求几十秒，服务端有变更立刻回，没变更等快超时再返回空。像 Nacos 客户端每 30 秒发一次长轮询，服务端 hold 住，一旦某个配置被改，立马推给所有挂着的客户端。

```
// 伪代码示意：长轮询监听  
configService.addListener("app.yml", config -> {  
    System.out.println("新配置来了: " + config);  
    reloadConfig(config);  
});
```

- 3) 客户端收到变更后，要能自动刷新 Bean 或配置项。Spring Cloud Config 配合 `@RefreshScope` 注解就能做到，改配置不重启服务。
- 4) 最后要考虑灰度发布。别一改配置几百个实例全炸了。可以按 namespace 或标签做分组推送，先打 10% 实例验证，没问题再全量。

整个链路其实是“集中存储 + 长轮询 + 本地缓存”。客户端本地也存一份，即使配置中心挂了也能降级使用。



电商系统中，用户下单后如何保证订单、库存、支付三个服务的数据一致性？请设计一个分布式事务方案

订单创建、库存扣减、支付状态更新跨了三个服务，任何一个环节出问题都会导致数据不一致。比如用户付完款，结果库存没扣，那就亏了。这种场景不能靠本地事务搞定，得用分布式事务方案。

1) 首选 **Seata 的 AT 模式**，对业务侵入小。你只需要在下单方法上加个 `@GlobalTransactional` 注解，Seata 会自动记录订单、库存、支付表的 before/after image，生成回滚日志。如果支付调用失败，它能自动根据日志反向补偿，把订单和库存状态回退。

2) 如果要求最终一致性，可以用 **可靠消息 + 最终一致性**。下单成功后发一条 MQ 消息给库存服务，库存消费成功再扣减，然后发下一条消息给支付服务。每个步骤都落库状态，配合定时对账任务兜底。RocketMQ 的事务消息正好支持 half message 和回调机制，能保证消息不丢。

3) 极端情况比如网络超时，不确定支付是否成功，这时候不能直接回滚，得走 **TCC 模式**：Try 阶段预占库存和支付额度，Confirm 真正扣减，Cancel 释放资源。虽然要写三个方法，但控制力更强，适合高并发资金场景。

TCC 性能最好但开发成本高，AT 模式适合快速落地，消息方案适合异步场景。选哪个，得看你们系统的容忍度和复杂度。

如何设计一个灰度发布系统？如何控制流量逐步切换到新版本？

灰度发布系统的核心目标是控制新版本的暴露范围，用最小风险验证功能正确性。关键在于 **流量路由** 和 **动态策略管理**。

1) 流量打标

请求进来时，根据 IP、用户 ID、设备指纹等提取特征，生成一个稳定的标签。比如按用户 ID 取模决定是否进入灰度池。这个标签决定了它后续走哪个服务实例。

2) 路由规则下发

用配置中心（比如 Nacos 或 Apollo）维护灰度规则。规则可以是“5% 的用户流量切到 v2”，也可以是“特定用户组全量切”。网关或客户端侧的负载均衡组件监听配置变化，实时更新路由逻辑。

3) 网关/SDK 执行分流

在 API 网关层做统一拦截最常见。比如基于 Spring Cloud Gateway 写个 GlobalFilter，解析请求标签，结合当前灰度比例，决定转发到 service-v1 还是 service-v2。如果用了 Service Mesh，也可以通过 Istio 的 VirtualService 做权重分发。

4) 渐进式放量

从 1% 开始，观察监控指标（错误率、延迟、业务埋点）。没问题就手动或自动提升到 5% → 20% → 100%。整个过程要配合 CI/CD 流水线，支持一键回滚。

```
// 示例：简单按用户ID取模分流
String uid = request.getHeader("X-User-ID");
int hash = Math.abs(uid.hashCode()) % 100;
String version = hash < grayRatio ? "v2" : "v1";
```

整个链路必须保证上下游版本兼容，避免 v2 调 v1 时协议不一致。日志和链路追踪也要带上版本标记，方便排查。



本资源来自面试鸭：<https://www.mianshiya.com>

推荐更多免费学编程资源：

1. 编程导航学习网站：[学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库：[开始刷题，面试遇原题!](#)
3. 精选简历模板大全：[1 分钟搞定简历!](#)
4. AI 资源导航网站：[获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试：[随时随地提升面试能力](#)